

## オペレーティングシステム 第1章 オペレーティングシステムとは

<https://github.com/tctsigemura/OSTextBook>

オペレーティングシステムとは

1 / 14

## オペレーティングシステムとは

Windows, macOS, Linux, FreeBSD, Android, iOS などのこと。

1. カーネル (OS の本体)
2. ライブラリ (プログラムが使用するサブルーチン, DLL)
3. ユーザインタフェース (GUI, CLI)
4. ユーティリティソフトウェア (ファイル操作, 時計, シェル, システム管理 ...)
5. プログラム開発環境 (エディタ, コンパイラ, アセンブラ, リンカ, インタプリタ)

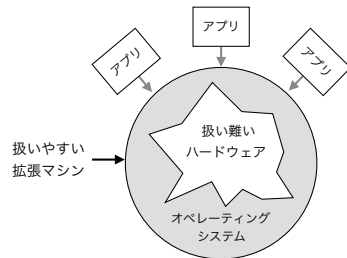
- 広義では上に列挙した全てのこと。
- 狭義では「カーネル」だけを指す。

この科目では、主に「カーネル」の仕組みを学ぶ。

オペレーティングシステムとは

2 / 14

## 拡張マシンとしてのオペレーティングシステム



- 抽象化 (ファイル, プロセス)
- 拡張マシン (システムコール)

オペレーティングシステムとは

3 / 14

## ハードウェア管理プログラムとしてのオペレーティングシステム



- 仮想化
- 時分割多重による仮想化
- 空間分割多重による仮想化

オペレーティングシステムとは

4 / 14

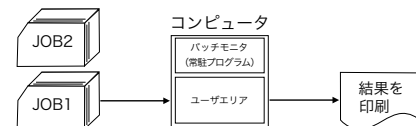
## 第1世代

- 真空管
- オペレーティングシステムなし
- 巨大 TeC 状態?

オペレーティングシステムとは

5 / 14

## 第2世代 (バッチ処理, その1)

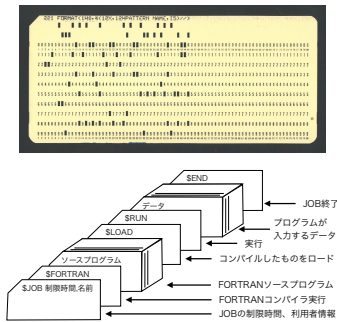


- メインフレーム
- バッチ, バッチ処理
- バッチモニタ
- JOB 制御言語 (JCL : Job Control Language)
- 実行モード
- システムコール
- 記憶保護

オペレーティングシステムとは

6 / 14

## 第2世代 (バッチ処理, その2)



- 紙カード
- JOB の構成

オペレーティングシステムとは

7 / 14

## 第3世代 (メインフレーム)



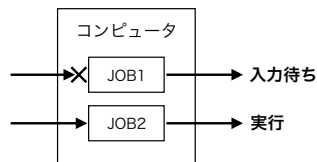
フォルクスワーゲンで使われる System/360  
ウィキメディア / Bundesarchiv, B 145 Bild-F038812-0014 / Schaack, Lothar / CC-BY-SA 3.0 de

- IBM System/360, シリーズ化
- 仮想記憶

オペレーティングシステムとは

8 / 14

## 第3世代 (マルチプログラミング)



- OS/360
- MULTICS (MULTIplexed Information and Computing Service)
- UNIX (ユニックス)
- Dynabook

オペレーティングシステムとは

9 / 14

## 第3世代 (タイムシェアリング : TSS)

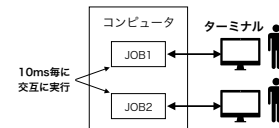


写真: <http://commons.wikimedia.org/wiki/File:Teletype0925Terminal.jpg> (バブリアドメイン)

- TSS (Time Sharing System)
- ターミナル

オペレーティングシステムとは

10 / 14

## 次の世代に向けて (Dynabook)

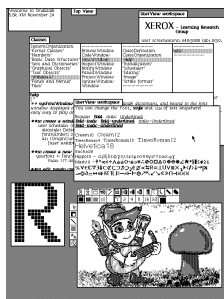


写真: ウィキメディア / SUMIM.ST /  
Alto や NoteTaker で動作したアラン・ケイ達の暫定 Dynabook 環境 (Smalltalk-76, 同-78 の頃) / CC-BY-SA 4.0

- GUI, マウス, パーソナルコンピュータ

オペレーティングシステムとは

11 / 14

## IBM PC



写真: IBM PC  
ウィキメディア / Bundesarchiv, B 145 Bild-F077948-0006 / Engelbert Reineke / CC-BY-SA 3.0 de

- 16bit, 32bit, 64bit
- IBM PC (MS-DOS, Windows)
- 互換機

オペレーティングシステムとは

12 / 14

## Apple Macintosh



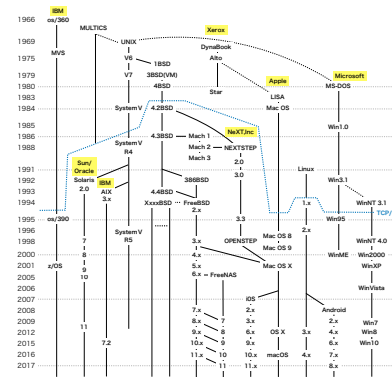
写真：初代 Macintosh  
 ウィキメディア / w.User:Gm wnr / File:Macintosh 128k transparency.png / GFDL

- 16bit, 32bit, 64bit
- Macintosh (Mac OS, Mac OS X, OS X, macOS)

オペレーティングシステムとは

13 / 14

## オペレーティングシステムの系統図



オペレーティングシステムとは

14 / 14

# オペレーティングシステム

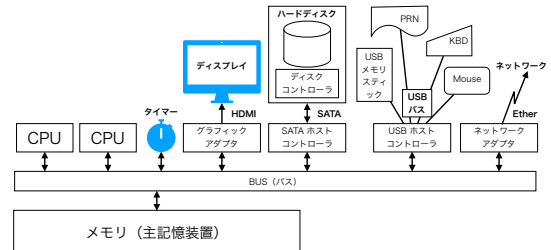
## 第2章 前提知識

<https://github.com/tctsigemura/OSTextBook>

### 前提知識

1 / 18

## ハードウェア構成

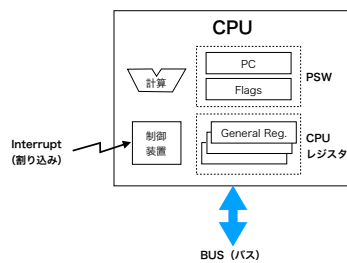


- SMP (Symmetric Multiprocessing)
- CPU, メモリ, タイマー, アダプタ, コントローラ, バス
- DMA (Direct Memory Access), I/O 完了割込み

## 前提知識

2 / 18

## CPUの構成

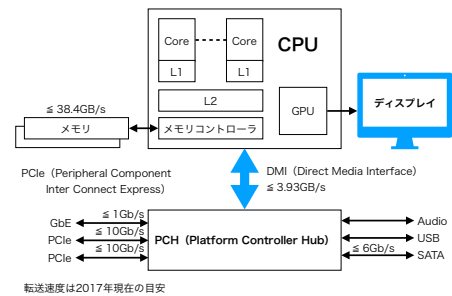


- PSW (Program Status Word)
- CPU レジスタ
- 割り込み (Interrupt)

### 前提知識

3 / 18

## デスクトップ・パーソナルコンピュータ



転送速度は2017年現在の目安

- CPU
- コア (Core)

## 前提知識

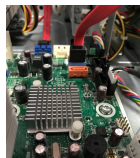
4 / 18

## デスクトップ・パーソナルコンピュータの内部

## PC内部の例

マザーボード  
CPU, メモリ, PCH,  
PCIe, USB, GbE(Net),  
HDMI, SATA  
電源装置

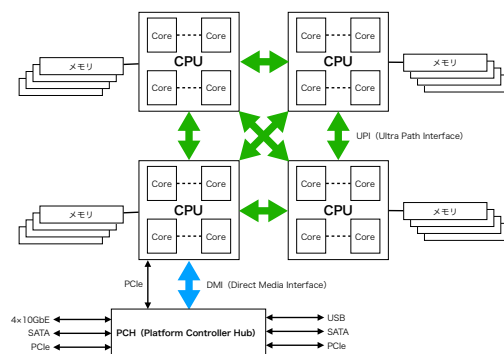
ストレージ  
HDD(SATA),  
DVD(SATA),  
カードリーダー(USB)



### 前提知識

5 / 18

## サーバコンピュータ



### 前提知識

6 / 18

## 割込み

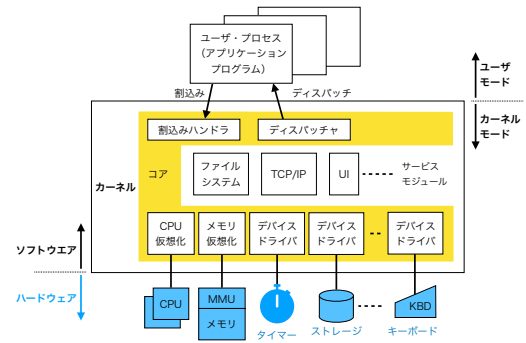
ユーザ・プロセスからカーネルに切り換わる唯一の方法

1. I/O 完了・タイマー  
ホストコントローラ、ネットワークアダプタ、タイマー等のコマンド完了など
2. システムコール  
ユーザ／プロセスが SVC (Supervisor Call) 命令を実行
3. 保護違反  
特権違反、メモリ保護違反
4. ソフトウェアのエラー  
オーバーフロー、ゼロ除算など
5. ハードウェアのエラー  
故障、電源異常

前提知識

7 / 18

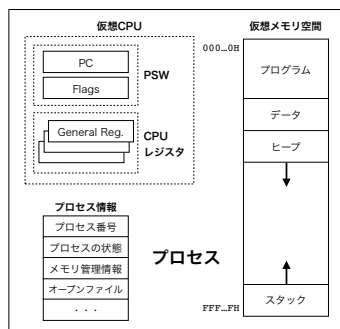
## オペレーティングシステムの構造



前提知識

8 / 18

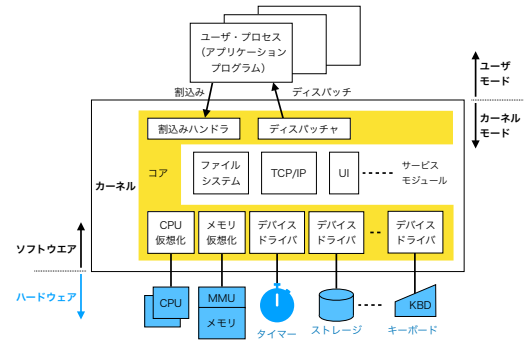
## プロセスの構造



前提知識

9 / 18

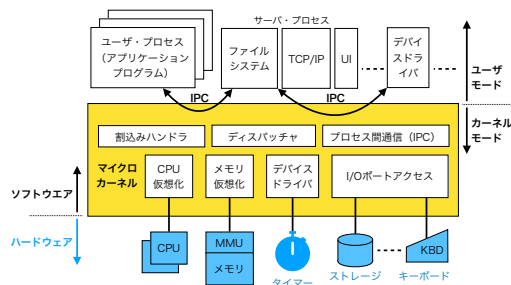
## 単層カーネル (モノリシック・カーネル)



前提知識

10 / 18

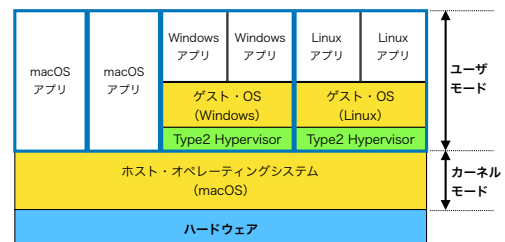
## マイクロカーネル (micro-kernel)



前提知識

11 / 18

## Type 2 ハイパーバイザ

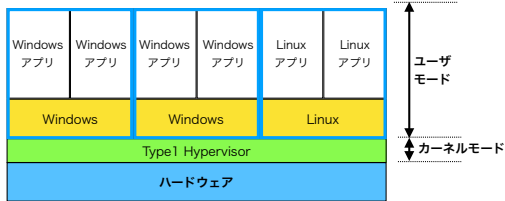


- ホスト・オペレーティングシステム
- ゲスト・オペレーティングシステム
- VMware Workstation, VirtualBox

前提知識

12 / 18

## Type 1 ハイパーバイザ



- メインフレーム：IBM z/VM
- PC サーバ：VMware vSphere, Xen, Hyper-V

前提知識

13 / 18

## 仮想アプライアンス

- 仮想マシンのディスクイメージの配布
- ソフトウェアの新しい流通手法

前提知識

14 / 18

## 実装例

## 第18章 TaC (Tokuyama Advanced educational Computer)

前提知識

15 / 18

## TeC7 と TaC



(a) TeC7 の写真



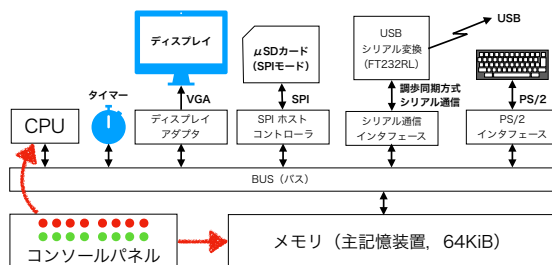
(b) TaC としての使用例

TeC7 は、TacOS を書き込んだマイクロ SD カードを装着すると、簡単な PC (TaC) として使用できる。

前提知識

16 / 18

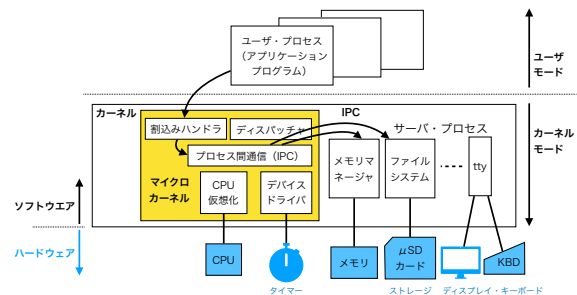
## TaC のハードウェア



前提知識

17 / 18

## TacOS の構造



前提知識

18 / 18

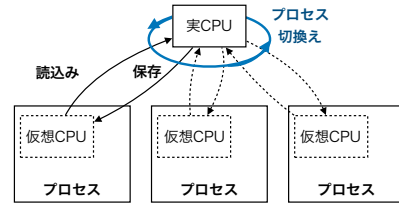
## オペレーティングシステム 第3章 CPUの仮想化

<https://github.com/tctsigemura/OSTextBook>

CPUの仮想化

1 / 35

### 時分割多重によるCPUの仮想化

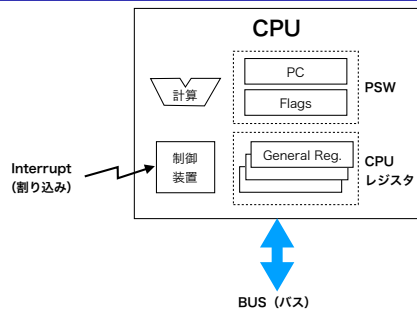


- 時分割多重：CPUが実行するプロセスを次々切替える。
- コンテキストスイッチ：CPUが実行するプロセスを切替えること。
- ディスパッチ：プロセスにCPUを割り付ける。(実行開始)
- ディスパッチャ：ディスパッチするプログラムのこと。

CPUの仮想化

2 / 35

### CPUの構造 (参考)

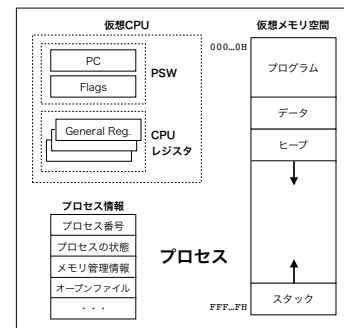


- コンテキスト = PSW + CPU レジスタ
- コンテキストを保存・ロードして次のプロセスに
- コンテキストスイッチ

CPUの仮想化

3 / 35

### プロセスの構造 (参考)

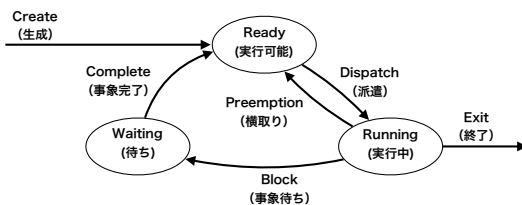


- 仮想CPUにコンテキストを保存

CPUの仮想化

4 / 35

### プロセスの状態遷移



- 基本的な三つの状態
- 六つの状態遷移

CPUの仮想化

5 / 35

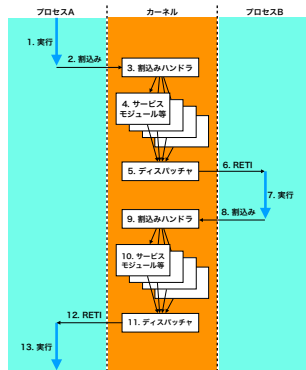
### プロセス切換えの原因

- イベント
  - プロセス自ら「システムコールを発行する」Blockする
  - 他のプロセスから「干渉を受け」Blockする
  - 他のプロセスから「干渉を受け」Preemptionする
  - 他のプロセスから「干渉を受け」Completeする
  - I/O完了やタイマ完了のイベントによりCompleteする
- タイムスライシング
  - クオンタムタイムを使い切ったプロセスはPreemptionする

CPUの仮想化

6 / 35

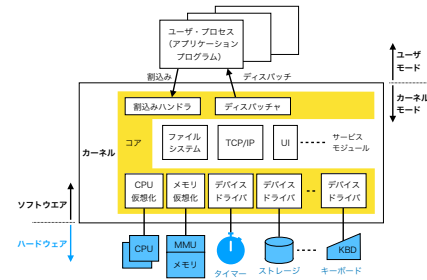
## プロセスの切換え手順



CPU の仮想化

7 / 35

## オペレーティングシステムの構造 (参考)

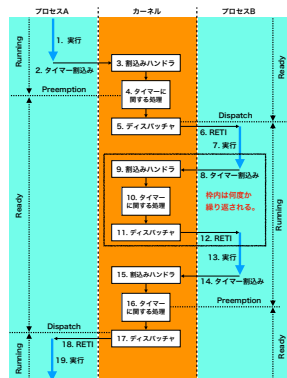


- 割込みハンドラ
- サービスモジュール
- ディスパッチャ

CPU の仮想化

8 / 35

## プロセスの切換の例

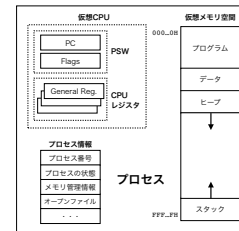


CPU の仮想化

9 / 35

## PCB (Process Control Block)

- プロセスを表現するカーネル内データ構造
- プロセス毎に存在する
- カーネル内のプロセステーブルに格納される



CPU の仮想化

10 / 35

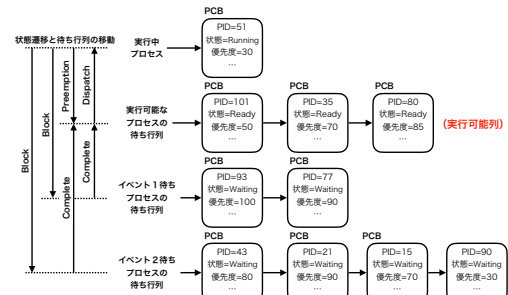
## PCB の内容

- 仮想 CPU
- プロセス番号
- 状態 (Running, Waiting, Ready 等)
- 優先度
- 統計情報 (CPU 利用時間等)
- 次回のアラーム時刻
- 親プロセス
- 子プロセス一覧
- シグナルハンドリング
- 使用中のメモリ
- オープン中のファイル
- カレントディレクトリ
- プロセス所有者のユーザ番号
- PCB のリストを作るためのポインタ

CPU の仮想化

11 / 35

## PCB のリスト



- Ready 状態 PCB のリスト = 実行可能列 (優先順位順にソート)
- イベント毎の Waiting 状態 PCB のリスト = イベント待ち行列

CPU の仮想化

12 / 35

## スレッド (Thread)

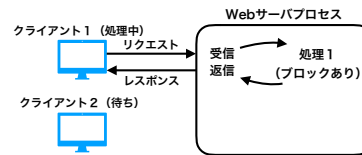
- CPU の仮想化によりマルチプログラミングが可能になった。
- プロセスが並行 (Concurrent) に実行できる。
- プロセスは一つの仮想 CPU を持っている。
- プロセスはコンピュータを仮想化したもの。
  - CPU が一つしかないコンピュータを仮想化している。
  - CPU を複数持つ SMP を仮想化するには不十分。
- 一つのプロセスが複数の仮想 CPU をもつモデルを導入する。
- プロセスが処理の流れスレッドを複数持つことができる。

CPU の仮想化

13 / 35

## スレッドの役割 (1)

マルチプログラミングを用いない Web サーバ



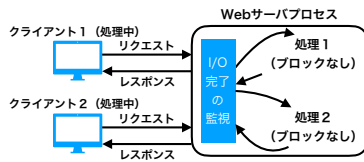
- 処理は順番に処理される。
- 前の処理が終わるまでクライアントは待たされる。

CPU の仮想化

14 / 35

## スレッドの役割 (2)

マルチプログラミングを用いない Web サーバ



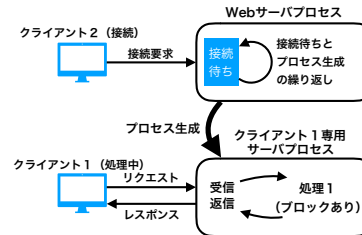
- 工夫すると並列して処理することも可能
- しかし、プログラミングが難しい。
- しかし、SMP が活かせない。

CPU の仮想化

15 / 35

## スレッドの役割 (3)

マルチプログラミングを用いる Web サーバ (プロセス版)



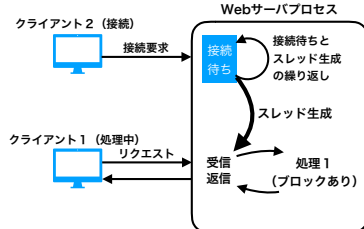
- クライアント毎にプロセスを起動 (fork()) する。
- プログラミングは易しい。
- しかし、処理が重いし、プロセス間の情報共有の効率が悪い。

CPU の仮想化

16 / 35

## スレッドの役割 (4)

マルチプログラミングを用いる Web サーバ (スレッド版)

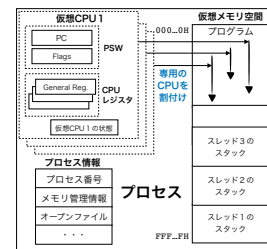


- クライアント毎にスレッドを起動する。
- プロセスの起動より 10~100 倍速い。
- スレッド間は情報を共有しやすい。
- プログラミングは少し難しい。

CPU の仮想化

17 / 35

## スレッドの形式 (1) - カーネルスレッド -

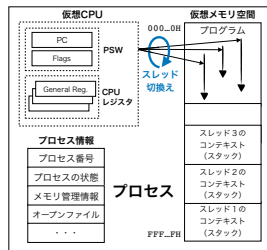


- プロセスが複数の仮想 CPU を持つ。

CPU の仮想化

18 / 35

## スレッドの形式 (2) - ユーザスレッド -



- ユーザプログラム (ライブラリ) の工夫でスレッドを実現する。
- 並行 (Parallel) 実行はできない。
- どれかのスレッドがブロックすると全スレッドが停止する。

CPU の仮想化

19 / 35

## スレッドの形式 (3) - スレッドモデル -

上記の2方式を組み合わせた3種類のスレッドモデルがある。

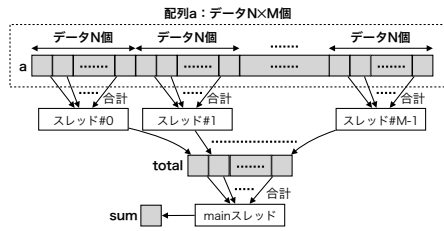
- *One-to-One Model*  
一つのスレッドを一つのカーネルスレッドで実行する。
- *Many-to-One Model*  
複数のスレッドを一つのカーネルスレッドで実行する。
- *Many-to-Many Model*  
複数のスレッドを複数のカーネルスレッドで実行する。

CPU の仮想化

20 / 35

## スレッドの使用例 (1)

M 個のスレッドで手分けをして合計を計算する様子  
(複数のカーネルスレッド (CPU) で手分けすることで短時間で処理が終わるはず)



CPU の仮想化

21 / 35

## スレッドの使用例 (2)

M 個のスレッドで合計を計算するプログラム

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <pthread.h>
4 #define N 1000 // 1スレッドの担当データ数
5 #define M 10 // スレッド数
6 pthread_t tid[M]; // M個のスレッドのスレッドID
7 pthread_attr_t attr[M]; // M個のスレッドの属性
8 int a[N*M]; // このデータの合計を求める
9 int total[M]; // 各スレッドの求めた部分合計
10 typedef struct { int no, min, max; } Args; // スレッドに渡す引数の型定義
11
12 void *thread(void *arg) { // 自スレッドの担当部分のデータの合計を求める
13     Args *args = arg; // m番目のスレッド
14     int sum = 0; // 合計を求める変数
15     for (int i=args->min; i<args->max; i++) { // a[N*m ... (N+1)*m] の
16         sum += a[i]; // 合計をsumに求める。
17     }
18     total[args->no]=sum; // 担当部分の合計を記録
19     return NULL; // スレッドを正常終了する
20 }

```

CPU の仮想化

22 / 35

## スレッドの使用例 (3)

```

22 int main() { // mainスレッドの実行はここから始まる
23     // 擬似的なデータを生成する
24     for (int i=0; i<M*N; i++) // 配列 a を初期化
25         a[i] = i+1;
26     // M個のスレッドを起動する
27     for (int m=0; m<M; m++) { // 各スレッドについて
28         Args *p = malloc(sizeof(Args)); // 引数領域を確保
29         p->no = m; // m番目のスレッド
30         p->min = N*m; // 担当範囲下限
31         p->max = N*(m+1); // 担当範囲上限
32         pthread_attr_init(&attr[m]); // アトリビュート初期化
33         pthread_create(&tid[m], &attr[m], thread, p); // スレッドを生成しスタート
34     }
35     // 各スレッドの終了を待ち、求めた小計を合算する
36     int sum = 0;
37     for (int m=0; m<M; m++) { // 各スレッドについて
38         pthread_join(tid[m], NULL); // 終了を待ち
39         sum += total[m]; // 小計を合算する
40     }
41     printf("1+2+...+M*d=%d\n", N*M, sum);
42     return 0;
43 }

```

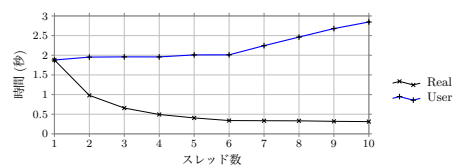
CPU の仮想化

23 / 35

## スレッドの使用例 (4)

実行時間の計測結果

|                 | 1      | 2      | 3     | 4      | 5      | 6     | 7     | 8      | 9     | 10     |
|-----------------|--------|--------|-------|--------|--------|-------|-------|--------|-------|--------|
| M               | 10,000 | 5,000  | 3,333 | 2,500  | 2,000  | 1,666 | 1,428 | 1,250  | 1,111 | 1,000  |
| M*N             | 10,000 | 10,000 | 9,999 | 10,000 | 10,000 | 9,996 | 9,996 | 10,000 | 9,999 | 10,000 |
| 移動時間 (s)        | 1.881  | 0.980  | 0.657 | 0.493  | 0.406  | 0.339 | 0.335 | 0.332  | 0.319 | 0.312  |
| ユーザ CPU 時間 (s)  | 1.879  | 1.953  | 1.959 | 1.958  | 2.009  | 2.011 | 2.244 | 2.462  | 2.679 | 2.846  |
| システム CPU 時間 (s) | 0.002  | 0.002  | 0.002 | 0.001  | 0.001  | 0.002 | 0.003 | 0.003  | 0.003 | 0.002  |

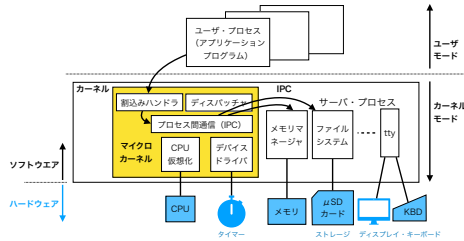


6 コアの Mac Pro で計測  
(Hyper-Threading のお陰で6 コアと12 コアの間中間的な振舞)

CPU の仮想化

24 / 35

## 実装例

第19章  
TacOSのCPU仮想化

CPUの仮想化

25 / 35

## TacOSのPCB

- 仮想CPU (sp)
- プロセス番号 (pid)
- 状態 (stat)
- 優先度 (nice, enice)
- プロセステーブルのインデックス (idx)
- イベント用カウンタとセマフォ (evtCnt, evtSem)
- プロセスのアドレス空間 (memBase, memLen)
- プロセスの親子関係の情報 (parent, exitStat)
- オープン中のファイル一覧 (fds[])
- PCBリストの管理 (prev, next)
- スタックオーバーフローの検知 (magic)

CPUの仮想化

26 / 35

## TacOSのPCB (前半)

```
// ----- プロセス関連 -----
#define PRC_MAX 10 // プロセスは最大 10 個
#define P_KERN_STKSIZ 200 // プロセス毎のカーネルスタックのサイズ
#define P_LOW_PRI 30000 // プロセスの最低優先度
#define P_RUN 1 // プロセスは実行可能または実行中
#define P_WAIT 2 // プロセスは待ち状態
#define P_ZOMBIE 3 // プロセスは実行終了
#define P_MAGIC 0xabcd // スタックオーバーフロー検知に使用
#define P_FILE_MAX 4 // プロセスがオープンできるファイルの最大数

// プロセスコントロールブロック (PCB)
// 優先度は値が小さいほど優先度が高い
1 struct PCB {
2     int sp; // PCB を表す構造体
3     // コンテキスト (他の CPU レジスタと PSW は
4     // プロセスのカーネルスタックに置く)
5     int pid; // プロセス番号
6     int stat; // プロセスの状態
7     int nice; // プロセスの本来優先度
8     int enice; // プロセスの実質優先度 (将来用)
9     int idx; // この PCB のプロセステーブル上のインデックス
```

CPUの仮想化

27 / 35

## TacOSのPCB (後半)

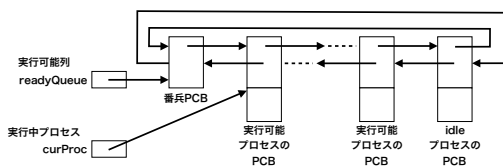
```
10 // プロセスのイベント用セマフォ
11 int evtCnt; // カウンタ (>0:sleep中, ==-1:wait中, ==0:未使用)
12 int evtSem; // イベント用セマフォの番号
13
14 // プロセスのアドレス空間(text, data, bss, ...)
15 char[] memBase; // プロセスのメモリ領域のアドレス
16 int memLen; // プロセスのメモリ領域の長さ
17
18 // プロセスの親子関係の情報
19 PCB parent; // 親プロセスへのポインタ
20 int exitStat; // プロセスの終了ステータス
21
22 // オープン中のファイル一覧
23 int[] fds; // オープン中のファイル一覧
24
25 // プロセスは重連結環状リストで管理
26 PCB prev; // PCB リスト (前へのポインタ)
27 PCB next; // PCB リスト (次へのポインタ)
28 int magic; // スタックオーバーフローを検知
29 };
```

CPUの仮想化

28 / 35

## TacOSの実行可能列

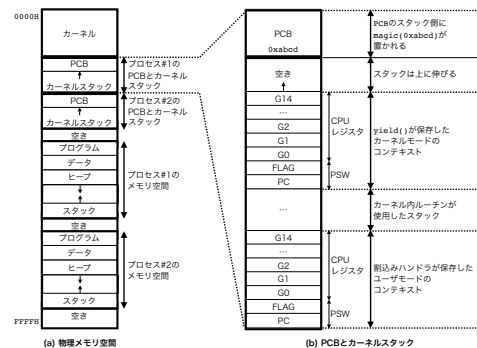
- PCBの双方向環状リスト
- 優先度順にソート (curProcは実行中のプロセス)
- 末尾にidleプロセスが常駐



CPUの仮想化

29 / 35

## TacOSのメモリ配置



CPUの仮想化

30 / 35

## TacOS のタイマー管理プログラム

```

3 // タイマー割り込みハンドラ(10ms 毎に割り込みによって起動される)
4 interrupt tmrIntr() {
5     boolean disp = false;                // ディスパッチの必要性
6
7     // 起きないといけないプロセスを起こしてまわる
8     for (int i=0; i<PRC_MAX; i=i+1) {
9         PCB p = procTbl[i];
10        if (p!=null && p.evtCnt>0) {
11            int cnt = p.evtCnt - TICK;      // タイマー稼働中なら
12            // 残り時間を計算
13            if (cnt<=0) {
14                cnt = 0;                  // 時間が来たら
15                // タイマーを停止し
16                disp = iSemV(p.evtSem) || disp; // プロセスを起こす
17            }
18            p.evtCnt = cnt;
19        }
20    }
21    if (disp) yield();                    // 必要ならディスパッチ
22 }

```

CPU の仮想化

31 / 35

## 参考 (普通の関数)

## C--言語の void 型関数

```

1 void func();
2
3 void sFunc() {
4     for (int i=0; i<10; i=i+1) {
5         func(); // 関数呼び出し
6     }
7 }

```

```

1 .sFunc  PUSH  FP
2         LD   FP,SP
3         PUSH G3
4         LD   G3,#0
5
6 .L1
7         CMP  G3,#10
8         JGE  .L2
9         CALL .func
10        LD   G0,G3
11        ADD  G0,#1
12        LD   G3,G0
13        JMP  .L1
14
15 .L2
16        POP  G3
17        POP  FP
18        RET

```

CPU の仮想化

32 / 35

## 参考 (interrupt 関数)

## C--言語の interrupt 型関数

```

1 void func();
2
3 interrupt iFunc() {
4     for (int i=0; i<10; i=i+1) {
5         func(); // 関数呼び出し
6     }
7 }

```

```

1 .iFunc  PUSH  G0
2         PUSH G1
3         PUSH G2
4         LD   FP,SP
5         PUSH G3
6         LD   G3,#0
7
8 .L1
9         CMP  G3,#10
10        JGE  .L2
11        CALL .func
12        LD   G0,G3
13        ADD  G0,#1
14        LD   G3,G0
15        JMP  .L1
16
17 .L2
18        POP  G3
19        POP  FP
20        POP  G2
21        POP  G1
22        POP  G0
23        RETI

```

CPU の仮想化

33 / 35

## TacOS のコンテキスト保存プログラム (yield())

```

1 _yield
2 ;--- G13(SP)以外の CPU レジスタと FLAG をカーネルスタックに退避 ---
3 push  g0          ; FLAG の保存場所を準備する
4 push  g0          ; G0 を保存
5 ld    g0,flag     ; FLAG を上で準備した位置に保存
6 st    g0,2,sp     ;
7 push  g1          ; G1 を保存
8 push  g2          ; G2 を保存
9
10 ...
11
12 push  g11         ; G11 を保存
13 push  fp          ; フレームポインタ(G12)を保存
14 push  usp         ; ユーザーモードスタックポインタ(G14)を保存
15
16 ;
17 ;----- G13(SP)を PCB に保存 -----
18 ld    g1,_curProc ; G1 <- curProc
19 st    sp,0,g1     ; [G1+0] は PCB の sp フィールド
20
21 ;
22 ;----- [curProc の magic フィールド]をチェック -----
23 ld    g0,30,g1    ; [G1+30] は PCB の magic フィールド
24 cmp   g0,#0xabcd  ; P_MAGIC と比較、一致しなければ
25 jnz   .stkOverflow ; カーネルスタックがオーバーフローしている
26
27
28

```

CPU の仮想化

34 / 35

## TacOS のコンテスト復旧プログラム (dispatch())

\_dispatch は \_yield (28 行) の直後にある。(つながっている！)

```

1 _dispatch
2 ;----- 次に実行するプロセスの G13(SP)を復元 -----
3 ld    g0,_readyQueue ; 実行可能列の番兵のアドレス
4 ld    g0,28,g0       ; [G0+28] は PCB の next フィールド(先頭の PCB)
5 st    g0,_curProc    ; 現在のプロセス(curProc)に設定する
6 ld    sp,0,g0        ; PCB から SP を取り出す
7
8 ;
9 ;----- G13(SP)以外の CPU レジスタを復元 -----
10 pop   usp           ; ユーザーモードスタックポインタ(G14)を復元
11 pop   fp            ; フレームポインタ(G12)を復元
12 pop   g11           ; G11 を復元
13 pop   g10           ; G10 を復元
14 pop   g9            ; G9 を復元
15
16 ...
17
18 pop   g1            ; G1 を復元
19 pop   g0            ; G0 を復元
20
21 ;
22 ;----- PSW(FLAG と PC)を復元 -----
23 ret    ; RETI 命令で一度に POP して復元する
24
25

```

CPU の仮想化

35 / 35

## オペレーティングシステム 第4章 スケジューリング

<https://github.com/tctsigemura/OSTextBook>

スケジューリング

1 / 17

## 評価基準

- スループット (Throughput)
- ターンアラウンド時間 (Turnaround time)
- レスポンス時間 (Response time)
- 締め切り (Deadline)
- その他 (公平性, 省エネ, 予測性など)

スケジューリング

2 / 17

## システムごとの目標

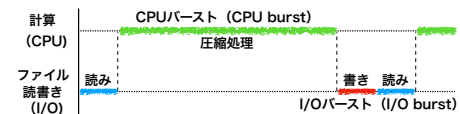
| コンピュータの種類       | 重視する性能             |
|-----------------|--------------------|
| メインフレーム (バッチ処理) | スループット, ターンアラウンド時間 |
| ネットワークサーバ       | レスポンス時間, スループット    |
| デスクトップパソコン      | レスポンス時間            |
| モバイルデバイス        | レスポンス時間, 省エネルギー    |
| 組込み制御           | 締め切り               |

スケジューリング

3 / 17

## CPU バウンドプロセス

動画圧縮の例

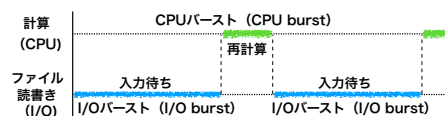


スケジューリング

4 / 17

## I/O バウンドプロセス

スプレッドシートの例



スケジューリング

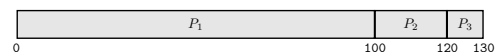
5 / 17

## FCFS スケジューリング (1)

FCFS (First-Come, First-Served)

- プリエンブションしないスケジューリング方式

| プロセス  | 到着時刻 | CPU バースト時間 (ms) |
|-------|------|-----------------|
| $P_1$ | 0    | 100             |
| $P_2$ | 0    | 20              |
| $P_3$ | 0    | 10              |



- $P_1, P_2, P_3$  の順に実行
- 平均ターンアラウンド時間  $((100 + 120 + 130) / 3 = 117 \text{ ms})$
- 最悪の平均ターンアラウンド時間を選択することもある。

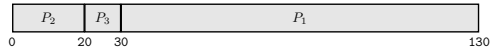
スケジューリング

6 / 17

## FCFS スケジューリング (2)

- 平均ターンアラウンド時間は 到着順により大きく変化する。

| プロセス  | 到着時刻 | CPU バースト時間 (ms) |
|-------|------|-----------------|
| $P_1$ | 0    | 100             |
| $P_2$ | 0    | 20              |
| $P_3$ | 0    | 10              |



- $P_2, P_3, P_1$  の順に実行
- 平均ターンアラウンド時間  $((20 + 30 + 130)/3 = 60 \text{ ms})$

スケジューリング

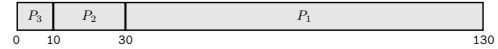
7 / 17

## SJF スケジューリング

SJF (Shortest-Job-First)

- プリエンブションしないスケジューリング方式

| プロセス  | 到着時刻 | CPU バースト時間 (ms) |
|-------|------|-----------------|
| $P_1$ | 0    | 100             |
| $P_2$ | 0    | 20              |
| $P_3$ | 0    | 10              |



- 平均ターンアラウンド時間  $((10 + 30 + 130)/3 = 57 \text{ ms})$

スケジューリング

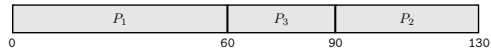
8 / 17

## SRTF スケジューリング (1)

SRTF (Shortest-Remaining-Time-First)

比較のための SJF スケジューリングの例

| プロセス  | 到着時刻 | CPU バースト時間 (ms) |
|-------|------|-----------------|
| $P_1$ | 0    | 60              |
| $P_2$ | 10   | 40              |
| $P_3$ | 60   | 30              |



- SJF はプリエンブションなし
- 平均ターンアラウンド時間  $((60 - 0) + (90 - 10) + (130 - 60))/3 = 70 \text{ ms})$

スケジューリング

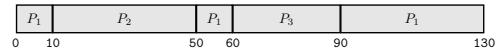
9 / 17

## SRTF スケジューリング (2)

SRTF (Shortest-Remaining-Time-First)

前の SJF と同じプロセスのを SRTF でスケジューリング

| プロセス  | 到着時刻 | CPU バースト時間 (ms) |
|-------|------|-----------------|
| $P_1$ | 0    | 60              |
| $P_2$ | 10   | 40              |
| $P_3$ | 60   | 30              |



- SRTF はプリエンブションあり
- 平均ターンアラウンド時間  $((130 - 0) + (50 - 10) + (90 - 60))/3 = 67 \text{ ms})$

スケジューリング

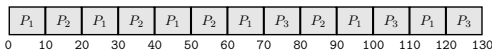
10 / 17

## RR スケジューリング (1)

RR (Round-Robin)

クオンタムタイムまでプリエンブションしない。

| プロセス  | 到着時刻 | CPU バースト時間 (ms) |
|-------|------|-----------------|
| $P_1$ | 0    | 60              |
| $P_2$ | 10   | 40              |
| $P_3$ | 60   | 30              |



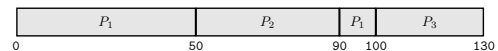
- クオンタムタイム = 10ms
- 平均ターンアラウンド時間  $((120 - 0) + (90 - 10) + (130 - 60))/3 = 90 \text{ ms})$

スケジューリング

11 / 17

## RR スケジューリング (2)

| プロセス  | 到着時刻 | CPU バースト時間 (ms) |
|-------|------|-----------------|
| $P_1$ | 0    | 60              |
| $P_2$ | 10   | 40              |
| $P_3$ | 60   | 30              |



- クオンタムタイム = 50ms
- 平均ターンアラウンド時間  $((100 - 0) + (90 - 10) + (130 - 60))/3 = 83 \text{ ms})$

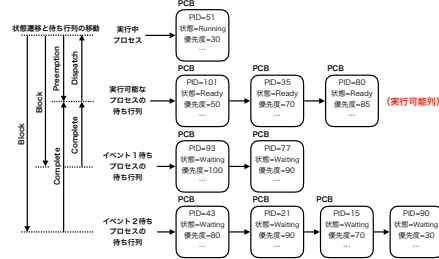
スケジューリング

12 / 17

## 優先度順スケジューリング

## Priority

- 実行可能列を優先度順でソートしておく。



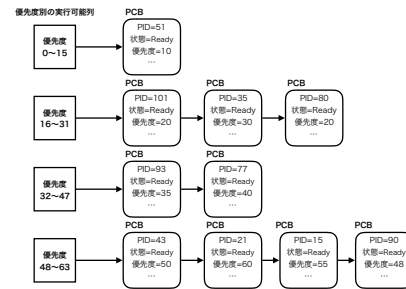
- 静的優先度／動的優先度
- スタベーション (starvation) : 飢餓
- エージング (aging) : 老化, 熟成

スケジューリング

13 / 17

## FB スケジューリング

## FB (Multilevel Feedback Queue)

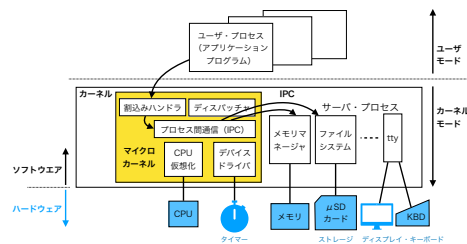


- エージング

スケジューリング

14 / 17

## 実装例

第19章 (6節)  
TacOSのスケジューラ

スケジューリング

15 / 17

## TacOSのスケジューラ

```

1 public void schProc(PCB proc) {
2     int r = setPri(DI|KERN); // 割り込み禁止、カーネル
3     int enice = proc.enice;
4     PCB head = readyQueue.next; // 実行可能列から
5     while (head.enice <= enice) // 優先度がより低い
6         head = head.next; // プロセスを探す
7     insProc(head, proc); // 見つけたプロセスの
8     setPri(r); // 直前に挿入する
9 } // 割り込み状態を復元する

```

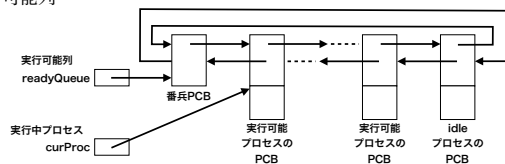
スケジューリング

16 / 17

## TacOSの実行可能列 (参考)

- yield()
- dispatch()

- 実行可能列



スケジューリング

17 / 17

## オペレーティングシステム 第5章 プロセス同期

<https://github.com/tctsigemura/OSTextBook>

プロセス同期

1 / 38

### 共有資源

- スレッド間の共有変数
- プロセス間の共有メモリ
- カーネル内のデータ構造
- ファイル
- 入出力装置
- その他

プロセス同期

2 / 38

### 競合 (Race Condition, Competition)

```
// スレッド間の共有変数
receipt DS 1 // 入金 (3万円)
payment DS 1 // 引き落とし (2万円)
account DS 1 // 残高 (10万円)

// 入金管理スレッド
// 会社から給料 (3万円) を受領し
// receipt に金額を格納した.

// 引き落とし管理スレッド
// カード会社から引き落としを
// 受信し payment に金額を格納した.

// 口座 account に足し込む
(1) LD G0,account
(2) ADD G0,receipt
(3) ST G0,account

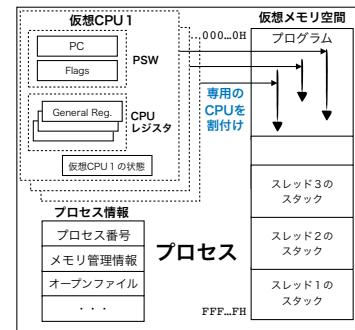
// 口座 account から差し引く
(a) LD G0,account
(b) SUB G0,payment
(c) ST G0,account

// 次の処理に進む
```

プロセス同期

3 / 38

### プロセスの構造とカーネルスレッド (参考)



- 仮想CPUにスレッドのコンテキストを保存

プロセス同期

4 / 38

### クリティカルセクション (Critical Section)

複数のプロセス (スレッド) が同時に実行すると競合が発生する!!  
 例えば共有変数を変更する処理はクリティカルセクションである。  
 (前の例で「(1) から (3)」と「(a) から (c)」)  
 (クリティカルリージョン (Critical Region) とも呼ぶ)  
 クリティカルセクションには複数のスレッドが入ってはならない。

クリティカルセクションの競合問題を効率よく解決するには、

- (1) 二つ以上のスレッドが同時に入らない。
- (2) 入っているスレッドがない時は、すぐに入れる。
- (3) 入るためにスレッドが永遠に待たされることがない。

プロセス同期

5 / 38

### 相互排除 (mutual exclusion)

複数のスレッドが同時にクリティカルセクションに入らない制御!!  
 (排他制御, 相互排除も呼ぶ)

相互排除を行うプログラムの部分のことを次のように呼ぶ。

- エントリーセクション (Entry Section)  
クリティカルセクションに入る手続き
- エグジットセクション (Exit Section)  
クリティカルセクションを出る手続き

プロセス同期

6 / 38

## 割り込み禁止

シングルプロセッサシステムで相互排除に使用できる.

| // 口座 account に足し込む | // 口座 account から差し引く |
|---------------------|----------------------|
| DI // Entry Section | DI // Entry Section  |
| (1) LD G0,account   | (a) LD G0,account    |
| (2) ADD G0,receipt  | (b) SUB G0,payment   |
| (3) ST G0,account   | (c) ST G0,account    |
| EI // Exit Section  | EI // Exit Section   |

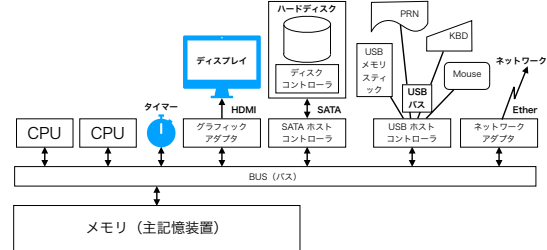
- エントリーセクション (Entry Section)  
割り込み禁止の場合は DI 命令
- エグジットセクション (Exit Section)  
割り込み禁止の場合は EI 命令

DI 命令, EI 命令は特権命令なのでカーネル内だけで可能.  
長時間の割り込み禁止は NG なので注意が必要.

プロセス同期

7 / 38

## SMP のハードウェア構成を思い出す



- SMP では割り込み禁止だけでは相互排除できない.
- CPU はメモリ（バス）を共有する（バスの利用は順番に行う）
- 割り込みは機械語命令の途中では発生しない.

プロセス同期

8 / 38

## 専用命令 (TS 命令)

TS (Test and Set) 命令は SMP システムでの相互排除に使用できる.  
「TS R, M」は以下を**アトミック (atomic)** に実行する.

1. バスをロックする
2.  $R \leftarrow [M]$
3. if ( $R=0$ )  $Zero \leftarrow 1$  else  $Zero \leftarrow 0$
4.  $[M] \leftarrow 1$
5. バスのロックを解除する

プロセス同期

9 / 38

## 専用命令 (TS 命令の使用例)

```
// エントリーセクション
L1 DI // クリティカルセクションでプリエンブションしないように
TS G0, FLG // ゼロを取得できるプロセス (スレッド) は一時には一つだけ
JZ L2 // ゼロを取得できた場合だけクリティカルセクションに入れる
EI // ビジーウェーティングの間はプリエンブションのチャンスを作る
JMP L1 // クリティカルセクションに入れない場合はビジーウェーティング

// クリティカルセクション
L2 ...

// エグジットセクション
LD G0, #0
ST G0, FLG // フラグのクリアは普通のST命令でOK
EI // クリティカルセクション終了, プリエンブションしても良い

// 非クリティカルセクション
...

// メモリ上に置いたフラグ (CPUのフラグと混同しないこと)
FLG DC 0 // 初期値ゼロ (TS命令により1に書き換えられる)
```

- ビジーウェーティング (Busy Waiting) の一種.

プロセス同期

10 / 38

## 専用命令 (SW 命令)

SW (Swap) 命令も SMP システムでの相互排除に使用できる.  
「SW R, M」は以下を**アトミック (atomic)** に実行する.

1. バスをロックする
2.  $T \leftarrow [M]$
3.  $[M] \leftarrow R$
4.  $R \leftarrow T$
5. バスのロックを解除する

ここで  $T$  は CPU 内部の一時的なレジスタ  
( $T$  レジスタの存在はプログラムから見えない)

- 次の例もビジーウェーティング (Busy Waiting) の一種.

プロセス同期

11 / 38

## 専用命令 (SW 命令の使用例)

```
// エントリーセクション
L1 DI // クリティカルセクションでプリエンブションしないように
LD G0, #1 // フラグに書き込む値
SW G0, FLG // ゼロを取得できるプロセス (スレッド) は一時には一つだけ
CMP G0, #0 // ゼロを取得できたかテスト
JZ L2 // ゼロを取得できた場合だけクリティカルセクションに入れる
EI // ビジーウェーティングの間はプリエンブションのチャンスを作る
JMP L1 // クリティカルセクションに入れない場合はビジーウェーティング

// クリティカルセクション
L2 ...

// エグジットセクション
LD G0, #0
ST G0, FLG // フラグのクリアは普通のST命令でOK
EI // クリティカルセクション終了, プリエンブションしても良い

// 非クリティカルセクション
...

// メモリ上に置いたフラグ (CPUのフラグと混同しないこと)
FLG DC 0 // 初期値ゼロ (SW命令により1に書き換えられる)
```

プロセス同期

12 / 38

## 専用命令 (CAS 命令)

CAS (Compare And Swap) 命令も SMP システムでの相互排除に使用できる。「CAS R0, R1, M」は、以下を**アトミック (atomic)** に実行する。

1. バスをロックする
2.  $T \leftarrow [M]$
3. if ( $T == R0$ ) {  $[M] \leftarrow R1$ ; Zero  $\leftarrow 1$ ; }  
else {  $R0 \leftarrow T$ ; Zero  $\leftarrow 0$ ; }
4. バスのロックを解除する

|                     |                      |
|---------------------|----------------------|
| // 口座 account に足し込む | // 口座 account から差し引く |
| L1 LD G1,account    | L2 LD G1,account     |
| LD G0,G1            | LD G0,G1             |
| ADD G1,receipt      | SUB G1,payment       |
| CAS G0,G1,account   | CAS G0,G1,account    |
| JNZ L1              | JNZ L2               |

**ロックフリー (Lock-free) なアルゴリズム**

プロセス同期

13 / 38

## フラグを用いる方法 (Peterson のアルゴリズム)

```
// スレッド間の共有変数
boolean flag[] = {false, false}; // クリティカルセクションに入りたい
int turn = 0; // 後でやってきたのはどちら

// スレッド 0
...

// エントリーセクション
flag[0] = true;
turn = 0;
while (turn==0 && flag[1]==true)
    ; // ビジーウェイティング

// クリティカルセクション
...

// エグジットセクション
flag[0] = false;

// 非クリティカルセクション
...
```

プロセス同期

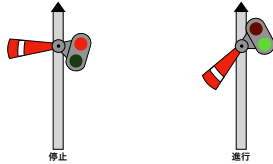
14 / 38

## セマフォ (Semaphore)

1965 年に E. W. Dijkstra が提案したデータ型である。

- **ビジーウェイティング (Busy Waiting)** を用いない
- オペレーティングシステムが提供する洗練された同期機構
- システムコール等でユーザプロセスに提供
- サブルーチンとしてサービスモジュール等に提供

セマフォ (Semaphore: 腕木式信号機) の元々の意味はこれ!



プロセス同期

15 / 38

## セマフォ (Semaphore)

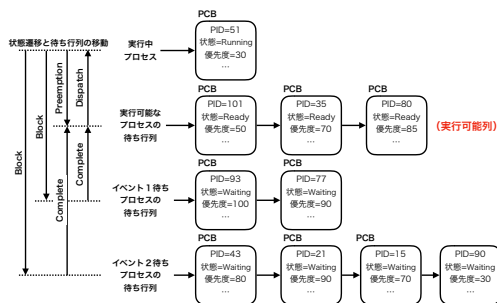
- セマフォはデータ構造 (**セマフォ型**, セマフォ構造体)  
例えば C 言語で: `typedef struct { ... } Semaphore;`
- カウンタは 0 以上の整数値 (0 は**赤信号**の意味)
- プロセスの待ち行列を作ることができる。
- セマフォ型の変数に P 操作と V 操作ができる。
- P 操作 (Proberen:try)
- V 操作 (Verhogen:raise)
- ユーザプロセスには P, V システムコールが提供される
- サービスモジュールやデバイスドライバには P, V サブルーチン

セマフォはプロセス (スレッド) の状態を待ち (Waiting) 状態に変える。ビジーウェイティング (Busy Waiting) では無いので CPU を無駄遣いすることはない。

プロセス同期

16 / 38

## PCB のリスト (参考)



- Ready 状態 PCB のリスト = 実行可能列 (優先順位順にソート)
- イベント毎 (**セマフォ毎**) の待ち行列

プロセス同期

17 / 38

## セマフォ (Semaphore) の P 操作

P 操作 (P(S))

1. セマフォ (S) の値が 1 以上ならセマフォの値を 1 減らす。
2. 値が 0 ならプロセス (スレッド) を待ち (Waiting) 状態にし、
3. セマフォの待ち行列に追加する。

クリティカルセクションのエントリーセクション等で使用できる。

```
void P(Semaphore S) {
    if (S > 0) {
        S--;
    } else {
        プロセスを待ち(Waiting)状態にする;
        プロセスを S の待ち行列に追加する;
    }
}
```

プロセス同期

18 / 38

## セマフォ (Semaphore) の V 操作

V操作 (V(S))

1. 待っているプロセス (スレッド) が無い場合は, セマフォ (S) の値を1増やす.
2. セマフォ (S) の待ち行列にプロセス (スレッド) がある場合は, それらの一つを起床させる.

クリティカルセクションのエグジジットセクション等で使用できる.

```
void V(Semaphore S) {
    if (S の待ち行列は空) {
        S++;
    } else {
        一つのプロセスを待ち行列から取り出す;
        そのプロセスを実行可能(Ready)状態にする;
    }
}
```

プロセス同期

19 / 38

## セマフォの使用例 (相互排除問題)

```
int    account;           // スレッド間の共有変数 (残高)
Semaphore accSem = 1;     // 初期値1のセマフォaccSem (accountのロック用)
void receiveThread() {
    for ( ; ; ) {
        int receipt = receiveMoney(); // 入金管理スレッド
        // 入金管理スレッドは以下を繰り返す
        P( &accSem ); // ネットワークから入金を受信する
        account = account + receipt; // account変数をロックするための P 操作
        V( &accSem ); // account変数を変更する(クリティカルセクション)
        // account変数をロック解除するための V 操作
    }
}
void payThread() {
    for ( ; ; ) {
        int payment = payMoney(); // 引落し管理スレッド
        // 引落し管理スレッドは以下を繰り返す
        P( &accSem ); // ネットワークから支払い額を受信する
        account = account - payment; // account変数をロックするための P 操作
        V( &accSem ); // account変数を変更する(クリティカルセクション)
        // account変数をロック解除するための V 操作
    }
}
```

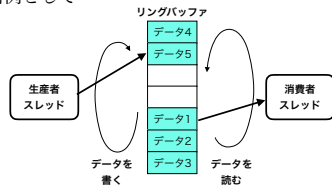
初期値が1のセマフォを用いる.

プロセス同期

20 / 38

## 生産者・消費者問題

セマフォの使用例として



- 生産者スレッドはデータをバッファに書き込む.
- 消費者スレッドはデータをバッファから読む.
- バッファが溢れないように両方で歩調を合わせる必要がある.

プロセス同期

21 / 38

## セマフォの使用例 (生産者・消費者問題)

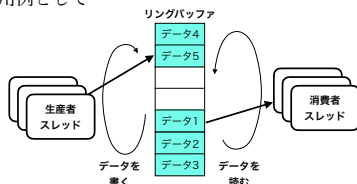
```
Data    buffer[N];        // スレッド間で共有するリングバッファ
Semaphore emptySem = N;   // リングバッファの空きスロット数を表すセマフォ
Semaphore fullSem = 0;    // リングバッファの使用済みスロット数を表すセマフォ
void producerThread() {
    int in = 0;            // 生産者スレッド
    for ( ; ; ) {
        Data d = produce(); // リングバッファの次回格納位置
        // 生産者スレッドは以下を繰り返す
        P( &emptySem ); // 新しいデータを作る
        buffer[ in ] = d; // リングバッファの空き数をデクリメント
        in = (in + 1) % N; // リングバッファにデータを格納
        V( &fullSem ); // 次回格納位置を更新
        // リングバッファのデータ数をインクリメント
    }
}
void consumerThread() {
    int out = 0;           // 消費者スレッド
    for ( ; ; ) {
        P( &fullSem ); // リングバッファの次回取り出し位置
        // 消費者スレッドは以下を繰り返す
        Data d = buffer[ out ]; // リングバッファのデータ数をデクリメント
        out = (out + 1) % N; // リングバッファからデータを取り出す
        V( &emptySem ); // 次回取り出し位置を更新
        consume( d ); // リングバッファの空き数をインクリメント
        // データを使用する
    }
}
```

プロセス同期

22 / 38

## 複数生産者・複数消費者問題

セマフォの使用例として



- 生産者スレッド同士で相互排除が必要.
- 消費者スレッド同士でも相互排除が必要.

プロセス同期

23 / 38

## セマフォの使用例 (複数生産者・複数消費者問題 1/2)

```
Data    buffer[N];        // スレッド間で共有するリングバッファ
Semaphore emptySem = N;   // リングバッファの空きスロット数を表すセマフォ
Semaphore fullSem = 0;    // リングバッファの使用済みスロット数を表すセマフォ
int    in = 0;            // 生産者スレッドの次回格納位置
Semaphore inSem = 1;      // in の排他制御用セマフォ
void producerThread() {
    for ( ; ; ) {
        Data d = produce(); // 生産者スレッド (複数のスレッドで並列実行する)
        // 生産者スレッドは以下を繰り返す
        P( &emptySem ); // 新しいデータを作る
        P( &inSem ); // リングバッファの空き数をデクリメント
        buffer[ in ] = d; // in にロックを掛ける
        in = (in + 1) % N; // リングバッファにデータを格納
        V( &inSem ); // 次回格納位置を更新
        V( &fullSem ); // in のロックを外す
        // リングバッファのデータ数をインクリメント
    }
}
```

- 複数の生産者スレッドが producerThread() を実行する.
- in 変数は生産者スレッド間の共有変数になった.
- in 変数の使用で相互排除するために inSem を準備した.

プロセス同期

24 / 38

## セマフォの使用例 (複数生産者・複数消費者問題 2/2)

```

int out = 0;                // リングバッファの次回取り出し位置
Semaphore outSem = 1;       // out の排他制御用セマフォ
void consumerThread() {
    for ( ; ; ) {
        P( &fullSem );      // 消費者スレッドは以下を繰り返す
        P( &outSem );        // リングバッファのデータ数をデクリメント
        Data d = buffer[ out ]; // out にロックを掛ける
        out = (out + 1) % N;   // リングバッファからデータを取り出す
        V( &emptySem );       // 次回取り出し位置を更新
        consume( d );         // out のロックを外す
    }
}

```

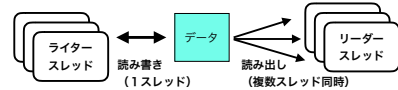
- 複数の消費者スレッドが consumerThread() を実行する。
- out 変数は消費者スレッド間の共有変数になった。
- out 変数の使用で相互排除するために outSem を準備した。

プロセス同期

25 / 38

## リーダ・ライタ問題

次の場合、単にロックするより並行性 (concurrency) を高くできる。



- ライタスレッドはデータ読んでは変更して書き込む。
- データ変更中は他のスレッドはデータにアクセスしてはならない。  
(排他ロック (exclusive lock))
- リーダスレッドはデータを変更しない。
- 複数のリーダスレッドが同時にデータにアクセスしても良い。  
しかし、その間、ライタスレッドはデータにアクセスできない。  
(共有ロック (shared lock))

プロセス同期

26 / 38

## セマフォの使用例 (リーダ・ライタ問題 1/2)

データとライタスレッド部分

```

Data records;              // 共有するデータ
Semaphore rwSem = 1;       // リーダとライタの排他用セマフォ
void writerThread() {
    for ( ; ; ) {
        Data d = produce(); // ライタスレッド (複数のスレッドで並列実行する)
        P( &rwSem );         // ライタスレッドは以下を繰り返す
        writeRecords( d );   // 新しいデータを作る
        V( &rwSem );         // 共有データにロックを掛ける
    }
}

```

- 複数のライタスレッドが writerThread() を実行する。
- rwSem=1 は、普通の相互排除と同じ。
- ライタスレッドは、排他ロック (exclusive lock) を用いる。

次ページのスライドがリーダスレッド部分

プロセス同期

27 / 38

## セマフォの使用例 (リーダ・ライタ問題 2/2)

```

int cnt = 0;               // リーダ間の共有変数 (読出し中のリーダ数)
Semaphore cntSem = 1;      // cnt の排他制御用セマフォ
void readerThread() {
    for ( ; ; ) {
        P( &cntSem );       // リーダスレッド (複数のスレッドで並列実行する)
        if ( cnt == 0 ) P( &rwSem ); // リーダスレッドは以下を繰り返す
        cnt = cnt + 1;      // cnt にロックを掛ける
        V( &cntSem );       // cnt のロックを外す
        Data d = readRecords(); // データを読み出す
        P( &cntSem );       // cnt にロックを掛ける
        cnt = cnt - 1;      // cnt をデクリメント
        if ( cnt == 0 ) V( &rwSem ); // 自分が最後のリーダなら、代表してロックを外す
        V( &cntSem );       // cnt のロックを外す
        consume( d );       // データを使用する
    }
}

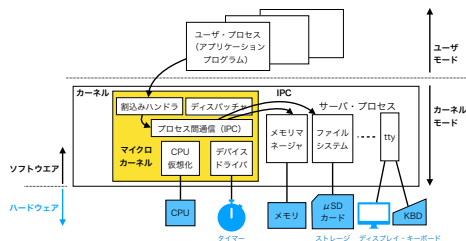
```

- 複数のリーダスレッドが readerThread() を実行する。
- cnt は、クリティカルセクション中のリーダスレッド数。
- cntSem=1 は cnt の相互排除に用いる。
- リーダスレッドは、共有ロック (shared lock) を用いる。

プロセス同期

28 / 38

## 実装例

第20章  
TacOSのセマフォ

プロセス同期

29 / 38

## TacOSのセマフォ構造体 (カーネル内)

```

#define SEM_MAX 30          // セマフォは最大 30 個

struct Sem {                // セマフォを表す構造体
    int cnt;                // カウンタ
    PCB queue;              // 待ち行列
};

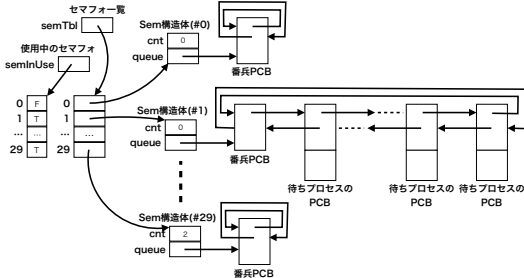
```

- セマフォは最大 30 個 (TaC のメモリは小さい)
- セマフォ構造体の名前は Sem
- cnt がセマフォの値 (0 以上)
- queue に、このセマフォを待っているプロセスの待ち行列を作る。

プロセス同期

30 / 38

## TacOS のセマフォ関連データ構造 (カーネル内)



- TacOS では、セマフォを semTbl のインデックスで識別する。
- Sem 構造体 (#0, #1, #29) は、未使用、待ちあり、待ちなしの例

プロセス同期

31 / 38

## TacOS でのセマフォの架空の使用例

```

1 #include <kernel.h>
2 int account; // スレッド間の共有変数 (残高)
3 int accSem; // accountのロック用セマフォの番号
4 void initProc() { // プロセスの初期化ルーチン
5     accSem = newSem(1); // 初期値1のセマフォを確保する
6 }
7
8 void receiveThread() { // 入金管理スレッド
9     for ( ; ; ) { // 入金管理スレッドは以下を繰り返す
10        int receipt = receiveMoney(); // ネットワークから入金を受信する
11        semP( accSem ); // account変数をロックするための P 操作
12        account = account + receipt; // account変数を変更する(クリティカルセクション)
13        semV( accSem ); // account変数をロック解除するための V 操作
14    }
15}
16
17 void payThread() { // 引落し管理スレッド
18     for ( ; ; ) { // 引落し管理スレッドは以下を繰り返す
19        int payment = payMoney(); // ネットワークから入金を受信する
20        semP( accSem ); // account変数をロックするための P 操作
21        account = account - payment; // account変数を変更する(クリティカルセクション)
22        semV( accSem ); // account変数をロック解除するための V 操作
23    }
24}

```

プロセス同期

32 / 38

## TacOS のセマフォ割当て解放ルーチン (カーネル内)

```

1 Sem[] semTbl=array(SEM_MAX); // セマフォの一覧表
2 boolean[] semInUse=array(SEM_MAX); // どれが使用中か (falseで初期化)
3
4 // セマフォの割当て
5 public int newSem(int init) {
6     int r = setPri(DI|KERN); // 割り込み禁止、カーネル
7     for (int i=0; i<SEM_MAX; i=i+1) { // 全てのセマフォについて
8         if (!semInUse[i]) { // 未使用のものを見つけたら
9             semInUse[i] = true; // 使用中に変更し
10            semTbl[i].cnt = init; // カウンタを初期化し
11            setPri(r); // 割り込み状態を復元し
12            return i; // セマフォ番号を返す
13        }
14    }
15    panic("newSem"); // 未使用が見つからなかった
16    return -1; // ここは実行されない
17}
18
19 // セマフォの解放
20 // (書き込み1回で仕事が終わるので割り込み許可でも大丈夫)
21 public void freeSem(int s) {
22     semInUse[s] = false; // 未使用に変更
23}

```

プロセス同期

33 / 38

## TacOS の P 操作ルーチン (カーネル内)

```

1 public void semP(int sd) {
2     int r = setPri(DI|KERN); // 割り込み禁止、カーネル
3     if (sd<0 || SEM_MAX<=sd || !semInUse[sd]) // 不正なセマフォ番号
4         panic("semP(%d)", sd);
5
6     Sem s = semTbl[sd];
7     if (s.cnt>0) { // カウンタから引けるなら
8         s.cnt = s.cnt - 1; // カウンタから引く
9     } else { // カウンタから引けないなら
10        delProc(curProc); // 実行可能列から外し
11        curProc.stat = P_WAIT; // 待ち状態に変更する
12        insProc(s.queue, curProc); // セマフォの行列に登録
13        yield(); // CPUを解放し
14    } // 他プロセスに切換える
15    setPri(r); // 割り込み状態を復元する
16}

```

プロセス同期

34 / 38

## TacOS の PCB リスト操作関数 (カーネル内)

```

1 // プロセスキューでp1の前にp2を挿入する p2 -> p1
2 void insProc(PCB p1, PCB p2) {
3     p2.next=p1;
4     p2.prev=p1.prev;
5     p1.prev=p2;
6     p2.prev.next=p2;
7 }
8
9 // プロセスキュー(実行可能列やセマフォの待ち行列)で p を削除する
10 void delProc(PCB p) {
11     p.prev.next=p.next;
12     p.next.prev=p.prev;
13}

```

プロセス同期

35 / 38

## TacOS の V 操作ルーチン (1/2) (カーネル内)

```

1 // ディスパッチを発生しないセマフォのV操作
2 // (V 操作をしたあとまだ仕事があるとき使用する)
3 // (kernel 内部専用、割り込み禁止で呼出す)
4 boolean iSemV(int sd) {
5     if (sd<0 || SEM_MAX<=sd || !semInUse[sd]) { // 不正なセマフォ番号
6         panic("iSemV(%d)", sd);
7     }
8     boolean ret = false; // 起床するプロセスなし
9     Sem s = semTbl[sd]; // 操作するセマフォ
10    PCB q = s.queue; // 待ち行列の番兵
11    PCB p = q.next; // 待ち行列の先頭プロセス
12    if (p==q) { // 待ちプロセスが無いなら
13        s.cnt = s.cnt + 1; // カウンタを足す
14    } else { // 待ちプロセスがあるなら
15        delProc(p); // 待ち行列から外す
16        p.stat = P_RUN; // 実行可能に変更
17        schProc(p); // 実行可能列に登録
18        ret = true; // 起床するプロセスあり
19    }
20    return ret; // 実行可能列に変化があった
21}

```

- iSemV() は割り込み禁止で呼び出す。

プロセス同期

36 / 38

## TacOS の V 操作ルーチン（2/2）（カーネル内）

```

23 // セマフォの V 操作
24 // 待ちプロセス無し : カウンタを1増やす
25 // 待ちプロセス有り : 待ち行列からプロセスを1つ外して実行可能にした後、
26 //                      ディスパッチャを呼び出す
27 public void semV(int sd) {
28     int r = setPri(DI|KERN);           // 割り込み禁止、カーネル
29     if (iSemV(sd)) {                   // V 操作し必要なら
30         yield();                       // プロセスを切り替える
31     }
32     setPri(r);                         // 割り込み状態を復元する
33 }

```

- iSemV() を呼び出す前に割り込み禁止にする。
- iSemV() が true で返ったらプロセスの切換えを試みる。
- yield() でプリエンプシオンしたプロセスは、yield() から実行が再開される。

プロセス同期

37 / 38

## TacOS の CPU フラグ操作関数（カーネル内）

```

1 ;; CPU のフラグの値を返すと同時に新しい値に変更
2 _setPri
3     ld     g0,2,sp    ; 引数の値を G0 に取り出す
4     push   g0         ; 新しい状態をスタックに積む
5     ld     g0,flag    ; 古いフラグの値を返す準備をする
6     reti                    ; reti は FLAG と PC を同時に pop する

```

- CPU の PSW のフラグに割り込み禁止ビットがある。
- C--言語から setPri() 関数として呼び出せるようにするには、アセンブリ言語プログラムで \_setPri ラベルを宣言する必要がある。
- C--言語プログラムは引数をスタックに積んで関数を CALL する。
- アセンブリ言語プログラムで引数を参照するには、(SP 相対で) スタックから取り出す。(SP+0 番地が PC, SP+2 番地が第1引数)
- 関数の返り値は、G0 レジスタに入れて返す。
- reti 命令はスタックからフラグと PC を一度に取り出す。

プロセス同期

38 / 38

## オペレーティングシステム 第6章 プロセス間通信

<https://github.com/tctsigemura/OSTextBook>

プロセス間通信

1 / 27

### プロセス間通信の必要性

#### プロセス間通信 (IPC : Inter-Process Communication)

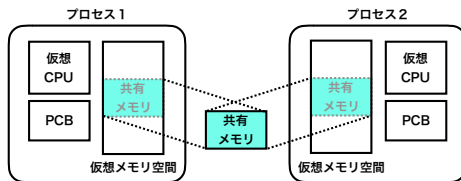
複数のプロセスが情報を共有し協調して処理を進めることができる。  
次のようなメリットが期待できる。

- 複数のプロセスが共通の情報へアクセスすることができる。
  - 並列処理による処理の高速化ができる可能性がある。
  - システムを見通しの良いモジュール化された構造で構築できる。
- プロセス間で情報を共有する代表的な機構として、**共有メモリとメッセージ通信**がある。

プロセス間通信

2 / 27

### 共有メモリ



- 同じ物理メモリを複数のプロセスの仮想メモリ空間に貼り付ける。
- MMU (Memory Management Unit) の働きで可能になる。
- 貼り付けが終わればシステムコールなしでデータ交換可能。
- プロセス間の同期機構は他に必要。

プロセス間通信

3 / 27

### UNIX の共有メモリシステムコール等 (前半)

共有メモリなどの識別に使用するキーを生成 (ライブラリ)  
`key_t ftok(const char *path, int id);`  
 返り値 : 引数から作成されるキー値  
`path` : 実際に存在するファイルのパス  
`id` : キーの作成に使用する追加の情報 (同じ`path`から異なるキーを作る)

共有メモリセグメントの作成 (システムコール)  
`int shmget(key_t key, size_t size, int flag);`  
 返り値 : 共有メモリセグメントID  
`key` : キー  
`size` : セグメントサイズ (バイト単位)  
`flag` : 作成フラグとモード

- `ftok()` は、`path` と `id` から一意な `key` 値を生成する。
- `shmget()` は、`key` 値で識別されるメモリセグメント ID を返す。
- `shmget()` は、メモリセグメントを作ることもできる。
- `flag` は、`rw-rw-rwx` と `IPC_CREAT` 等のフラグ

プロセス間通信

4 / 27

### UNIX の共有メモリシステムコール等 (後半)

共有メモリセグメントをプロセスの仮想アドレス空間に貼り付ける (システムコール)  
`void *shmat(int shmId, void *addr, int flag);`  
 返り値 : 共有メモリセグメントを配置したアドレス  
`shmId` : 共有メモリセグメントID  
`addr` : 貼り付けるアドレス (NULL(0)は、カーネルに任せる)  
`flag` : 貼り付け方法等

共有メモリセグメントをプロセスの仮想アドレス空間から取り除く (システムコール)  
`int shmdt(void *addr);`  
 返り値 : 0=正常, -1=エラー  
`addr` : 取り除く共有メモリセグメントのアドレス

共有メモリセグメントの制御 (システムコール)  
`int shmctl(int shmId, int cmd, struct shm_ds *buf);`  
 返り値 : 0=正常, -1=エラー  
`shmId` : 共有メモリセグメントID  
`cmd` : 削除(IPC\_RMID)等のコマンド  
`buf` : コマンドのパラメータ

プロセス間通信

5 / 27

### UNIX の共有メモリサーバ例 (前半)

```
1 // 共有メモリサーバ (ipcUnixSharedMemoryServer.c) : 共有メモリからデータを読みだし表示
2 #include <stdio.h>
3 #include <stdlib.h>
4 #include <string.h>
5 #include <unistd.h>
6 #include <sys/types.h>
7 #include <sys/ipc.h>
8 #include <sys/shm.h>
9 #define SHMSZ 512 // 共有メモリのサイズ
10 int main() {
11     key_t key=ftok("shm.dat", 'R'); // キーを作る
12     if (key==-1) { // エラーチェック
13         perror("shm.dat");
14         exit(1);
15     }
16     int shmId=shmget(key, SHMSZ, IPC_CREAT|0666); // 共有メモリを作る
17     if (shmId<0) { // エラーチェック
18         perror("shmget");
19         exit(1);
20     }
21 }
```

プロセス間通信

6 / 27

## UNIX の共有メモリサーバ例 (後半)

```

21 char *data=shmat(shmid,NULL,0);           // 共有メモリを貼り付ける
22 if (data==(char *)-1) {                   // エラーチェック
23     perror("shmat");
24     exit(1);
25 }
26 strcpy(data, "initialization...\n");      // 共有メモリに書き込む
27 while(1) {                                // 共有メモリの内容を表示
28     printf("sheared memory:%s",data);    // 5秒に1度メモリを表示
29     if (strcmp(data, "end\n") == 0) break; // "end"なら終了
30     sleep(5);
31 }
32 if (shmdt(data) == -1){                   // 共有メモリをアドレス空間
33     perror("shmdt");                      // と切り離す
34     exit(1);
35 }
36 if (shmctl(shmid, IPC_RMID, 0) == -1){    // 共有メモリを廃棄する
37     perror("shmctl");
38     exit(1);
39 }
40 return 0;
41 }

```

プロセス間通信

7 / 27

## UNIX の共有メモリクライアント例 (前半)

```

1 // 共有メモリクライアント (ipcUnixSharedMemoryClient.c) : 共有メモリにデータを書き込む
2 #include <stdio.h>
3 #include <stdlib.h>
4 #include <errno.h>
5 #include <sys/types.h>
6 #include <sys/ipc.h>
7 #include <sys/shm.h>
8 #define SHMSZ 512                          // メモリのサイズ
9 int main() {
10     int shmid;
11     key_t key;
12     char *data, *s;
13     if ((key=ftok("shm.dat", 'R')) == -1) { // サーバ側と同じキーを作る
14         perror("shm.dat");
15         exit(1);
16     }

```

プロセス間通信

8 / 27

## UNIX の共有メモリクライアント例 (後半)

```

17 if ((shmid=shmget(key,SHMSZ,0666))<0) { // 共有メモリを取得する
18     perror("shmget");
19     exit(1);
20 }
21 data=shmat(shmid,NULL,0);               // 共有メモリを貼り付ける
22 if (data == (char *)-1) {               // エラーチェック
23     perror("shmat");
24     exit(1);
25 }
26 printf("Enter a string: ");
27 fgets(data,SHMSZ,stdin);               // 共有メモリに直接入力する
28 if (shmdt(data)==-1){                   // 共有メモリをメモリ空間と
29     perror("shmdt");                     // 切り離す
30     exit(1);
31 }
32 return 0;
33 }

```

プロセス間通信

9 / 27

## UNIX の共有メモリプログラム実行例

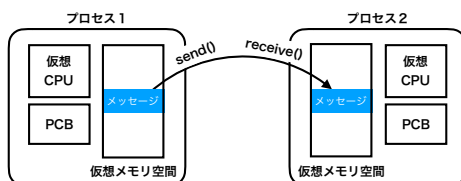
| [Terminal No.1]                  | [Terminal No.2]                  |
|----------------------------------|----------------------------------|
| \$ ./ipcUnixSharedMemoryServer   | \$ ./ipcUnixSharedMemoryClient   |
| sheared memory:initialization... | sheared memory:initialization... |
| sheared memory:initialization... | Enter a string: abcdefg          |
| sheared memory:abcdefg           | sheared memory:abcdefg           |
| sheared memory:abcdefg           | \$ ./ipcUnixSharedMemoryClient   |
| sheared memory:abcdefg           | Enter a string: 1234567          |
| sheared memory:1234567           | \$ ./ipcUnixSharedMemoryClient   |
| sheared memory:1234567           | Enter a string: end              |
| sheared memory:end               | \$                               |

- このプログラムは相互排除をやっていない。
- このプログラムは**使用してはならない**。

プロセス間通信

10 / 27

## メッセージ通信

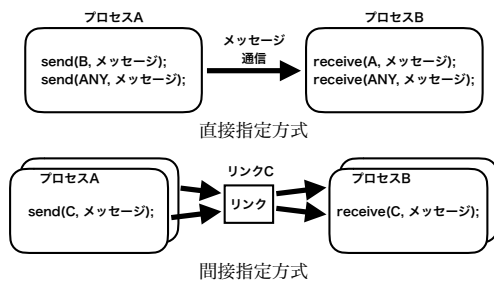


- send() システムコールでメッセージを送信する。
- receive() システムコールでメッセージを受信する。
- メッセージ通信は同期機構も含んでいる。

プロセス間通信

11 / 27

## 通信相手の指定方式 (Naming)



- 直接指定方式でも ANY を用いることで多対多通信が可能。
- 間接指定方式は自然に多対多通信が可能

プロセス間通信

12 / 27

## 通信方式

一般に

- バッファリング (あり／なし)
- メッセージ長 (固定／可変)
- メッセージ形式 (タグあり／なし)
- 同期方式
  - 非同期方式 (ノンブロッキング)
  - 同期方式 (ブロッキング)
  - ランデブー方式 (クライアント・サーバモデルに特化)

UNIX の場合は

- 間接指定方式
- バッファリング=あり
- メッセージ長=可変長
- メッセージ形式=タグあり
- 同期方式／非同期方式どちらも可能

プロセス間通信

13 / 27

## UNIX のメッセージ通信システムコールなど (前半)

```

メッセージ構造体 (以下の構造体を自分で宣言して使用する)
struct msgbuf {
    long mtype;        // メッセージの型
    char mtext[N];     // メッセージの本体 (Nバイト)
};

```

```

メッセージキューのIDを返す。
int msgget(key_t key, int msgflg); (システムコール)
返り値: メッセージキューID
key    : キー (ftok()で作成したもの)
msgflg : IPC_CREAT等のフラグとアクセス許可ビット

```

- mtype がタグの役割を持つ。
- key は共有メモリで紹介したものと同一 (ftok() 関数で作る)。
- 「メッセージキュー」= 「リンク」
- msgsnd(), msgrcv() で msgflg に IPC\_NOWAIT を指定すると非同期。

プロセス間通信

14 / 27

## UNIX のメッセージ通信システムコールなど (後半)

```

int msgsnd(int msqid, const void *msgp, size_t msgsz, int msgflg);
返り値: 0=正常, -1=エラー
msqid  : メッセージキューID
msgp   : メッセージ構造体のポインタ
msgsz  : メッセージ本体のバイト数
msgflg : IPC_NOWAIT等のフラグ

```

メッセージキューからメッセージを受信する (システムコール)

```

int msgrcv(int msqid, const void *msgp, size_t msgsz, long msgtyp, int msgflg);
返り値: -1=エラー、受信したメッセージの本体バイト数
msqid  : メッセージキューID
msgp   : メッセージ構造体のポインタ
msgsz  : メッセージ本体の最大バイト数
msgtyp : 受信するメッセージの型
msgflg : IPC_NOWAIT等のフラグ

```

メッセージキューの制御 (システムコール)

```

int msgctl(int msqid, int cmd, struct msgid_ds *buf);
返り値: -1=エラー、0<=コマンドにより異なる
msqid  : メッセージキューID
cmd    : 削除 (IPC_RMID) 等のコマンド
buf    : コマンドのパラメータ

```

プロセス間通信

15 / 27

## UNIX のメッセージ通信プログラム例 1

```

1 // ipcUnixMessage.h : メッセージ構造体の宣言
2 #define MAXMSG 100 // メッセージ本体の長さ
3 struct msgBuf {    // メッセージ格納用構造体
4     long mtype;    // メッセージの型
5     char mtext[MAXMSG]; // メッセージの本体
6 };

```

```

1 // メッセージ送信プログラム (ipcUnixMessageWriter.c): メッセージキューを作成し送信する
2 #include <stdio.h>
3 #include <stdlib.h>
4 #include <sys/types.h>
5 #include <sys/ipc.h>
6 #include <sys/msg.h>
7 #include "ipcUnixMessage.h" // msgBuf 構造体の宣言
8 int main() {
9     struct msgBuf buf; // メッセージ領域
10    int msqid; // メッセージキュー ID
11    key_t key; // メッセージキューの名前
12    if ((key=ftok("msgq.dat", 'b'))==-1) { // ftok はファイル名から
13        perror("ftok"); // 重複のない名前(キー)を
14        exit(1); // 生成する
15    }

```

プロセス間通信

16 / 27

## UNIX のメッセージ通信プログラム例 2

```

16 if ((msqid=msgget(key,0644|IPC_CREAT))==-1) { // メッセージキューを作る
17     perror("msgget");
18     exit(1);
19 }
20 printf("Enter lines of text, ^D to quit:\n");
21 buf.mtype = 1; // メッセージの型
22 while (fgets(buf.mtext,MAXMSG,stdin)!=NULL) { // キーボードから1行入力
23     if (msgsnd(msqid,&buf,MAXMSG,0)==-1) { // メッセージを送信
24         perror("msgsnd");
25         break;
26     }
27 }
28 if (msgctl(msqid,IPC_RMID,NULL) == -1) { // メッセージキューを削除
29     perror("msgctl");
30     exit(1);
31 }
32 exit(0);
33 }

```

プロセス間通信

17 / 27

## UNIX のメッセージ通信プログラム例 3

```

1 // メッセージ受信プログラム (ipcUnixMessageReader.c): メッセージキューから受信する
2 #include <stdio.h>
3 #include <stdlib.h>
4 #include <sys/types.h>
5 #include <sys/ipc.h>
6 #include <sys/msg.h>
7 #include "ipcUnixMessage.h"
8 int main() {
9     struct msgBuf buf;
10    int msqid;
11    key_t key;
12    if ((key=ftok("msgq.dat", 'b'))==-1) { // 送信側と同じキーを作る
13        perror("ftok");
14        exit(1);
15    }
16    if ((msqid=msgget(key,0644))==-1) { // ipcUnixMessageWriter が作った
17        perror("msgget"); // メッセージキューを取得
18        exit(1);
19    }

```

プロセス間通信

18 / 27

## UNIXのメッセージ通信プログラム例4

```

20 printf("ready to receive messages.\n");
21 for(;;) {
22     if (msgrcv(msgid,&buf,MAXMSG,0,0)==-1) { // 先頭のメッセージを読み出す
23         perror("msgrcv"); // メッセージキューが削除され
24         exit(1); // エラーが発生したら終了
25     }
26     printf("%ld:%s",buf.mtype,buf.mtext); // 受信したメッセージを表示
27 }
28 exit(0);
29 }

```

```

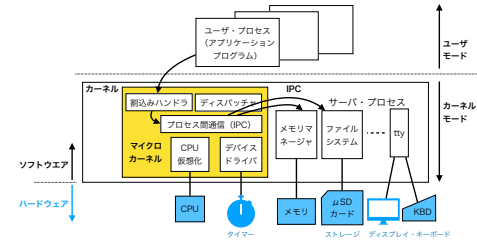
[Terminal No.1] | [Terminal No.2]
$ ./ipcUnixMessageWriter | $ ./ipcUnixMessageReader
Enter lines of text, ^D to quit: | ready to receive messages.
abcdeffg | 1:abcdeffg
1234567 | 1:1234567
^D | msgrcv: Identifier removed
$ | $

```

プロセス間通信

19 / 27

## 実装例

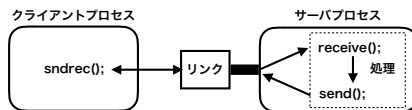
第21章  
TacOSのメッセージ通信

プロセス間通信

20 / 27

## TacOSのメッセージ通信機構

クライアント・サーバモデルに特化したランデブー方式のプロセス間通信機構を提供する。



1. サーバプロセスがリンクを所有し通信を待ち受ける。
2. クライアントプロセスはsndrec()関数でメッセージを送信する。
3. サーバプロセスはreceive()関数を用いてメッセージを受信する。
4. サーバプロセスはメッセージの内容に合った処理を行う。
5. サーバプロセスは処理結果をsend()関数を用いて返信する。
6. sndrec()関数が完了しクライアントは処理結果を受取る。

プロセス間通信

21 / 27

## TacOSのリンク構造体

```

1 #define LINK_MAX 5 // リンクは最大 5 個
2
3 struct Link { // リンクを表す構造体
4     PCB server; // リンクを所持するサーバ
5     PCB client; // リンクを使用中のクライアント
6     int s1; // サーバがメッセージ受信待ちに使用するセマフォ
7     int s2; // クライアント同士が相互排除に使用するセマフォ
8     int s3; // クライアントがメッセージ返信待ちに使用するセマフォ
9     int op; // メッセージの種類
10    int prm1; // メッセージのパラメータ1
11    int prm2; // メッセージのパラメータ2
12    int prm3; // メッセージのパラメータ3
13 };

```

- リンクはサーバが所有する。
- セマフォを用いて相互排除と同期を行う。
- リンクに書き込めるメッセージの形式は固定。

プロセス間通信

22 / 27

## TacOSのリンク作成ルーチン

```

1 Link[] linkTbl = array(LINK_MAX); // リンクの一覧表
2 int linkID = -1; // リンクの通し番号
3
4 // リンクを生成する (サーバが実行する)
5 public int newLink() {
6     int r = setPri(DI[KERN]); // 割り込み禁止、カーネル
7     linkID = linkID + 1; // 通し番号を進める
8     #ifdef DEBUG
9     printf("newLink:ID=%d,SERVER=%d\n",linkID,curProc.pid);
10    #endif
11    if (linkID >= LINK_MAX) // リンクが多すぎる
12        panic("newLink");
13
14    Link l = linkTbl[linkID]; // 新しく割り当てるリンク
15    l.server = curProc; // リンクの所有者を記憶
16    l.s1 = newSem(0); // server が受信待ちに使用
17    l.s2 = newSem(1); // client が相互排除に使用
18    l.s3 = newSem(0); // client が返信待ちに使用
19    setPri(r); // 割り込み復元
20    return linkID; // 割当てたリンクの番号
21 }

```

- 割り込み禁止による相互排除を行っている。

プロセス間通信

23 / 27

## TacOSのメッセージ通信ルーチン (サーバ用)

```

1 // サーバ用の待ち受けルーチン
2 public Link receive(int num) {
3     Link l = linkTbl[num];
4     if (l.server != curProc) panic("receive"); // 登録されたサーバではない
5     semP(l.s1); // サーバをブロック
6     return l;
7 }
8
9 // サーバ用の送信ルーチン
10 public void send(int num, int res) {
11     Link l = linkTbl[num];
12     if (l.server != curProc) panic("send"); // 登録されたサーバではない
13     l.op = res; // 処理結果を書込む
14     semV(l.s3); // クライアントを起こす
15 }

```

- receive() はリンクにメッセージが届くのを待つ。
- receive() はメッセージが書き込まれたリンクを返す。
- send() はリンクに処理結果 (16bit) を返信する。

プロセス間通信

24 / 27

## TacOS のメッセージ通信使用例 (サーバ側)

```

1 // プロセスマネージャサーバのメインルーチン
2 public void pmMain() {
3     pmLink = newLink();                // リンクを生成する
4     while (true) {                    // システムコールを待つ
5         Link l = receive(pmLink);      // システムコールを受信
6         int r = pmSysCall(l.op, l.prm1, l.prm2, l.prm3, l.client); // システムコール実行
7         send(pmLink, r);               // 結果を返す
8     }
9 }

```

- プロセスマネージャ (サーバプロセス) の例。
- サーバはリンクを作成した後、受信、処理、返信を繰り返す。
- receive() を用いてリンクからメッセージを受信。
- pmSysCall() がプロセスマネージャの処理ルーチン。
- send() を用いてクライアントに処理結果を返す。

プロセス間通信

25 / 27

## TacOS のメッセージ通信ルーチン (クライアント用)

```

1 // クライアント用メッセージ送受信ルーチン
2 public int sndrec(int num, int op, int prm1, int prm2, int prm3) {
3     Link l = linkTbl[num];           // 他のクライアントと相互
4     semP(l.s2);                       // 排除しリンクを確保
5     l.client = curProc;               // リンク使用中プロセス記録
6     l.op = op;                        // メッセージを書込む
7     l.prm1 = prm1;
8     l.prm2 = prm2;
9     l.prm3 = prm3;
10    int r = setPri(DI|KERN);           // 割り込み禁止、カーネル
11    iSemV(l.s1);                       // サーバを起こす
12    semP(l.s3);                       // 返信があるまでブロック
13    setPri(r);                         // 割り込み復元
14    int res = l.op;                   // 返信を取り出す
15    semV(l.s2);                       // リンクを解放
16    return res;
17 }

```

- iSemv() を使用するので割り込み禁止による相互排除が必要。
- クライアント間での相互排除にセマフォ s2 を使用。

プロセス間通信

26 / 27

## TacOS のメッセージ通信使用例 (クライアント側)

```

1 public int exec(char[] path, char[] argv) {
2     int r = sndrec(pmLink, EXEC, _Atoi(path), _Atoi(argv), 0);
3     return r;                         // 新しい子の PID を返す
4 }

```

- exec システムコールを例にする。
- exec は path のプログラムを新しいプロセスで実行する。
- 上のプログラムは SVC 割り込みハンドラから呼出される。
- SVC 割り込みハンドラはシステムコールの種類を判断し、exec システムコールの場合に上のルーチンを呼出す。
- 割り込みハンドラはプロセスのコンテキストで実行される。
- exec システムコールはプロセスマネージャが処理する。
- プロセスマネージャへのメッセージ通信により処理を依頼する。
- \_Atoi() 関数は参照 (アドレス) を int 型に変換する。

プロセス間通信

27 / 27

## オペレーティングシステム 第 7 章 モニタ

<https://github.com/tctsigemura/OSTextBook>

モニタ

1 / 12

### モニタ (Monitor) の概要

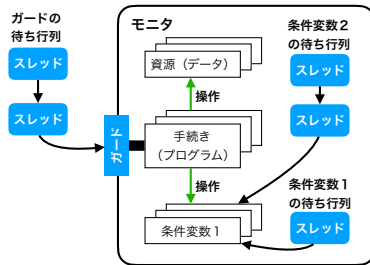
プロセス間同期のために使用できる高級言語の機能のこと。  
モニタは**抽象データ型** (Java 言語のクラスのようなもの) である。

- プログラマが定義できる型である。(抽象データ型で一般的)
- データと操作をまとめて定義する。(抽象データ型で一般的)
- 同期のための機能が組込まれている。(モニタ独特)

モニタ

2 / 12

### モニタの模式図



- 資源 (データ, 変数), 手続き (メソッド), ガード, 条件変数
- 図では省略してあるが**初期化プログラム**

モニタ

3 / 12

### モニタの構成要素

**資源 (データ, 変数)** 複数のスレッドによって共有される変数のことである。モニタの外から直接アクセスすることはできない。

**手続き (操作, メソッド)** 外部から呼び出されるプログラムである。手続きの実行は**ガード**の働きにより排他的に行われ、同時に実行される手続きは必ず一つ以内である。

**ガード** モニタに一つのガードが存在し、手続きを排他的に実行するために用いられる。

**条件変数** 条件変数には wait と signal の二つの操作ができる。wait 操作を行ったスレッドは**ガードを外して**条件変数の待ち行列に入る。signal 操作は条件変数の待ち行列から一つのスレッドを選んで実行可能にする。

**初期化プログラム** モニタのインスタンスを作成する時に、初期化のために実行されるプログラムである。

モニタ

4 / 12

### モニタによる相互排除の実現 (仮想言語)

```

1 // 銀行口座の残高を管理するモニタ
2 monitor MonAccount {
3   // 資源
4   int money; // スレッド間の共有変数 (残高)
5   // 初期化プログラム
6   MonAccount(int m) {
7     money = m; // 口座の残高を初期化する
8   }
9   // 手続き
10  public void receive(int r) { // 入金手続き
11    money = money + r;
12  }
13  public void pay(int p) { // 引落し手続き
14    money = money - p;
15  }
16 }

```

- 資源はスレッド間の共有変数 money である。
- 初期化プログラムはモニタのインスタンス生成時に実行される。
- 手続き (クリティカルセクション) は、ガードにより自動的に相互排除される。

モニタ

5 / 12

### モニタによる相互排除の利用 (仮想言語)

```

1 class MonAccountMain {
2   static MonAccount account = new MonAccount(0); // 残高0円の口座を作る
3   static void receiveThread() { // 入金管理スレッド
4     for ( ; ; ) {
5       int receipt = receiveMoney(); // ネットワークから入金を受信
6       account.receive(receipt); // 口座に入金する
7     }
8   }
9   static void payThread() { // 引落し管理スレッド
10    for ( ; ; ) {
11      int payment = payMoney(); // ネットワークから支払いを受信
12      account.pay(payment); // 口座から引落す
13    }
14  }
15  public static void main(String[] args) {
16    入金管理スレッドを起動;
17    引落し管理スレッドを起動;
18  }
19 }

```

- モニタのインスタンスを生成し、それに対して操作する。
- ガードは自動的なので排他忘れが無い。

モニタ

6 / 12

## セマフォを用いた相互排除問題の解 (参考)

```

int    account;           // スレッド間の共有変数 (残高)
Semaphore accSem = 1;     // 初期値1のセマフォ accSem (accountのロック用)
void receiveThread() {
    for ( ; ; ) {         // 入金管理スレッド
        // 入金管理スレッドは以下を繰り返す
        int receipt = receiveMoney(); // ネットワークから入金を受信する
        P( &accSem );           // account変数をロックするための P 操作
        account = account + receipt; // account変数を変更する (クリティカルセクション)
        V( &accSem );           // account変数をロック解除するための V 操作
    }
}
void payThread() {
    for ( ; ; ) {         // 引渡し管理スレッド
        // 引渡し管理スレッドは以下を繰り返す
        int payment = payMoney(); // ネットワークから支払い額を受信する
        P( &accSem );           // account変数をロックするための P 操作
        account = account - payment; // account変数を変更する (クリティカルセクション)
        V( &accSem );           // account変数をロック解除するための V 操作
    }
}

```

- P 操作と V 操作の使用はプログラマまかせ。
- 間違うとタイミング依存の発見の難しいバグになる。

モニタ

7 / 12

## モニタによるキューの実現 (仮想言語、前半)

```

1  monitor BoundedBuffer {
2      // 資源 (リングバッファ)
3      int N;
4      int[] buf;
5      int first, last, cnt;
6      // 条件変数
7      Condition empty;
8      Condition full;
9      // 初期化
10     BoundedBuffer(int n) {
11         N = n;
12         buf = new int[N];
13         first = last = cnt = 0;
14     }

```

- この例ではリングバッファのデータ構造が資源である。
- 資源はモニタ外部から直接アクセスすることはできない。
- 条件変数 empty はキューが空の時、消費者スレッドが待合せに使用。
- 条件変数 full はキュー満杯時に、生産者スレッドが待合せに使用。

モニタ

8 / 12

## モニタによるキューの実現 (仮想言語、後半)

```

15 // 手続き
16 public void append(int x) { // (1)
17     if (cnt==N) full.wait(); // (1)
18     buf[last] = x;         // (3)
19     last = (last + 1) % N; // (3)
20     cnt++;                 // (3)
21     empty.signal();        // (3)
22 }
23 public int remove() { // (2)
24     if (cnt==0) empty.wait(); // (2)
25     int x = buf[first]; // (2)
26     first = (first + 1) % N; // (2)
27     cnt--;              // (2)
28     full.signal();      // (2)
29     return x;           // (4)
30 }
31 }

```

- wait() はスレッドを状態変数の待ち行列に入れる。
- signal() は状態変数の待ちスレッドを一つ起こす。
- signal() で起床したスレッドは**ただちに**実行される。

モニタ

9 / 12

## モニタによる複数生産者と複数消費者問題の解 (仮想言語)

```

1  class BoundedBufferMain {
2      static BoundedBuffer queue = new BoundedBuffer(10); // 大きさ10のキュー
3      static void producer() { // 生産者スレッドが実行
4          while(true) {
5              int x = データを作る();
6              queue.append(x); // キューにデータを追加
7          }
8      }
9      static void consumer() { // 消費者スレッドが実行
10         while(true) { // キューから
11             int x = queue.remove(); // データを取り出す
12             データを使用する(x);
13         }
14     }
15     public static void main(String[] args) { // main から実行を開始
16         生産者スレッドを起動;
17         消費者スレッドを起動;
18     }
19 }

```

- モニタにより定義されたキューのインスタンスを使用した解

モニタ

10 / 12

## Java による複数生産者と複数消費者問題の解 (前半)

```

1  public class MonBoundedBuffer {
2      // 資源 (リングバッファ)
3      private int N;
4      private int[] buf;
5      private int first, last, cnt;
6      // 初期化
7      public MonBoundedBuffer(int n) {
8          N = n;
9          buf = new int[N];
10         first = last = cnt = 0;
11     }
12     // 手続き
13     private void await() {
14         try{wait();}catch(InterruptedException e){}
15     }

```

- 資源には private を明示
- 初期化はコンストラクタにより実現
- Java の wait() は try-catch 構文で使用する必要がある。
- 手続きは synchronized 修飾したメソッド

モニタ

11 / 12

## Java による複数生産者と複数消費者問題の解 (後半)

```

16     public synchronized void append(int x) { // (1)
17         while (cnt==N) await(); // (1)
18         buf[last] = x; // (3)
19         last = (last + 1) % N; // (3)
20         cnt++; // (3)
21         if (cnt==1) notify(); // (3)
22     }
23     public synchronized int remove() { // (2)
24         while (cnt==0) await(); // (2)
25         int x = buf[first]; // (2)
26         first = (first + 1) % N; // (2)
27         cnt--; // (2)
28         if (cnt==N-1) notify(); // (2)
29         return x; // (2)
30     }
31 }

```

- Java の wait() は別の理由でも終了するので while の中で使用
- Java の notify() は待ちスレッドを一つ起こす。
- 条件変数は一つしかないなので、利用方法に工夫が必要
- remove() 最後の行の実行順序がモニタと異なる。

モニタ

12 / 12

## オペレーティングシステム 番外 デッドロック

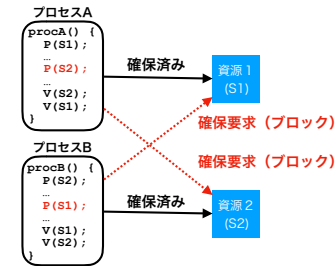
<https://github.com/tctsigemura/OSTextBook>

デッドロック

1 / 5

## デッドロック (deadlock) とは

資源を確保したい複数のプロセス (スレッド) があるとき、資源が解放されるのを待ち合う状態。



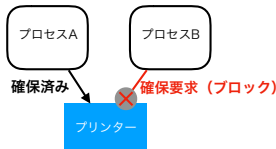
デッドロック

2 / 5

## デッドロックが発生する条件

デッドロックが発生するための必要条件

- (1) 相互排除 (資源が排他的に利用される)
  - (2) 確保待ち (資源を確保した状態で待つ)
  - (3) 横取り不可 (使用中の資源を横取りできない)
  - (4) 循環待ち (待ちがループしている)
- (1), (3) は資源の種類によってはどうしようもない。  
プリンタは、相互排除、横取り不可な資源の例。

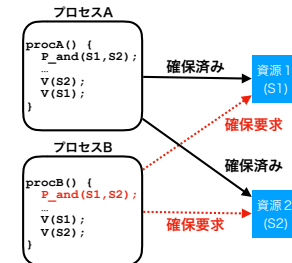


デッドロック

3 / 5

## デッドロックの発生を防止する方法 (1)

前出の (2), (4) のどちらかを防止すればデッドロックは発生しない。  
資源の一括確保 (P\_AND) ... (2) 確保待ちにしない。



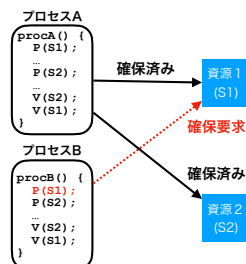
資源の利用率が悪くなる。

デッドロック

4 / 5

## デッドロックの発生を防止する方法 (2)

資源の確保順序に制約 ... (4) 循環待ちにしない。



不公平が生じるかも (順番付けできない例「食事する哲学者問題」)。

デッドロック

5 / 5

## オペレーティングシステム 第8章 主記憶 (メモリ)

<https://github.com/tctsigemura/OSTextBook>

主記憶

1 / 15

### 主記憶

主記憶はプログラム、データ、スタック等を置くメモリのこと  
CPU と同様に重要な装置

- TeC の 256 バイトの RAM
- H8/3664 の 32KiB の ROM と 2KiB の RAM
- PC のメモリ (4GiB ~ 16GiB 程度?)

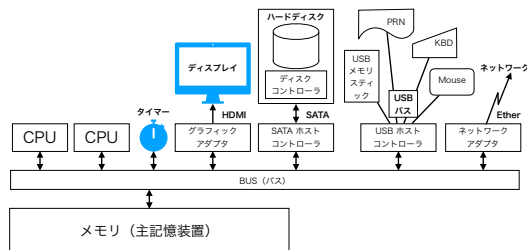
この章では、主記憶を管理し複数のプロセスに適切に割り振り、かつ、プロセス同士が干渉しないように分離する方法を学ぶ。

主記憶

2 / 15

### ハードウェア構成

メモリを共有する SMP (Symmetric Multiprocessing) システム

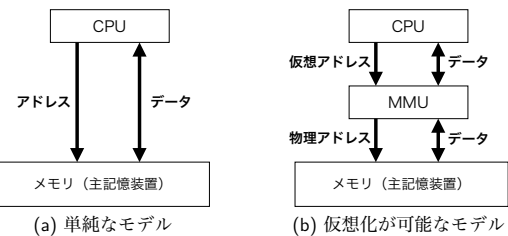


この講義で前提にしているコンピュータの構成

主記憶

3 / 15

### 本章で用いるモデル



MMU (Memory Management Unit : メモリ管理装置)

- メモリ保護機構, メモリ再配置機構, 仮想記憶
- 仮想アドレス, 物理アドレス

主記憶

4 / 15

### メモリ保護機構

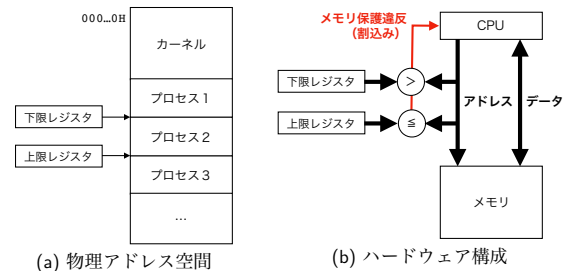
- CPU を仮想化した  
複数のプロセスを同時にロードし並列実行できるようになった
  - ユーザプロセスが複数存在する  
プロセスが他のプロセスや OS を破壊しないか?  
他のプロセスの**メモリ**を保護する機構が必要
- プロセスは自身に割当てられたメモリ以外をアクセスできないようにする。

主記憶

5 / 15

### 上限・下限レジスタ

- プロセスがアクセスしても良いアドレスの範囲を設定する。
- プロセスのメモリアクセスはアドレスをチェックする (ハード)
- レジスタを操作できるのはカーネルだけ。



(a) 物理アドレス空間

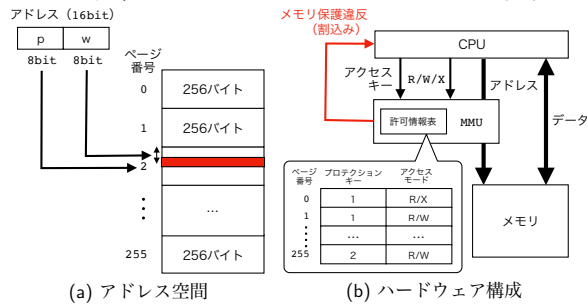
(b) ハードウェア構成

主記憶

6 / 15

## ロック/キー機構

- メモリをページに分割する (例: 64KiB を 256 ページに)
- ページ毎に、アクセスしてよいプロセスとアクセス方法を記録

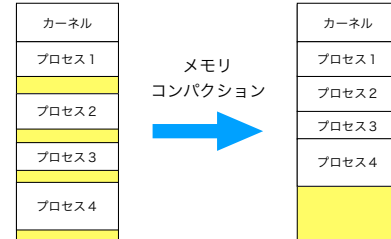


主記憶

7 / 15

## メモリフラグメントとコンパクション

- 様々なサイズのプロセスが存在する。
- プロセスの生成・終了が繰り返される。
- メモリフラグメント (断片) が沢山できる。
- フラグメントを解消するために動的再配置が必要!



主記憶

8 / 15

## 再配置可能オブジェクトファイル

ロード時に機械語に含まれるアドレスを確定

- コンパイル時にはロードアドレスが分からない。
- ロード時にアドレスを確定できる仕組みが必要
- 実行可能形式ファイルに機械語と再配置表を格納
- ロード時にプログラムやデータに含まれるアドレスを変更

例: 1234H 番地にロードされた場合

JMP 0100H の機械語は JMP 1334 に変更

主記憶

9 / 15

## 動的再配置に応用できるか?

この方法では動的再配置までは無理

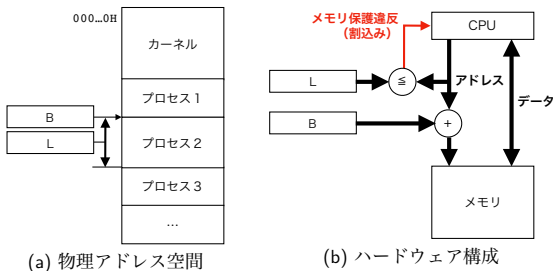
- CPU レジスタにアドレスがあるかも知れない。
- アドレスがスタックに PUSH されているかも。
- malloc() で確保した領域にも含まれるかも  
リスト構造の次のノードへのポインタなど

主記憶

10 / 15

## リロケーションレジスタ

- 動的再配置を可能にするハードウェア機構
- メモリ保護の機能も持つ。
- ロードアドレス (B:Base) と大きさ (L:Limit) を記録

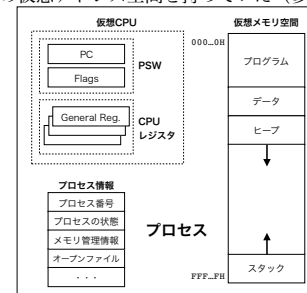


主記憶

11 / 15

## 仮想アドレス空間

プロセスは専用の仮想アドレス空間を持っていた (参考)

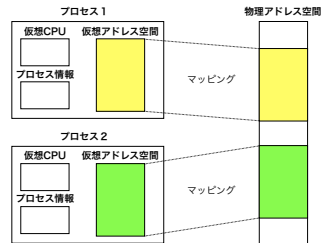


主記憶

12 / 15

## 主記憶の仮想化

- プロセス毎に独立した仮想アドレス空間を持つ.
- 仮想アドレスから物理アドレスにMMUがマッピングする.
- 通常は**多重仮想記憶方式**を用いる.

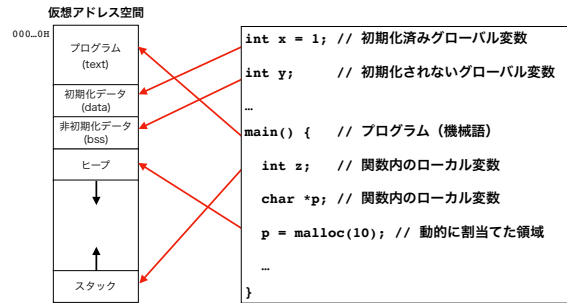


主記憶

13 / 15

## 仮想化アドレス空間の配置

- (伝統的な) UNIX の配置を C 言語プログラムと対比
- ヒープとスタックの間はどれだけ空けておくか?



主記憶

14 / 15

## 前のプログラムをアセンブリ言語に変換したもの

```

_x      DW      1      // int x = 1;      DW命令はdataセグメントに出力
_y      WS      1      // int y;          WS命令はbssセグメントに出力
_main   PUSH    FP      // void main() {   機械語命令は
LD      FP,SP      //                  textセグメントに出力
PUSH    G3         //      int z;
PUSH    G4         //      char *p;
LD      G0,#10     //
PUSH    G0         //
CALL    _malloc    //      p = malloc(10);
ADD     SP,#2      //
LD      G4,G0      //
POP     G4         //
POP     G3         //
POP     FP         //
RET                     // }

```

主記憶

15 / 15

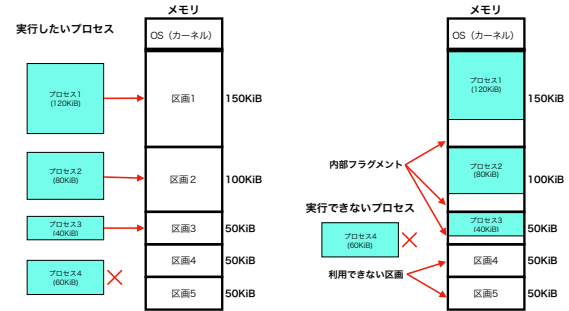
## オペレーティングシステム 第9章 メモリ割付け方式

<https://github.com/tctsigemura/OSTextBook>

メモリ割付け方式

1 / 18

### 固定区画方式



(a) 区画を選択しプロセスをロード (b) プロセスを実行  
予めメモリを大小数種類の区画に分割しておく。

メモリ割付け方式

2 / 18

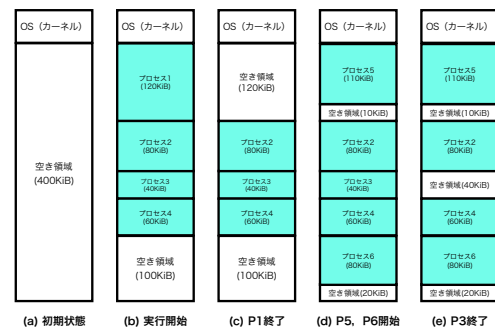
### 固定区画方式の特徴

- ① 空き領域の管理が容易である。
- ② 領域内部に無駄な領域（内部フラグメント）が生じる。
- ③ 小さな領域が複数空いていても大きなプロセスは実行できない。
- ④ 実行可能なプロセスのサイズに強い制約がある。  
(図の例では、151KiBのプロセスは実行できない。)
- ⑤ 同時に実行できるプロセスの数に制約がある。  
(図の例では、同時に五つ以上のプロセスは実行できない。)

メモリ割付け方式

3 / 18

### 可変区画方式

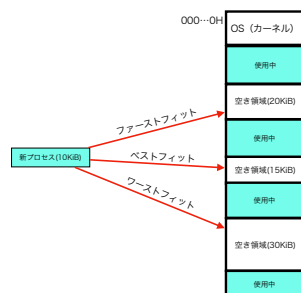


必要に応じて空き領域から区画を作る。  
外部フラグメントが生じる。

メモリ割付け方式

4 / 18

### 可変区画方式の領域選択方式



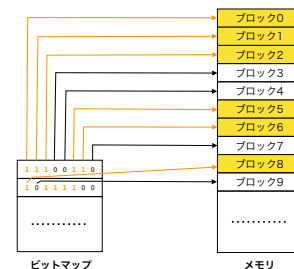
- ファーストフィット (first-fit) 方式：アドレス順にさがす。
- ベストフィット (best-fit) 方式：最小の領域を選択する。
- ワーストフィット (worst-fit) 方式：最大の領域を選択する。

メモリ割付け方式

5 / 18

### 空き領域の管理方式（ビットマップ方式）

どこに利用可能な空き領域があるかビットマップで管理する。



- メモリを一定の大きさのブロックに分割する。
- ビットマップの1ビットが1ブロックに対応する。

メモリ割付け方式

6 / 18

## ビットマップの大きさ

ビットマップの大きさを計算してみる。

- メモリサイズ: 8GiB
- ブロックサイズ: 4KiB
- ブロック数:  $8GiB \div 4KiB = (8 \times 2^{30}) \div (4 \times 2^{10}) = 2 \times 2^{20} = 2Mi$
- ビットマップのサイズ:  $(2 \times 2^{20}) \div 8 = 2^{18} = 256KiB$

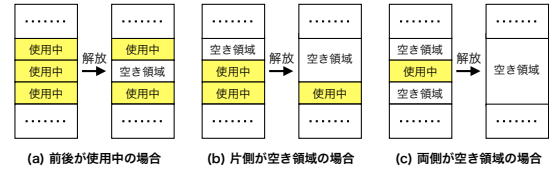
無視できるほど小さくはない。  
ビットマップを小さくするにはブロックサイズを大きくすればよい。  
内部フラグメントが大きくなる。

メモリ割付け方式

7 / 18

## 空き領域の管理方式 (リスト方式)

空き領域をリストにして管理する。  
様々なサイズの空き領域が混在しても良い。

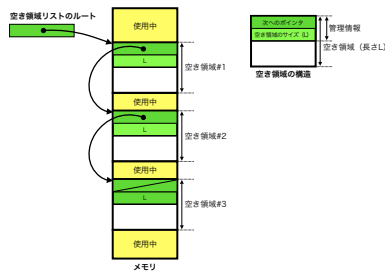


使用中の領域が解放されると隣接する空き領域と合体させる。

メモリ割付け方式

8 / 18

## 空き領域リストのデータ構造

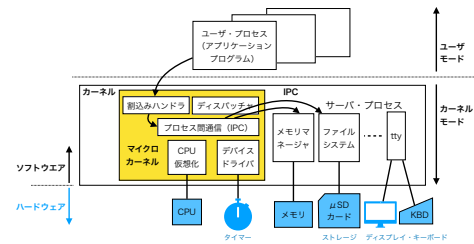


- 空き領域の一部を管理データの格納に使用する。
- アドレス順のリストにして管理する。
- ファーストフィットの探索に都合が良い。
- 隣接領域との合体にも都合が良い。

メモリ割付け方式

9 / 18

## 実装例

第22章  
TacOSのメモリ管理

メモリ割付け方式

10 / 18

## メモリ管理の実装例

TacOSのメモリ管理プログラムを実装例とする。

- 可変区画方式
- ファーストフィット
- OSがユーザプロセス領域の割当てに使用
- プロセスがヒープ領域を管理するプログラムも同じアルゴリズム

メモリ割付け方式

11 / 18

## データ構造の初期化

```

1  #define MBSIZE sizeof(MemBlk)           // MemBlk のバイト数
2  #define MAGIC (memPool)                 // 番兵のアドレスを使用する
3
4  // 空き領域はリストにして管理される
5  struct MemBlk {
6      MemBlk next;                       // 空き領域管理用の構造体
7      int size;                          // 次の空き領域アドレス
8  };                                     // 空き領域サイズ
9
10 //-----
11 // 初期化ルーチン
12 //-----
13 // メモリ管理の初期化
14 MemBlk memPool = {null, 0};             // 空き領域リストの番兵
15 public int _end();                       // カーネルの BBS 領域の最後
16
17 void mmInit() {
18     memPool.next = _ItoA(addrOf(_end)); // 空き領域
19     memPool.next.size = 0xe000 - addrOf(_end); // 空きメモリサイズ
20     memPool.next.next = null;
21 }

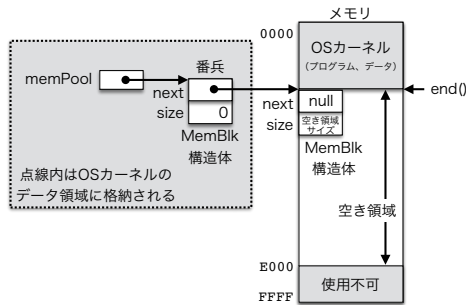
```

- memInit() はカーネル起動時に一度だけ実行される。

メモリ割付け方式

12 / 18

## 初期化直後のデータ構造



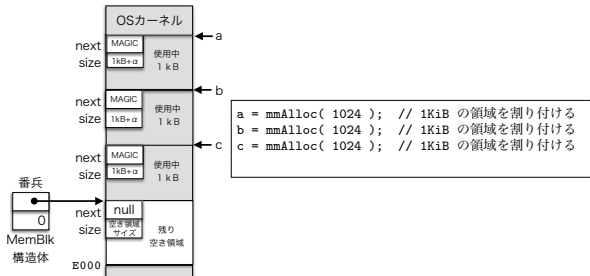
- `_end()` はカーネルサイズにより決まる。
- E000 より後ろはビデオメモリや IPL ROM がある。

## メモリ割付け方式

13 / 18

## メモリの割付け

右のプログラムで a, b, c を割付けたときのデータ構造



```
a = mmAlloc( 1024 ); // 1KiB の領域を割り付ける
b = mmAlloc( 1024 ); // 1KiB の領域を割り付ける
c = mmAlloc( 1024 ); // 1KiB の領域を割り付ける
```

MemBlk  
構造体

メモリ割付け方式

14 / 18

## メモリの割付けプログラム

```

2 // メモリを割り付ける
3 int memAlloc(int size) {
4     int s = (size + MSIZE + 1) <- 1; // メモリ割り当て
5     Memblk p = memPool; // 制御デンプク大きい領域に
6     Memblk m = p.next; // 直前の領域
7 // 対象となる領域
8 while (u_Cmp(m.size, s) < 0) {
9     p = m; // 領域が小さい間
10    m = m.next; // リストを子繰る
11    if (m == null) return 0; // メモリが不足する場合は
12 // エラーを表す null を返す
13 }
14 if (u_Cmp(m.size, s + MSIZE * 2) < 0) {
15     if (memPool.next == &m && m.next == null)
16         return 0; // 分ける領域がない領域サイズ
17 // リストの長さがゼロにならない
18 // による
19     p.next = m.next; // リスト
20 // 領域を分ける値があるから
21     Memblk n = _addrAdd(m, s); // 残り領域
22     n.next = m.next;
23     n.size = m.size - s;
24     p.next = n;
25     m.size = s;
26 }
27 n.next = MAGIC; // マグックナンバー格納
28 return _Atol(_addrAdd(m, MSIZE * 2)); // 管理領域を除いて返す

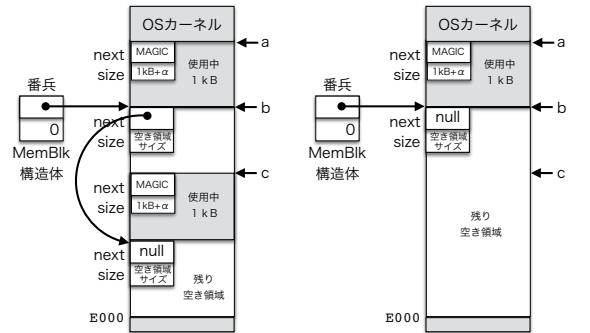
```

## メモリ割付け方式

15 / 18

## 領域の解放

### b, cを開放したときのデータ構造



## メモリ割付け方式

16 / 18

## メモリの解放プログラム (前半)

```

1 // メモリを解放する
2 int mmFree(void[] mem) { // 領域解放
3     MemBlk q = _addrAdd(mem, -MBSIZE); // 解放する領域
4     MemBlk p = memPool; // 直前の空き領域
5     MemBlk m = p.next; // 直後の空き領域
6
7     if (q.next!=MAGIC)
8         badaddr(); // 領域マジックナンバー確認
9
10    while (_aCmp(m, q)<0) { // 解放する領域の位置を探す
11        p = m;
12        m = m.next;
13        if (m==null) break;
14    }
15
16    void[] q1 = _addrAdd(q, q.size); // 解放する領域の最後

```

- 領域の本当の先頭アドレスを計算する。
- MAGICを確認する。
- 空き領域リストを辿り、挿入位置を決める。

## メモリ割付け方式

17 / 18

## メモリの解放プログラム（後半）

```

17 | void[] pl = _addrAdd(p, p.size); // 直前の領域の最後
18 |
19 | if (_aCmp(q,pl)<0 || m!=null&&_aCmp(m,ql)<0) // 未割り当て領域では？
20 |     badaddr();
21 |
22 | if (pl==q) { // 直前の領域に隣接している
23 |     p.size = p.size + q.size;
24 |     if (ql==m) { // 直後の領域とも隣接してる
25 |         p.size = p.size + m.size;
26 |         p.next = m.next;
27 |     }
28 |     else if (ql==m) { // 直後の領域に隣接している
29 |         q.size = q.size + m.size;
30 |         q.next = m.next;
31 |         p.next = q;
32 |     } else {
33 |         p.next = q;
34 |         q.next = m;
35 |     }
36 |     return 0;
37 | }

```

## メモリ割付け方式

18 / 18

## オペレーティングシステム 第10章 セグメンテーション

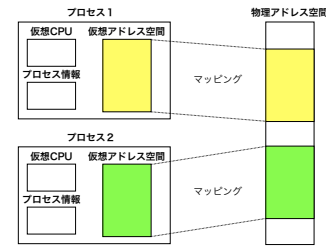
<https://github.com/tctsigemura/OSTextBook>

セグメンテーション

1 / 16

### リロケーションレジスタ方式 (復習)

プロセスの仮想アドレス空間を物理アドレス空間にマッピングする。



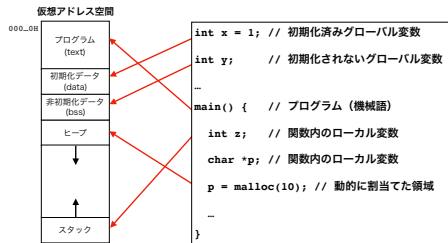
- プロセス毎に独立した仮想アドレスを持てる。
- 仮想アドレス空間はいつも0番地から開始できる。
- 動的なメモリコンパクションができる。

セグメンテーション

2 / 16

### リロケーションレジスタ方式の問題点 (1/2)

プロセスの仮想アドレス空間は次のような領域から構成された。



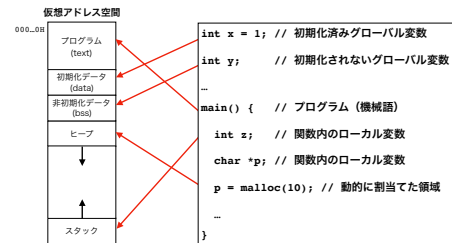
- 必要なメモリの見積もりが難しい。  
実行時にヒープ領域やスタック領域が不足する可能性がある。  
実行前に必要なメモリサイズを見積もる必要があり使い勝手が悪い。

セグメンテーション

3 / 16

### リロケーションレジスタ方式の問題点 (2/2)

プロセスの仮想アドレス空間は次のような領域から構成された。



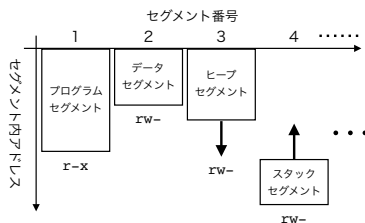
- 領域の性質に応じたメモリ保護ができない。
  - プログラム領域は読み出しと実行だけ許可する。
  - データ、ヒープ、スタック領域に読み出しと書き込みだけ許可する。

セグメンテーション

4 / 16

### セグメント

プロセスに複数のアドレス空間を持たせる。



- 複数持つことができるアドレス空間=セグメント
- 前の例で「プログラム」、「データ」等をセグメントにする。
- セグメント毎にサイズや保護モード (rwx) を決める。
- セグメント番号とセグメント内アドレスの二次元空間になった。  
仮想アドレス空間内の配置問題が解決!!

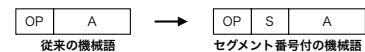
セグメンテーション

5 / 16

### セグメント番号

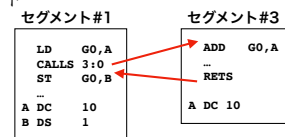
セグメント番号はどこから供給するのか?

- 機械語命令にセグメント番号を持たせる。



プログラムが大きくなる。

- カレントセグメント

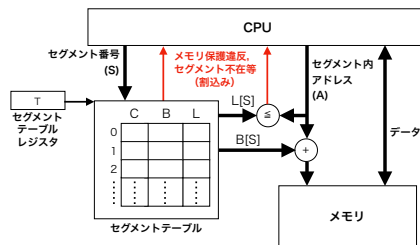


CALLS, RETS 命令でセグメントが自動的に切り換わる。  
IA-32 ではカレントセグメントが複数ある。  
(CS: プログラム, DS: データ, SS: スタックセグメント等)

セグメンテーション

6 / 16

## セグメンテーション機構 (1/3)



- CPU はセグメント番号 (S) とセグメント内アドレス (A) を出力
- セグメントテーブルを使用して物理アドレスに変換する。
- 本当は、セグメントテーブルはメモリ上に置く。
- セグメントテーブルレジスタはセグメントテーブルのアドレス

セグメンテーション

7 / 16

## セグメンテーション機構 (2/3)

## セグメントテーブル

- セグメント番号をインデクスにして S でエントリを選択する。
- B (Base) フィールドはセグメントのベースアドレス
- L (Limit) フィールドはセグメントのサイズ
- B と L はリローケーションレジスタと同じ。
- C (制御) フィールドは以下のビットを含む。

| 名称                       | ビット数 | 意味            |
|--------------------------|------|---------------|
| V (Valid)                | 1    | メモリにロードされている。 |
| D (Dirty)                | 1    | ロード後に変更された。   |
| RWX (Read/Write/Execute) | 3    | 許されるアクセス方法。   |

V=0 ならセグメント不在割込みを発生

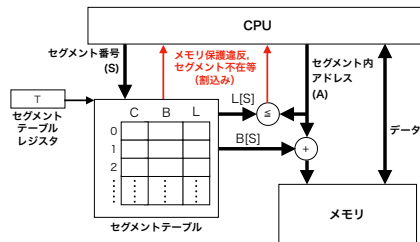
D=0 ならスワップアウトで書き戻しが不要

RWX はそのセグメントに許される操作 (メモリ保護違反割込み)

セグメンテーション

8 / 16

## セグメンテーション機構 (3/3)



## 物理アドレスへの変換

- 1 セグメントテーブルのエントリを読み出す。
- 2 C フィールド (V, RWX) をチェックする。
- 3 セグメント内アドレスをチェックする。
- 4  $B[S] + A$  が物理アドレスになる。

セグメンテーション

9 / 16

## セグメントテーブルエントリのキャッシング

## 物理アドレスへの変換

- メモリアクセスの度にセグメントテーブルを参照？
- メモリアクセス回数が二倍になる。
- メモリはいつも混み合っている (次のページ)。
- 一度参照したセグメントテーブルエントリは CPU または MMU にキャッシュするべきだ。
- IA-32 ではカレントセグメントレジスタに裏レジスタがある。

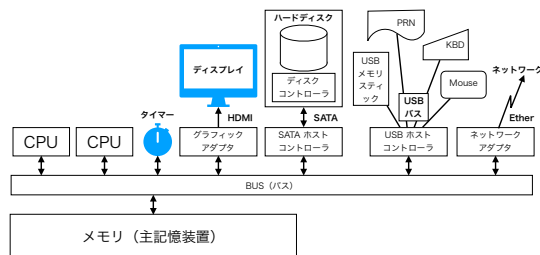
| セグメントレジスタ | 裏レジスタ       |
|-----------|-------------|
| CS セレクタ   | ベース リミット 属性 |
| DS セレクタ   | ベース リミット 属性 |
| SS セレクタ   | ベース リミット 属性 |
| ES セレクタ   | ベース リミット 属性 |
| FS セレクタ   | ベース リミット 属性 |
| GS セレクタ   | ベース リミット 属性 |

裏レジスタは自動的にロード、使用される。

セグメンテーション

10 / 16

## メモリはいつも混み合っている (参考)

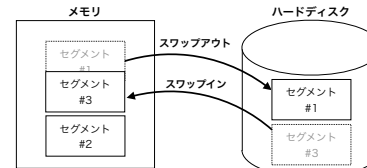


- 命令もデータも全てメモリにある。
- CPU も I/O 装置もメモリにアクセスする。
- フォン・ノイマン・ボトルネック (Von Neumann bottleneck)

セグメンテーション

11 / 16

## セグメンテーション機構による仮想記憶



全体がメモリに収まらない大きなプログラムも実行できる。

- 例えばセグメント 3 のサブルーチンを実行したい (CALL 3:0)
- セグメント 3 はセグメント不在なので OS に切り換わる。
- メモリに入り切らないのでセグメント 1 をスワップアウト
- メモリの空き領域にセグメント 3 をスワップイン

知らない間に OS が自動的にスワッピングを行う。(仮想記憶!!)

セグメンテーション

12 / 16

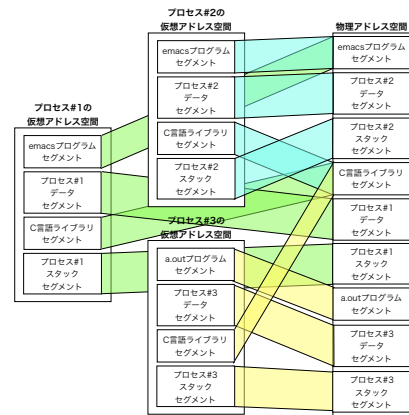
## セグメントの共用

プロセス間でセグメントを共用しメモリを節約する。

- プログラムや定数等を書き変わらない。(共有可)
  - プログラム本体
  - サブルーチン (ライブラリ)
  - 定数表
- データ、ヒープ、スタック等を書き換わる。(共有不可)  
プロセス毎に別のコピーを持つ必要がある。

セグメンテーション

13 / 16



セグメンテーション

14 / 16

## セグメンテーションの利点・欠点 (1/2)

利点

- セグメントには、例えば「C言語ライブラリセグメント」のような、論理的な意味を持たせることができる。
- セグメントの論理的な意味を反映したメモリ保護が可能である。
- プログラムやデータの共用が容易である。
- セグメントの長さは自由に決められるので内部フラグメントが発生しない。
- セグメントの長さは動的に変化させることも可能である。
- セグメント単位のスワッピングを用いて仮想記憶を実現できる。

セグメンテーション

15 / 16

## セグメンテーションの利点・欠点 (2/2)

欠点

- 物理アドレス空間に外部フラグメントが生じる。
- 外部フラグメントの解消にはメモリコンパクションが必要である。
- 物理メモリ上に連続した領域が必要である。
- 物理メモリより大きいセグメントを作ることができない。

これらの欠点を克服するために、ページングを組み合わせたシステムがある。(IA-32, MULTICS)

セグメンテーション

16 / 16

## オペレーティングシステム 第 11 章 ページング

<https://github.com/tctsigemura/OSTextBook>

ページング

1 / 24

## ページング

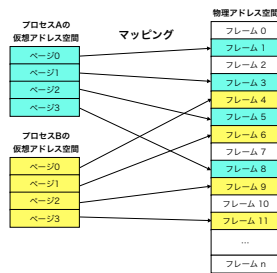
ページングは以下のようなメモリ管理方式である。

- メモリを一樣なページに分割し、ページ単位で管理する。
- メモリより広い仮想アドレス空間を使用できる。
- 外部フラグメンテーションを生じない。
- メモリコンパクションが不要である。
- Windows, macOS, Linux 等、現代の多くの OS が採用している。
- 用語
  - ページ：仮想アドレス空間をページに分割する。
  - フレーム：物理アドレス空間をフレームに分割する。

ページング

2 / 24

## 基本概念

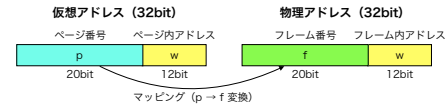


- ページをフレームにマッピングする。
- ページサイズとフレームサイズは同じ。
- どのフレームも任意プロセスの任意ページにマッピングできる。

ページング

3 / 24

## ページとフレーム

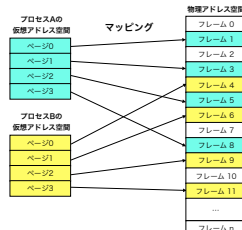


- 仮想アドレスの上位ビットがページ番号 (p)
- 仮想アドレスの下位ビットがページ内アドレス (w)
- ページサイズは2の累乗にする。
- ページ内アドレスがw ビットならページサイズは $2^w$  バイト
- 物理アドレスの上位ビットがフレーム番号 (f)
- 物理アドレスの下位ビットがフレーム内アドレス (w)
- ページ内アドレスとフレーム内アドレスは同じ (w)
- p を f に変換することでページをフレームにマッピングする。
- p と f のビット数は異なっても良い。

ページング

4 / 24

## マッピング関数

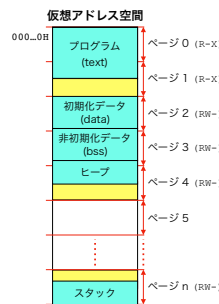


- $p \rightarrow f$  変換関数をマッピング関数と呼ぶ。
- ページテーブル (表) を用いて実装する。
- MMU がページテーブルを参照してマッピングする。
- プロセス毎にマッピング関数は異なる。
- ディスパッチャが MMU を操作しマッピングを入れ替える。

ページング

5 / 24

## フラグメンテーション

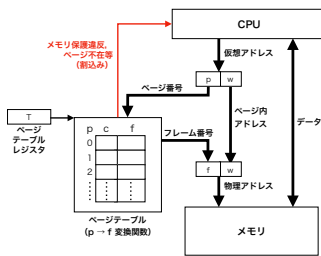


- 外部フラグメンテーションは解決した。
    - どのフレームでも任意のプロセスの任意のページにマッピングできる。
    - メモリコンパクションも不要になった。
  - 内部フラグメンテーションが発生する。ページ毎にメモリ保護モードを設定する。
    - プログラム領域は  $r-x$  にする。
    - データ領域は  $r-w$  にする。
    - あな部分にはフレームを割当てない。(sparse address spaces)
    - スタック領域も  $r-w$  にする。
- 領域サイズはページの倍数ではない。フラグメント部分のアクセスは不正だが検知できない。フラグメントサイズの平均はページサイズの  $1/2$ 。

ページング

6 / 24

## ページング機構の概要



- ページテーブルの一つのエントリをページ番号 (p) で選択する。
- 選択したエントリに格納されているフレーム番号 (f) を取り出す。
- フレーム番号 (f) とページ内アドレス (w) を結合し物理アドレスにする。

ページング

7/24

## ページテーブルエントリ

- *f* フィールド：フレーム番号
- *c* フィールド：制御

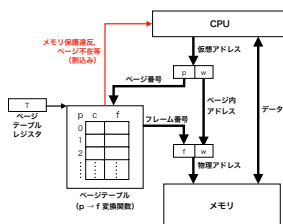
| 名称                       | ビット数 | 意味              |
|--------------------------|------|-----------------|
| V (Valid)                | 1    | フレームが割り付けられている。 |
| R (Reference)            | 1    | ページの内容が参照された。   |
| D (Dirty)                | 1    | ページの内容が変更された。   |
| RWX (Read/Write/execute) | 3    | 許されるアクセス方法。     |

- V=0 ならページ不在割込み
- R はページの使用頻度の測定等に使用
- D=0 ならスワップアウト不要
- RWX によりメモリ保護 (メモリ保護割込み)

ページング

8/24

## ページテーブル

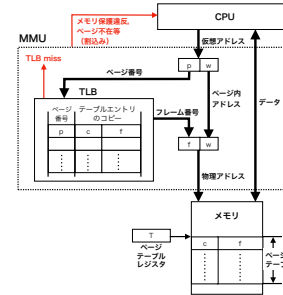


- ページテーブルはかなり大きな表である。
- ページ番号が 20 ビットなら  $2^{20} = 1\text{Mi}$  エントリ
- エントリのサイズが 4 バイトと仮定すると全体で 4MiB
- プロセス毎に必要なのでディスパッチの度にロードするのも大変
- ページテーブルレジスタにアドレスを記録しメモリ上に置く

ページング

9/24

## TLB (Translation Look-aside Buffer)

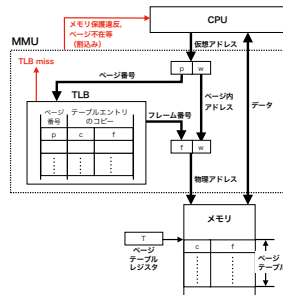


- メモリ上にあるページテーブルにアクセスするには時間がかかる。
- 変換結果 (p と f の対応) を TLB にキャッシュする。
- TLB (数十から数千エントリ) は高速な連想メモリ。

ページング

10/24

## TLB (Translation Look-aside Buffer)



- 1) ページ番号 (p) で TLB を検索しエントリを選択する。 (TLB miss)
- 2) RWX をチェックする。 (メモリ保護例外)
- 3) フレーム番号 (f) を出力する。

ページング

11/24

## Page Table Walk

- TLB miss のときページテーブルを検索すること。
- ハードウェアで自動的に行う場合  
ページテーブルレジスタからページテーブルの位置を知る。  
ハードウェアを用いることで高速化
- ソフトウェアで行う場合  
TLB miss 割込みを発生し OS に切替える。  
ハードウェアが単純 => チップ面積に余裕 => TLB のエントリ数を増やす => TLB miss の頻度を低くする。
- ページテーブルのエントリ V=0 の場合  
ページ不在割込みを発生
- ページテーブルのエントリ V=1 の場合
  1. TLB のエントリの一つをページテーブルに書き戻す。
  2. TLB の空いたエントリにページテーブルからロード

ページング

12/24

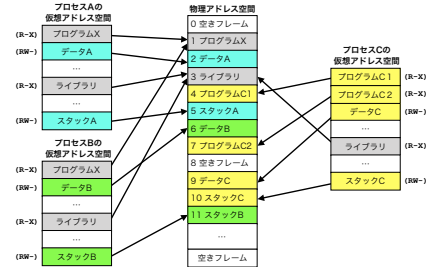
## TLB エントリのクリア

- プロセススイッチのとき
- ページテーブルに変更があったとき
- TLB の内容は変更前のページテーブルのエントリなのでクリアする必要がある。
- TLB のクリアは大きなペナルティを伴うので避けたい。
- TLB のエントリがプロセス番号を含む方式
- TLB のエントリを個別にクリアできる方式

ページング

13 / 24

## フレームの共用



- プロセスが変更しないページ (R-X) は共用できる。
- ページテーブルの操作により実現
- ライブラリは位置独立コードでなければならない。

ページング

14 / 24

## 位置独立コード

位置独立コードのイメージ

```
CALL 200,PC // 200 番地先にあるサブルーチン実行
JMP 100,PC // 100 番地先にジャンプする
LD G0,4,FP // ローカル変数はスタック上
ST G0,40,G11 // グローバル変数はレジスタ相対で参照
```

- ライブラリはマッピングされる仮想アドレスが変化する。
- どのアドレスにマッピングされても大丈夫なプログラム => 位置独立コード
- PC 相対で JMP や CALL を行う。
- データはレジスタをベースにアクセスする。

ページング

15 / 24

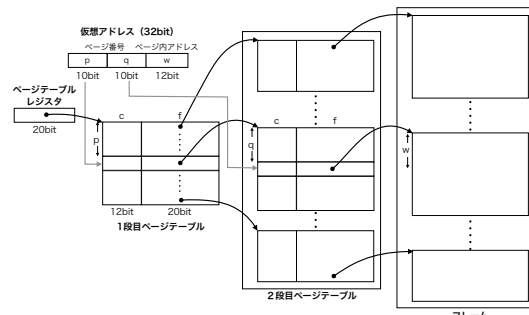
## ページテーブルの編成方法

- ページテーブルは大きくなりすぎる。(32 ビット CPU の例)
  - 仮想アドレス 32 ビット、ページサイズ 4KiB、エントリ 4B の例  
 $2^{32} \div 4\text{KiB} = 2^{32} \div 2^{12} = 2^{20} = 1\text{Mi}$  エントリ  
 $1\text{Mi}$  エントリ  $\times 4\text{B} = 4\text{MiB}$
  - 32 ビット CPU の普及が始まった当時の PC は、メモリを 4MiB ~ 16MiB しか搭載していなかった。
- ページテーブルは大きくなりすぎる。(64 ビット CPU の例)
  - 仮想アドレス 48 ビット、ページサイズ 4KiB、エントリ 8B の例  
 $2^{48} \div 4\text{KiB} = 2^{48} \div 2^{12} = 2^{36} = 64\text{Gi}$  エントリ  
 $64\text{Gi}$  エントリ  $\times 8\text{B} = 512\text{GiB}$
  - 現代の 64 ビット PC のメモリは、4GiB ~ 16GiB 程度？
- ページテーブルを小さくする工夫が必要！！

ページング

16 / 24

## 二段のページテーブル (IA-32 の例)

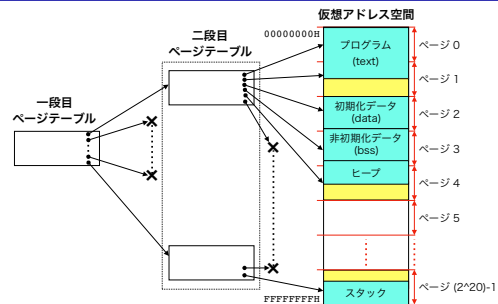


- 一段目のページテーブルサイズ 4KiB = フレームサイズ
- 二段目のページテーブルの区画サイズ 4KiB = フレームサイズ

ページング

17 / 24

## ページテーブルフレームの節約



- 仮想アドレス空間のあな部分の二段目を省略
- 一段目のページテーブルエントリの V=0 にする。
- 従来 1,025 フレーム => 3 フレーム

ページング

18 / 24

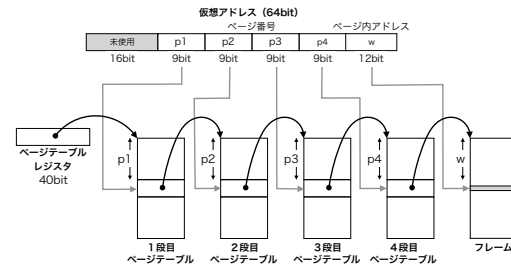
## 64 ビット仮想アドレス空間 (x86-64 の例)

- 実質 48 ビット仮想アドレス => 256TiB (十分大きい)
- 仮に二段のページテーブルならページテーブルの区画は 18 ビット (p), 18 ビット (q), 12 ビット (w) と仮定  
 $2^{18} \times 8B = 2MiB$
- プロセスあたり最低でも 3 区画必要  
 $2MiB \times 3 = 6MiB$
- 400 個のプロセスがあったとすると  
 $6MiB \times 400 = 2.4GiB$  (8GiB の 30%)
- 二段のページテーブルでは区画が大きくなりすぎる.
- 区画を小さくするために段数を多くする.

ページング

19 / 24

## 四段のページテーブル (x86-64 の例)

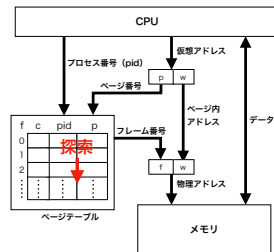


- ページサイズ (フレームサイズ) は 4KiB
- ページテーブルの区画は  $2^9 \times 8B = 4KiB$
- ページテーブルは最低 7 フレーム (28KiB)
- 400 プロセスでも約 11MiB で済む (8GiB の 0.13%)

ページング

20 / 24

## 逆引きページテーブル



- テーブルでフレーム番号とページ番号の立場が逆転 (逆引き)
- システム全体でページテーブル一つ (プロセス毎ではない)
- どの仮想アドレス空間のエントリが識別するための pid あり

ページング

21 / 24

## 逆引きページテーブル

ページテーブルのサイズ

- 8GiB のメモリを 4KiB のページで分割する場合のエントリ数  
 $8GiB \div 4KiB = 2^{33} \div 2^{12} = 2Mi$  エントリ
- 1 エントリ 8 バイト仮定すると  
 $2Mi \times 8B = 12MiB$
- システム全体で 12MiB で済む (8GiB の 0.2%)

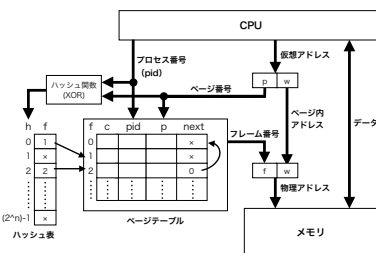
Page Table Walk

- CPU は仮想アドレスの他に pid (プロセス番号を出力)
- ページテーブルを pid と p (ページ番号) で探索する
- 線形探索などを用いると遅くて実用にならない

ページング

22 / 24

## 逆引きページテーブル (IBM 801 の例)



- ハッシュ表を用いて探索を高速化
- ハッシュ表の大きさは二の累乗 ( $0 \sim 2^n - 1$ )
- ページテーブルは next を使用してチェーンを作る

ページング

23 / 24

## 逆引きページテーブル (IBM 801 の例)

ハッシュ表を用いた Page Table Walk

- pid と p を用いてハッシュ関数を計算する ( $h \leftarrow f(pid, p)$ )  
 (ハッシュ関数は pid と p の XOR... 速度優先)
- ハッシュ表の第 h エントリを見る
  - 空 (図では x) ならページ不在 (割込み!)
  - 空でなければページテーブルのインデックス (f)
- ページテーブルの第 f エントリの内容を見る
  - pid, p が一致 → この時の f をフレーム番号にする (完了!)
  - pid, p が一致しない → next を見る
    - 空 (図では x) ならページ不在 (割込み!)
    - 空でなければ  
 ページテーブルのインデックス (f) を更新して再度トライ

TLB: 不可欠!

ページング

24 / 24

## オペレーティングシステム 第 12 章 仮想記憶

<https://github.com/tctsigemura/OSTextBook>

仮想記憶

1 / 34

### 基本概念

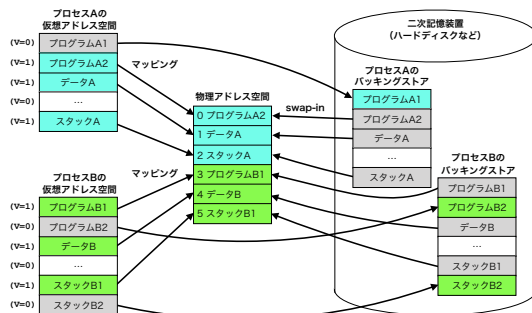
ページングをベースに仮想記憶を実現する。

- システムの使用メモリ合計が物理メモリより大きい。→ 実行可
- 単一のプログラムがメモリより大きい。→ 実行可
- ページテーブルの V=0 を上手く使用する。
- V=0 のページにアクセスするとページ不在割込み → OS へ
- ページテーブルの V=0 に二つの場合がある。
  1. 無効な領域 → プロセス終了
  2. バックキングストアに退避中 → 復旧して再開
- プロセス生成時にバックキングストアにプロセスのイメージを作る。
- Windows, macOS, Linux 等、現代の OS のほとんどが採用している。

仮想記憶

2 / 34

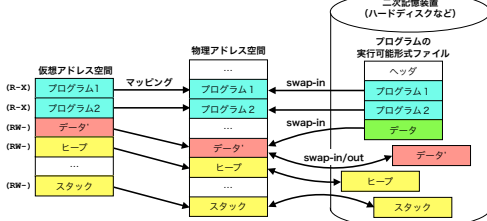
### 仮想記憶の基本



仮想記憶

3 / 34

### デマンドページング (Demand Paging)

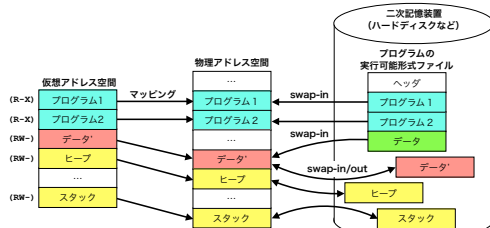


- ページを swap-in するための方式の一つ。
- 全てのページが不在の状態からスタートする。
- ページ不在を起こしたページを swap-in する。  
(使用しないページを読み込むような無駄が無い)

仮想記憶

4 / 34

### プログラムファイルの直接 swap-in による実行

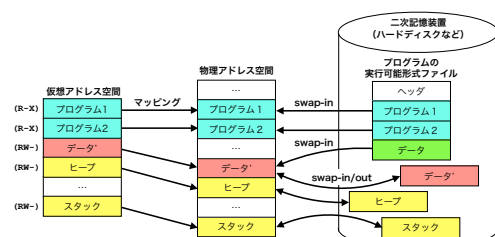


- デマンドページング用の実行可能形式ファイルを用いる。  
(このファイルはページサイズを意識した構造になっている)
- プログラムはファイルから swap-in する (R-X に設定)。
- 初期化データはファイルから swap-in する (RW- に設定)。
- 非初期化データ、ヒープ、スタックはゼロにする (RW- に設定)。

仮想記憶

5 / 34

### プログラムの swap-out

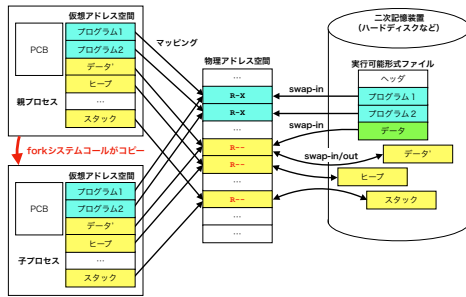


- フレームが枯渇したら使用頻度の低いフレームを解放し再利用する。
- プログラム (R-X) は変化しないので swap-out しない。
- 初期化データ (RW-) はバックキングストアに swap-out する。
- 非初期化データ、ヒープ、スタックも swap-out する。

仮想記憶

6 / 34

## Copy on Write (1)

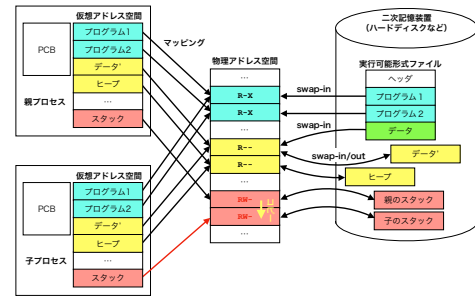


- fork-exec ではアドレス空間のコピーに無駄が多い。→ vfork
- vfork は使いにくい、使いやすい fork を改良する。
- fork の後、書き込み可能ページを一時的に R-- に設定しておく。

仮想記憶

7 / 34

## Copy on Write (2)

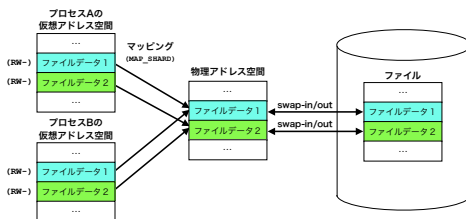


- 例えばスタックに書き込むとメモリ保護違反が起こる。
- この時点で OS が新しいフレームを割当て、内容をコピーする。
- ページを RW-- に変更しプロセスを再開する。

仮想記憶

8 / 34

## メモリマップドファイル (1)



- 仮想記憶機構を用いたファイルへのアクセス手段である。
- プロセスはメモリ上の配列のようにファイルにアクセスできる。
- ファイルアクセスで、一々システムコールを使用しない。(軽いファイルアクセス手段)
- 同じファイルを複数プロセスがマッピング → 共有メモリになる。

仮想記憶

9 / 34

## メモリマップドファイル (2)

UNIX のメモリマップドファイルの例 (mmap システムコール)

```
void * mmap(void *addr, size_t len, int prot, int flags, int fd, off_t offset);
```

- 戻り値: マップされた領域の先頭アドレスが返される。
- addr: マップしたい仮想アドレス空間の先頭アドレスを渡す。
- len: マップする領域の大きさを渡す。
- prot: 保護モード (protection: RWX) を表す値を渡す。
- flags: 共用する (MAP\_SHARED) / しない (MAP\_PRIVATE) 等
- fd: オープン済みファイルのファイルディスクリプタを渡す。
- offset: ファイル中のマッピング位置。
- アドレスや長さはページサイズの整数倍にする。

仮想記憶

10 / 34

## メモリマップドファイル (3)

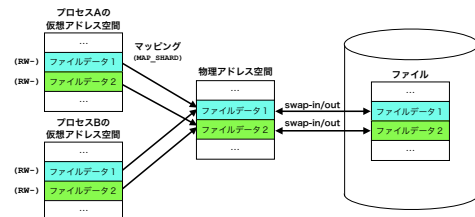
```
1 #include <stdio.h>           // perrorのために必要
2 #include <fcntl.h>           // openのために必要
3 #include <unistd.h>          // closeのために必要
4 #include <sys/mman.h>        // mmapのために必要
5 int main() {
6     int fd;
7     char *p, *fname="a.txt";
8     fd = open(fname, O_RDWR); // 予め作成してある4KiBのファイルを開く
9     if (fd < 0) {
10         perror(fname);
11         return 1;
12     }
13     p = mmap(NULL, 4096, PROT_READ|PROT_WRITE, MAP_FILE|MAP_SHARED, fd, 0);
14     if (p == MAP_FAILED) {
15         perror("mmap");
16         return 1;
17     }
18     close(fd);
19     for (int i=0; i<4096; i++) { // マップしたらクローズして良い
20         p[i] = 'A' + (i % 26); // ファイルにA~Zを繰り返し書き込む
21     }
22     return 0;
23 }
```

仮想記憶

11 / 34

## メモリマップドファイル (4)

メモリマップドファイルの仕組み



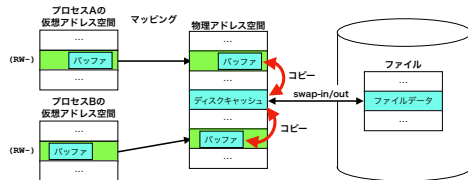
- ファイルの読み込みはデマンドページングの要領で行う。
- ファイルの書き込みは
  - Dirty ページを定期的にファイルに書き戻す。
  - プロセスの終了やマッピングの解消時に書き戻す。

仮想記憶

12 / 34

## メモリマップドファイル (5)

read/write システムコールとの比較



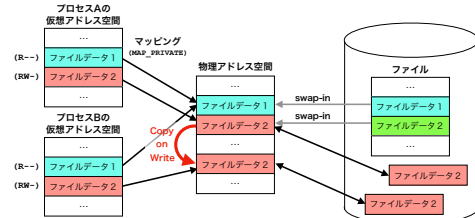
- ファイルを操作する度にシステムコールを発行する。  
(システムコールは重い処理)
- ディスクキャッシュとプログラムのバッファ間でメモリコピー  
(メモリコピーは重い処理)

仮想記憶

13 / 34

## メモリマップドファイル (6)

プロセスにローカルなマッピング



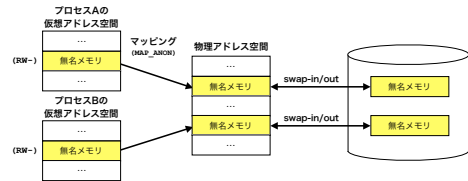
- これまでは MAP\_SHARED の例だった。
- MAP\_PRIVATE の例を紹介する。
- 最初は「ファイルデータ 1」のように共有される (R--).
- 書き換えが発生した時点でコピーを作る (Copy on Write).
- 「ファイルデータ 2」のようにプロセスは別々のコピーを参照する。

仮想記憶

14 / 34

## メモリマップドファイル (7)

無名メモリ



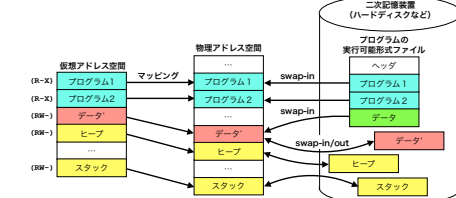
- ファイルに関連付けられないがメモリ領域。
- MAP\_ANON フラグを用いる。
- アクセスがあった時点で作成される (デマンドページング)。
- ページの初期値はゼロ。

仮想記憶

15 / 34

## メモリマップドファイル (8)

プログラムの実行とメモリマップドファイル



- 実行形式ファイルをメモリにマッピングする。
- プログラムは、R-X, MAP\_SHARED/PRIVATE でマッピングする。  
(プログラムはプロセス間で共有される)
- 初期化データは、RW-, MAP\_PRIVATE でマッピングする。
- 非初期化データ、ヒープ、スタックは無名メモリ (RW-, MAP\_ANON)

仮想記憶

16 / 34

## ページ置換えアルゴリズム

ページングによる仮想記憶で重要な三つのアルゴリズム

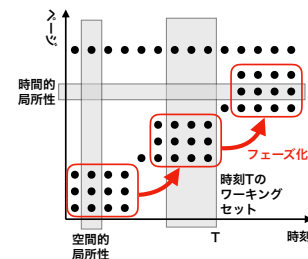
1. ページ読み込みアルゴリズム：いつページを swap-in するか決める。  
普通は、既に学んだデマンドページングを用いる。
2. ページ置き換えアルゴリズム：フレーム不足時に、どのページを再利用するか決める。
3. フレーム割付けアルゴリズム：どのフレームを使用するか決める。

ページ置き換えアルゴリズムが、将来、使用されないフレームをうまく選択しないと、swap-out したページが直後に swap-in されることになり、システムの性能が著しく低下する。

仮想記憶

17 / 34

## 局所性・ワーキングセット・フェーズ化 (1)

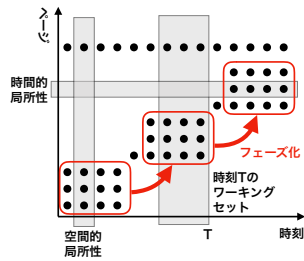


**局所性** 短い時間に着目すると、一部の連続したページが集中的にアクセスされる。→ 空間的局所性  
あるページに着目すると一部の連続した時刻にアクセスが集中している。→ 時間的局所性

仮想記憶

18 / 34

## 局所性・ワーキングセット・フェーズ化 (2)

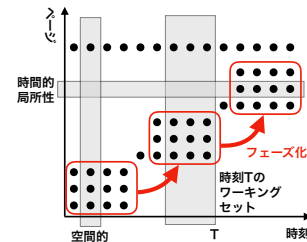


**ワーキングセット** ある時間にアクセスされるページの集合のこと。メモリに入り切らなくなると急激に性能が低下する。  
→ スラッシング

仮想記憶

19 / 34

## 局所性・ワーキングセット・フェーズ化 (3)



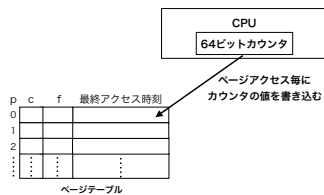
**フェーズ化現象** ワーキングセットが急激に変化する現象のこと。「入力フェーズ」、「計算フェーズ」、「出力フェーズ」フェーズ遷移時は局所性が失われる → ページ不在集中

仮想記憶

20 / 34

## LRU (Least Recently Used) アルゴリズム (1)

仮定：最近アクセスされていないページは、この先、アクセスされない。  
(時間的局所性があるなら最適な方法。)

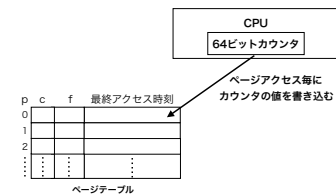


1. メモリアクセス毎にページテーブルにカウンタの値を書く。
2. ページテーブルをスキャンし最も古いページを見つける。
3. 見つけたページを swap-out し目的のページを swap-in する。(置換え)

仮想記憶

21 / 34

## LRU (Least Recently Used) アルゴリズム (2)



問題点 (LRU の完全な実装は困難と言われている)

1. ハードウェアのコスト
  2. ページ不在時の処理の重さ。(ページ不在は頻繁に発生)
- macOS の vm\_stat で調べると毎秒数千回のページ不在！！

仮想記憶

22 / 34

## LFU (Least Frequently Used) アルゴリズム

NFU (Not Frequently Used) と呼ばれる。

- LRU の近似方式の一種である。
- ページテーブルの R ビットを使用。
- フレーム毎にカウンタを準備。
- 特別なハードウェアは不要。

アルゴリズム

1. R ビットとフレームのカウンタをゼロにクリアする。
2. 定期的 (例えば TICK=20ms 毎) にページテーブルをスキャンする。R=1 のエントリを見つけたら対応するフレームのカウンタをインクリメントし、R をゼロにクリアする。
3. ページ不在時にフレームが不足したなら、カウンタの値が最小のフレームを置き換える。

仮想記憶

23 / 34

## エージングアルゴリズム

LFU (Least Frequently Used) アルゴリズムの改良

LFU の問題点

一度カウンタの値が大きくなると、使用されなくなっても置き換えが起らない。

LFU の改良

定期的にページテーブルをスキャンする際のカウンタの更新方法を次のように改良する。

**R=1 のフレーム**  $cnt \leftarrow cnt \div 2 + 0x8000$  (カウンタは 16bit と仮定)

**R=0 のフレーム**  $cnt \leftarrow cnt \div 2$

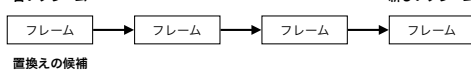
この改良により、過去の R ビットの影響が徐々に小さくなる。

仮想記憶

24 / 34

## FIFO (First-In First-Out) アルゴリズム

仮定: 「長くメモリに滞在しているページは役割を終えている」  
特別なハードウェアを用いることなく、ソフトウェアだけで実現できる。



アルゴリズム

1. swap-in したフレームをリストの最後に追加する。
2. フレームが不足時は、リストの先頭のフレームを置き換える。

ページテーブルのスキャンが不要なので非常に軽い。  
常時使用されるページも時間が経過すると swap-out される問題がある。  
Belady の異常な振る舞いをすることがある。

仮想記憶

25 / 34

## Belady の異常な振る舞いの例

FIFO アルゴリズムを用い、  
ページ参照ストリング (W : 1 2 3 4 1 2 5 1 2 3 4 5) の場合

- フレーム数 (m=3) の場合 (ページ不在 9 回)

| W | 1  | 2  | 3  | 4  | 1  | 2  | 5  | 1 | 2 | 3  | 4  | 5 |
|---|----|----|----|----|----|----|----|---|---|----|----|---|
| S | *1 | *2 | *3 | *4 | *1 | *2 | *5 | 5 | 5 | *3 | *4 | 4 |
|   |    | 1  | 2  | 3  | 4  | 1  | 2  | 2 | 2 | 5  | 3  | 3 |
|   |    |    | 1  | 2  | 3  | 4  | 1  | 1 | 1 | 2  | 5  | 5 |

- フレーム数 (m=4) の場合 (ページ不在 10 回)

| W | 1  | 2  | 3  | 4  | 1 | 2 | 5  | 1  | 2  | 3  | 4  | 5  |
|---|----|----|----|----|---|---|----|----|----|----|----|----|
| S | *1 | *2 | *3 | *4 | 4 | 4 | *5 | *1 | *2 | *3 | *4 | *5 |
|   |    | 1  | 2  | 3  | 3 | 3 | 4  | 5  | 1  | 2  | 3  | 4  |
|   |    |    | 1  | 2  | 2 | 2 | 3  | 4  | 5  | 1  | 2  | 3  |
|   |    |    |    | 1  | 1 | 1 | 2  | 3  | 4  | 5  | 1  | 2  |

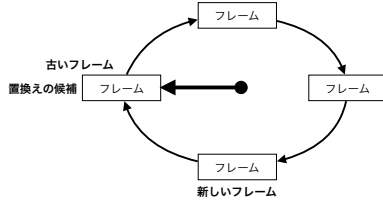
メモリが多い方 (m=4) のページ不在回数が多い。

仮想記憶

26 / 34

## Clock アルゴリズム

環状リストを用いる。R ビットも使用する。

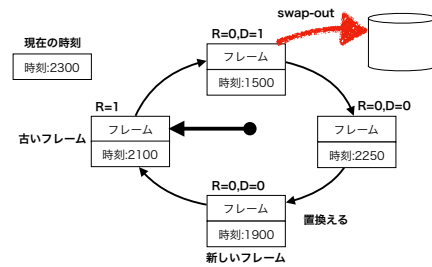


1. swap-in する度にフレームを環状リストに挿入していく。
2. 定期的 (例えば TICK=20ms 毎) に R ビットをクリアする。
3. 時計の針が指しているフレームの R ビットを調べる。  
**R=0 の場合** ページは古い+最近アクセスされていない。→ 置換え  
**R=1 の場合** ページは最近アクセスされている。→ 針を進める  
 最悪でも時計の針が一周回ると R=0 のページが見つかる。

仮想記憶

27 / 34

## WSClock アルゴリズム (1)

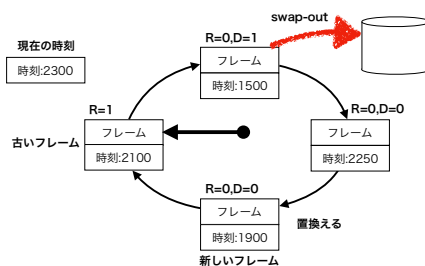


- ワーキングセットを考慮した Clock アルゴリズムである。
- 単純でパフォーマンスが良いので広く使用されている。
- アクセス時刻を記録した環状リストに用いる。

仮想記憶

28 / 34

## WSClock アルゴリズム (2)

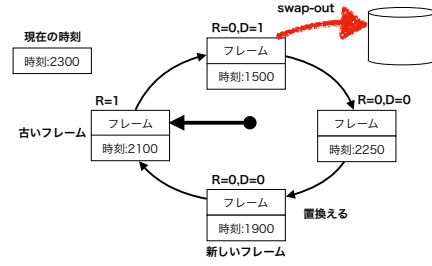


- 時刻が古くなっているフレームはワーキングセット外と判断。
- ページテーブルの R ビットと D ビットも使用する。

仮想記憶

29 / 34

## WSClock アルゴリズム (3)

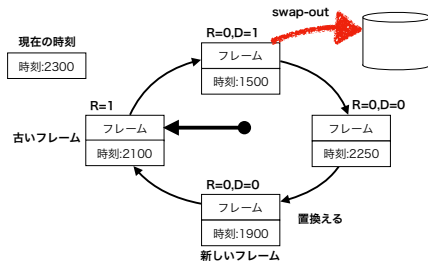


1. swap-in する度にフレームを環状リストに挿入していく。
2. 定期的に全テーブルエントリの R ビットをクリアする。  
 その際、R=1 だったフレームだけに現在時刻を記録する。

仮想記憶

30 / 34

## WSClock アルゴリズム (4)

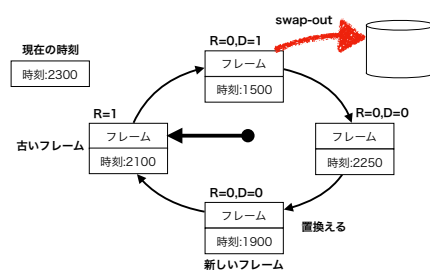


3. フレーム不足なら時計の針のフレームを調べる。  
**R=1 の場合** R ビットをクリアして次のフレームに進む。  
**時刻が新しい場合** 次のフレームに進む。  
**時刻が古い場合** ページはワーキングセットに含まれていない。  
 ページの置換え処理を行う。

仮想記憶

31 / 34

## WSClock アルゴリズム (5)

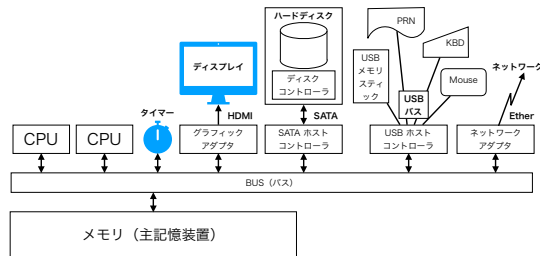


3. フレーム不足なら時計の針のフレームを調べる。  
**時刻が古い場合** ページの置換え処理を行う。  
**D=1 の場合** swap-out を予約し次のフレームに進む。  
**D=0 の場合** このフレームを置き換える。

仮想記憶

32 / 34

## フレーム割り付けアルゴリズム (1)

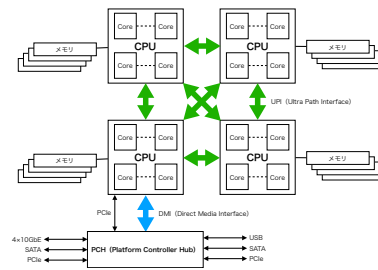


- フレームはどれも均質
- SMP システムであってもそうである。

仮想記憶

33 / 34

## フレーム割り付けアルゴリズム (2)



- 近年のサーバ用 SMP では均質ではない。
- メモリとCPUからなるノードを相互接続している。
- 自ノードと他ノードでメモリアクセス時間が異なる。
- このことを考慮したフレーム割付とCPUスケジューリングが必要。

仮想記憶

34 / 34

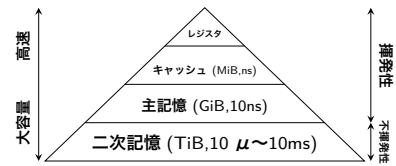
## オペレーティングシステム 第13章 二次記憶装置 (ストレージ)

<https://github.com/tctsigemura/OSTextBook>

二次記憶装置

1 / 16

### 記憶装置の階層 (1)



- レジスタはCPUレジスタのこと。  
容量は数十～数百バイト程度、高速アクセスが可能、揮発性
- 主記憶 (メモリ)  
アクセス時間は数十ナノ秒程度  
容量は数Giバイト～数十Giバイト程度、揮発性
- 二次記憶装置  
ハードディスクやSSD (Solid State Drive) のこと。  
アクセス時間は数ミリ秒～数十ミリ秒 (ハードディスク)、不揮発性

二次記憶装置

2 / 16

### 記憶装置の階層 (2)

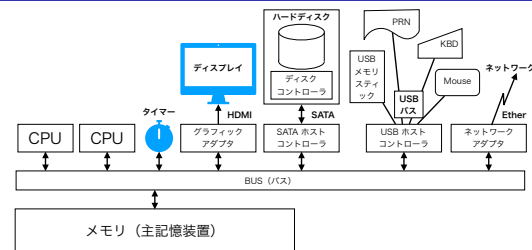
夫々の特性に合った使い方を。  
二次記憶装置の特性は次の通り。

- 大容量 (ビット単価が安い)  
オペレーティングシステム、アプリケーション、データなどの全てを格納できる。
- 不揮発性 (電源を切っても消えない)  
プログラムやデータの永続的な置き場所として適している。

二次記憶装置

3 / 16

### 二次記憶装置の種類 (1)



接続方式

- CPUからはホストコントローラを介してアクセスする。
- 二次記憶装置はホストコントローラの先に接続される。
- USBメモリスティックやポータブルハードディスクは取り外し可能。
- 取り外し可能 => データ交換、バックアップ用途にも適する。

二次記憶装置

4 / 16

### 二次記憶装置の種類 (2)



テープ型装置

- データのバックアップや輸送用 (ビット単価が安い)
- シーケンシャルアクセス専用
- 読み出し位置まで進むために数分!!

二次記憶装置

5 / 16

### 二次記憶装置の種類 (3)



ディスク型装置

- ランダムアクセスが可能
- ハードディスクのこと (CD-ROMなどの光ディスクも仲間)
- SSD, USBメモリ, その他メモリカードも仲間

二次記憶装置

6 / 16

## ハードディスク (1)

### ハードディスク

- データは磁気的に円盤の表面に記録され不揮発性.
- アドレスを指定してランダムアクセスが可能.
- セクタ単位で読み書きを行う.
- システムの起動ドライブ (OS, アプリ, データ全てが置かれる)
- 仮想記憶のバックストレージとしても使用される.
- ハードディスク管理が, OS の性能や使い勝手を左右する.
- ファイル管理機構はハードディスクを前提にしていることが多い

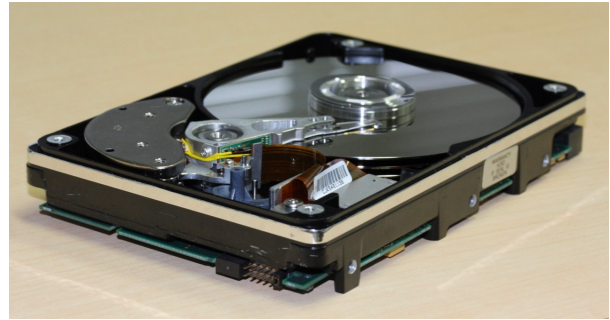
### セクタ・トラック・シリンドラ

- 同心円のトラック (Track)
- トラックを区切ったセクタ (Sector)
- トラックをまとめたシリンドラ (Cylinder)

二次記憶装置

7 / 16

## ハードディスク (2)



二次記憶装置

8 / 16

## ハードディスク (3)

### セクタのアドレッシング

512 バイト (4KiB) のセクタのアドレス付け方法

- CHS (Cylinder Head Sector) 方式
  - Cylinder Head Sector の三次元アドレス.
  - Head は Track と同じ意味.
  - CHS は PC の世界で使用されてきた用語.
  - ハードディスクの物理的な構造通りのアドレッシング.
  - 過去, 長く使われてきた方式.
- LBA (Logical Block Addressing)
  - セクタの通し番号 (一次元) を用いる.
  - ハードディスクブラックボックス化 (物理構造が不明)
  - CHS は煩雑なだけでメリットがなくなった.

二次記憶装置

9 / 16

## フォーマット (1)

### ハードディスクの初期化の例

1. 低レベル (物理) フォーマット  
ディスクの表面に磁気的にトラックを書き込む.
2. パーティション (区画) に分割
  - 装置全体を一つのボリューム => 大きすぎる
  - 区画に分割し区画をボリュームとして扱う => オペレーティングシステムのパーティション
  - ユーザデータのパーティション => ここだけバックアップ
  - 複数のオペレーティングシステムをインストール  
第1パーティション (ボリューム) に Windows  
第2パーティション (ボリューム) に Linux  
第3パーティション (ボリューム) に FreeBSD
3. 高レベル (論理) フォーマット  
各ボリュームの内部に該当オペレーティングシステムの空のファイルシステムを作る.

二次記憶装置

10 / 16

## フォーマット (2)

### PC用ハードディスクのパーティションの例

| MBR       |
|-----------|
| パーティション 1 |
| パーティション 2 |
| パーティション 3 |
| パーティション 4 |

- MBR (Master Boot Record)
  - ハードディスクの先頭セクタ (LBA0) に格納
  - MBR のサイズは 512 バイト
  - 内容はブートプログラムとパーティションテーブル

二次記憶装置

11 / 16

## フォーマット (3)

### PC用ハードディスクの MBR の内容

|                          |
|--------------------------|
| ブートプログラム<br>(446 バイト)    |
| パーティション<br>テーブル (64 バイト) |
| シグネチャ (2 バイト)            |

- MBR (Master Boot Record) (512 バイト)
  - ブートプログラム (446 バイト)  
PC の機械語プログラム (OS を起動するためのプログラム)
  - パーティションテーブル (64 バイト)  
各パーティションの位置と大きさ等を記録する 4 行の表
  - シグネチャ (2 バイト)  
フォーマットされている目印 (55H, AAH)

二次記憶装置

12 / 16

## フォーマット (4)

PC用ハードディスクのパーティションテーブルの例

| Flag<br>(1) | Start<br>CHS (3) | Type<br>(1) | End<br>CHS (3) | Start<br>LBA (4) | Size<br>(4) |
|-------------|------------------|-------------|----------------|------------------|-------------|
| 80H         | ???              | 06H         | ???            | 0000003FH        | 00003F00H   |
| 00H         | ???              | A5H         | ???            | 00003F3FH        | 0000BD00H   |
| 00H         | ???              | 00H         | ???            | ????????         | ???????     |
| 00H         | ???              | 00H         | ???            | ????????         | ???????     |

| 項目        | バイト数 | 意味                        | Type | 意味          |
|-----------|------|---------------------------|------|-------------|
| Flag      | 1    | 80H アクティブ/<br>00H インアクティブ | 00H  | 空き          |
| Start CHS | 3    | 開始アドレス (CHS 表現)           | 01H  | FAT12       |
| Type      | 1    | ファイルシステムの種類               | 04H  | FAT16(小)    |
| End CHS   | 3    | 終了アドレス (CHS 表現)           | 06H  | FAT16(大)    |
| Start LBA | 4    | 開始アドレス (LBA 表現)           | 07H  | NTFS        |
| Size      | 4    | セクタ数 (LBA 表現)             | 0BH  | FAT32       |
|           |      |                           | 83H  | Linux(ext2) |
|           |      |                           | A5H  | FreeBSD     |

二次記憶装置

13 / 16

## ブートストラップ (1)

PC の場合を例にブートストラップを説明する。

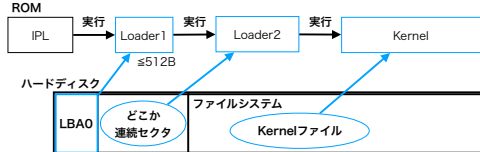
- ハードディスクから OS を起動する作業のこと。
- OS のカーネルを格納したファイルを見つけてロード・実行する。  
=> ブートストラップは後で変更できる必要がある。
- 以下に説明する段階を経て OS をブートする。
- 以下の方法が PC では標準的であるが様々な変種がある。  
(段階が多い場合、強力なブートマネージャを備えている場合)

二次記憶装置

14 / 16

## ブートストラップ (2)

ハードディスク = ボリュームの場合



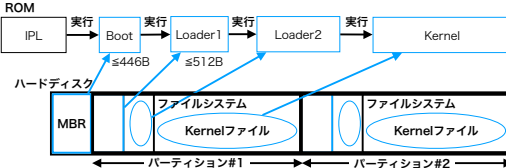
- IPL (Initial Program Loader)  
PC の ROM に格納されており電源 ON と同時に動作開始
- ブートローダ (第1段階: Loader1) 512バイト以内  
LBA0 に格納され IPL によってロード・実行される。
- ブートローダ (第2段階: Loader2)  
ディスク上のどこか連続セクタに格納され Loader1 がロード・実行。  
サイズに制限がない => 高機能にできる。
- OS のカーネル  
ファイルシステムにファイルとして格納され Loader2 がロード・実行。

二次記憶装置

15 / 16

## ブートストラップ (3)

パーティション = ボリュームの場合



- IPL (Initial Program Loader)
- ブートセクタ・ブートマネージャ (Boot) 446バイト以内  
LBA0 (MBR) に格納され IPL によってロード・実行される。  
メニューを表示してユーザに OS のパーティションを選択させる。  
(勝手に次に進むものもある。)
- ブートローダ (第1段階: Loader1) 512バイト以内
- ブートローダ (第2段階: Loader2)
- OS のカーネル

二次記憶装置

16 / 16

## オペレーティングシステム 第 14 章 ファイルシステムの概念

<https://github.com/tctsigemura/OSTextBook>

ファイルシステムの概念

1 / 18

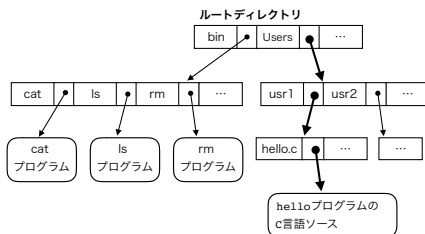
## ファイルシステム

- ファイルシステムは二次記憶装置を
  - 管理する。(どのセクタが、どのファイルの一部?)
  - 抽象化する。(ハードディスク → ファイル)
  - 仮想化する。(1 台のハードディスク → 多数のファイル)
- ファイルは一次元のバイト列 (バイトストリーム)  
オペレーティングシステムはファイルの構造を決めない。
- ファイルは名前を持つ。
- 名前とバイト位置でデータが決まる。  
名前 = ファイル名, バイト位置 = ファイル内オフセット

ファイルシステムの概念

2 / 18

## ファイルの名前付け



- ファイルは木構造のディレクトリシステムに格納する。
- ディレクトリは名前とファイル本体のポインタを格納する。
- 階層構造を持った名前 (パス) でファイルを特定する。
- 絶対パスはルートディレクトリを起点にする。
- 相対パスはワーキングディレクトリを起点にする。

ファイルシステムの概念

3 / 18

## ファイルの別名 (1)

別名があると便利な例 (最新のファイルはいつも同じ名前)

ある日

|                |                      |
|----------------|----------------------|
| 2017_06_30.log | 2017 年 6 月 30 日のファイル |
| 2017_07_01.log | 2017 年 7 月 1 日のファイル  |
| 2017_07_02.log | 2017 年 7 月 2 日のファイル  |
| today.log      | → 2017_07_02.log     |

次の日

|                |                     |
|----------------|---------------------|
| 2017_07_01.log | 2017 年 7 月 1 日のファイル |
| 2017_07_02.log | 2017 年 7 月 2 日のファイル |
| 2017_07_03.log | 2017 年 7 月 3 日のファイル |
| today.log      | → 2017_07_03.log    |

ファイルシステムの概念

4 / 18

## ファイルの別名 (2)

- ハードリンク
  - ファイルシステムの仕組みとして OS カーネルに組み込む。
  - ファイル本体が複数のディレクトリ・エントリから指される。
  - リンクカウントを用いる。
  - ディレクトリをリンクするとループ検出が厄介 → 禁止!
- シンボリックリンク
  - ファイルシステムの仕組みとして OS カーネルに組み込む。
  - 他ファイルのパスを格納した特別なファイル。
  - リンク切れ状態が許される。(Web ページのリンクに似ている)
- ファイルシステムの外で実装されるリンク
  - Windows のショートカット, macOS のエイリアスなど
  - ファイルシステム本体が持つリンク機構は一定ではない。
  - 現代の OS は同時に複数のファイルシステムを使用する。
  - アプリに近い側でどのファイルシステムでも共通の仕組みを提供

ファイルシステムの概念

5 / 18

## ファイルの別名 (3)

HFS+ファイルシステム上の macOS のエイリアスの例

```
1 $ ls -l@ a.txt*
2 -rw-r--r--  1 sigemura  admin    5 Jun 27 10:19 a.txt
3 -rw-r--r--@ 1 sigemura  admin  1012 Jun 27 10:19 a.txtのエイリアス
4 com.apple.FinderInfo          32
```

3 行 拡張属性付きの通常ファイルとしてエイリアスが存在

4 行 拡張属性の名前は com.apple.FinderInfo

4 行 拡張属性のサイズは 32 バイト

ファイルシステムのより汎用的な機構である拡張属性を利用して、エイリアスを実装している。

ファイルシステムの概念

6 / 18

## ファイルの別名 (4)

FAT ファイルシステム上の macOS のエイリアスの例

```

1 $ ls -la@ .* a.txt*
2 -rwxrwxrwx 1 sigemura staff 4096 Jun 27 09:55 ._a.txtのエイリアス
3 -rwxrwxrwx 1 sigemura staff 5 Jun 27 09:55 a.txt
4 -rwxrwxrwx@ 1 sigemura staff 1040 Jun 27 09:55 a.txtのエイリアス
5 com.apple.FinderInfo 32
6 $ rm ._a.txtのエイリアス
7 $ ls -la@ a.txt*
8 -rwxrwxrwx 1 sigemura staff 5 Jun 27 09:55 a.txt
9 -rwxrwxrwx 1 sigemura staff 1040 Jun 27 09:55 a.txtのエイリアス

```

4,5 行 拡張属性付きの通常ファイルとしてエイリアスが存在

2 行 隠しファイルができています！！

6 行 隠しファイルを消してみる、

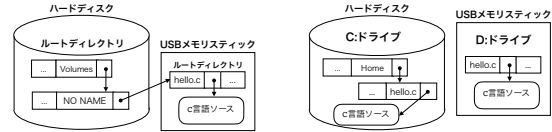
9 行 拡張属性が消えてしまった！！

FAT ファイルシステムの規約の範囲でエイリアスを実装している。

ファイルシステムの概念

7 / 18

## ボリュームのマウント



(a) マウント方式

(b) ドライブレター方式

- 二つ目以降のボリュームの接続方法
- マウント方式
  - ボリュームを既存のディレクトリに接続する、
  - /Volumes/NO NAME/hello.c が USB メモリの C プログラム
- ドライブレター方式
  - ボリュームを区別するドライブレターを用いる、
  - D:\hello.c が USB メモリの C プログラム

ファイルシステムの概念

8 / 18

## ファイルの属性 (1)

- 名前：ファイル名をファイルの属性と考える場合もある、
- 識別子：ファイル本体の番号など、
- 型 (タイプ)：通常ファイル、ディレクトリ、リンクなど、
- 保護：rwxrwxrwx など、(後で詳しく)
- 日時：作成日時、最終変更日時など、
- 所有者：所有者、グループなど、
- 位置：ディスク上のどこにファイル本体があるか、(データを格納したブロック (セクタ) の番号など)
- サイズ：ファイルのバイト数、
- 拡張属性：名前付きの小さな追加データ、ファイルシステムで用途を定めていない、

ファイルシステムの概念

9 / 18

## ファイルの属性 (2)

```

1 $ ls -l@ b.txt*
2 -rw-r--r-- 2 sigemura staff 123 Jun 25 19:38 b.txt
3 -rw-r--r--@ 1 sigemura staff 836 Jun 25 19:39 b.txtのエイリアス
4 com.apple.FinderInfo 32
5 $ xattr -l b.txtのエイリアス
6 com.apple.FinderInfo:
7 00000000 61 6C 69 73 4D 41 43 53 80 00 00 00 00 00 00 00 |alisMACS.....|
8 00000010 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 |.....|

```

1 行 拡張属性付きでファイル一覧を表示、

4 行 拡張属性付きのファイルがある、

5 行 拡張属性の内容を表示してみる、

この例の拡張属性は、以下のようなものであった、

- 属性の名前：com.apple.FinderInfo
- 属性の大きさ：32 バイト
- 意味：ファイルがエイリアスである、(ファイル本体がエイリアスのデータ)



ファイルシステムの概念

10 / 18

## アクセス制御 (1)

ファイルの保護属性に基づき、ファイルに誰が何をできるか制御する、

- ビット表現の保護モード
  - UNIX で使用される rwxrwxrwx のような情報、
  - UNIX の場合、「所有者、グループ、その他」のユーザについて
    - r : 読める (Read),
    - w : 書ける (Write),
    - x : 実行できる (eXecute)

を指定する、



ファイルシステムの概念

11 / 18

## アクセス制御 (2)

- ACL (Access Control List)  
ファイル毎に、ユーザやグループを指定して細かな制御が可能

```

1 $ ls -le a.txt
2 -rw-r--r-- 1 sigemura staff 4 Jul 5 21:55 a.txt
3 $ chmod +a "group:admin allow write" a.txt
4 $ chmod +a "group:admin deny delete" a.txt
5 $ ls -le a.txt
6 -rw-r--r--+ 1 sigemura staff 4 Jul 5 21:55 a.txt
7 0: group:admin deny delete
8 1: group:admin allow write

```

1 行 a.txt に ACL が無いことを確認した、

3,4 行 chmod コマンドで a.txt に ACL 追加した、

7,8 行 二行の ACL が確認できる、

- リストの先頭から順に評価する、
- 許可・不許可が決まったら評価を完了する、
- ACL で決まらない場合は rwx を使用する、

ファイルシステムの概念

12 / 18

## ファイルの種類

- ファイルシステム (OS カーネル) で決まっている種類 (通常ファイル・ディレクトリ・リンクなど)
- アプリケーションなどが決めている種類 (通常ファイルの拡張子で区別する)

| 拡張子                 | 意味                                 |
|---------------------|------------------------------------|
| .c, .java, .s 等     | ソース・プログラム (C 言語, Java 言語, アセンブリ言語) |
| .py, .pl, .php 等    | スクリプト言語のプログラム (python, perl, PHP)  |
| .txt, .html, .xml 等 | プレーンテキスト, マークアップ言語                 |
| .jpg, .png, .bmp 等  | 画像データ                              |
| .mp3, .m4a, .wma 等  | 音声データ                              |
| .mpg, .mp4, .wmv 等  | 動画データ                              |
| .pdf, .ps, .eps 等   | 印刷・表示用の文書ファイル                      |
| .zip, .tar, .tbz 等  | アーカイブファイル                          |
| .exe, .app, 拡張子無し   | 実行形式プログラム (Windows, macOS, UNIX)   |
| .doc, .docx         | MS Word 文書                         |

.app だけはディレクトリの拡張子

ファイルシステムの概念

13 / 18

## ファイルシステムの操作 (1)

### ディレクトリ操作

| 機能             | 対応する UNIX の API                      |
|----------------|--------------------------------------|
| ファイルの作成        | creat, open(... 0_CREAT ...) システムコール |
| ディレクトリの作成      | mkdir システムコール                        |
| ファイルの削除        | unlink システムコール                       |
| ディレクトリの削除      | rmdir システムコール                        |
| リンクの作成         | link, symlink システムコール                |
| リンクの削除         | unlink システムコール                       |
| 名前の変更 (移動)     | rename システムコール                       |
| ディレクトリエントリの読出し | opendir, readdir, closedir 関数        |

- ファイルの作成は creat システムコールでもできる。
- ディレクトリの読み出しはライブラリ関数で行う。
- rename システムコールはファイルの移動もできる。

ファイルシステムの概念

14 / 18

## ファイルシステムの操作 (2)

### ファイル操作

| 機能            | 対応する UNIX の API                     |
|---------------|-------------------------------------|
| ファイルを開く       | open システムコール                        |
| データを読む        | read システムコール                        |
| データを書く        | write システムコール                       |
| 読み書き位置を移動     | lseek システムコール                       |
| ファイルを閉じる      | close システムコール                       |
| ファイルの切り詰め     | truncate, open(... 0_TRUNC) システムコール |
| ファイルのプログラムを実行 | execve システムコール                      |
| ファイルの属性変更     | chmod, chown, chgrp, utimes システムコール |
| ファイル属性の読出し    | stat システムコール                        |

- open はファイルの保護属性をチェックする。
- 切り詰めは専用の truncate システムコールも使える。
- ファイルの属性の読み書きができるべき。

ファイルシステムの概念

15 / 18

## ファイルシステムの操作 (3)

### ファイルの共有とロック

```
#include <sys/file.h>
#define LOCK_SH 1 // 共有ロック
#define LOCK_EX 2 // 排他ロック
#define LOCK_NB 4 // ブロックしない
#define LOCK_UN 8 // ロック解除
int flock(int fd, int operation);
```

- LOCK\_SH: 共有ロック (shared lock)
- LOCK\_EX: 排他ロック (exclusive lock)
- LOCK\_NB: ロックできない時, ブロックしないでエラー
- open システムコールにもロックの機能がある。

### ワーキングディレクトリの変更

```
#include <unistd.h>
int chdir(const char *path);
```

ファイルシステムの概念

16 / 18

## ファイルシステムの健全性 (1)

### 一貫性チェック

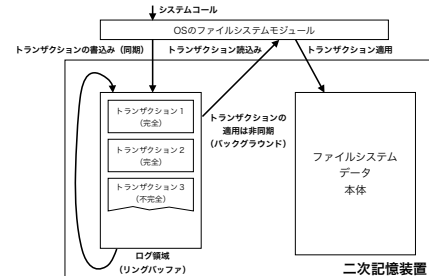
- 正常終了時にはファイルシステムにアンマウントの印をする。
- OS の起動時に印がなかったら一貫性チェックをする。
- メタデータの矛盾を解消するだけ。
- ファイルが消えたり, データが消えたりは修復できない。

ファイルシステムの概念

17 / 18

## ファイルシステムの健全性 (2)

### ジャーナリング・ファイルシステム



- データベースの WAL (Write Ahead Logging) のアイデア。
- NTFS, ext3, ext4, HFS+ 等が該当する。

ファイルシステムの概念

18 / 18

## オペレーティングシステム 第 1 5 章 FAT ファイルシステム

<https://github.com/tctsigemura/OSTextBook>

FAT ファイルシステム

1 / 8

## FAT ファイルシステム

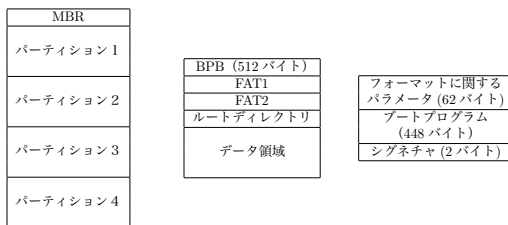
- 1980 年代から PC のファイルシステムとして利用されてきた。
- 仕様が公開されているので様々なところで利用されている。
  - USB メモリ, SD カード, その他メモリカード
  - Windows, Linux, macOS は FAT ファイルシステムをサポート
    - 同じ USB メモリを使用してデータ交換が可能
  - デジカメ, 音楽プレーヤ, デジタルテレビ, カーナビ...
    - デジカメの SD カードを PC で読める。
- ファイル名は半角 8 文字 + 3 文字 (例: IMG\_1234.JPG)
- FAT にはいくつかの種類がある。

| 種類    | 最大ボリュームサイズ | 最大ファイルサイズ | ファイル名  |
|-------|------------|-----------|--------|
| FAT12 | 32MiB      | 32MiB     | 8+3 文字 |
| FAT16 | 2GiB       | 2GiB      | 8+3 文字 |
| FAT32 | 2TiB       | 4GiB      | 8+3 文字 |
| exFAT | 16EiB      | 16EiB     | 255 文字 |

FAT ファイルシステム

2 / 8

## ボリューム内部の構造



- (ディスク装置全体, または, パーティション) = ボリューム
- パーティションの位置と大きさは MBR のテーブルで決まった。
- ボリュームの内部構造は FAT ファイルシステムのルールで決まる。
- ボリューム内部のコンポーネントの大きさなどは BPB から分かる。
- FAT は大切なデータなので 2 重化してある。
- データ領域はクラスタと呼ばれるブロック単位で扱う。

FAT ファイルシステム

3 / 8

## BPB (BIOS Parameter Block)

| パラメータ       | 意味               | 位置  | 長さ  | 値の例            |
|-------------|------------------|-----|-----|----------------|
| ジャンプ命令      | ジャンプ機械語命令        | 0   | 3   | 0xeb 0x3e 0x90 |
| セクタサイズ      | 1 セクタのバイト数       | 11  | 2   | 512 バイト        |
| クラスタサイズ     | 1 クラスタのセクタ数      | 13  | 1   | 64 セクタ         |
| 予約セクタ数      | 予約セクタ数 (BPB を含む) | 14  | 2   | 1 セクタ          |
| FAT 数       | FAT を何重に記録するか    | 16  | 1   | 2 個            |
| rootDir サイズ | ディレクトリエントリ数      | 17  | 2   | 512 エントリ       |
| 総セクタ数 16    | ボリュームのサイズ        | 19  | 2   | 0              |
| FAT サイズ     | FAT のセクタ数        | 22  | 2   | 245 セクタ        |
| 総セクタ数 32    | ボリュームのサイズ        | 32  | 4   | 3,999,681 セクタ  |
| ボリュームラベル    | ボリュームの名前         | 43  | 11  | "MICRODRIVE"   |
| ブートプログラム    | ブートプログラム         | 62  | 448 |                |
| シグネチャ       | フォーマット済みマーク      | 510 | 2   | 0x55 0xaa      |

(値の例はボリュームサイズ 2GiB, クラスタサイズ 32KiB, FAT16 の場合)  
 ("総セクタ数 16" で表現できない場合は「総セクタ数 32」を使用する)

- 論理フォーマット時に決めたパラメータを記録している。
- 値の例は, ボリューム=2GiB, クラスタ=32KiB, FAT16
- ブートプログラムは初代 IBM PC の機械語で作成してある。

FAT ファイルシステム

4 / 8

## ディレクトリエントリ

| Bytes | 8        | 3   | 1   | 10       | 2    | 2    | 2   | 4    |
|-------|----------|-----|-----|----------|------|------|-----|------|
|       | FileName | Ext | Atr | Reserved | Time | Date | Cls | Size |

- ルートディレクトリやディレクトリファイルに格納される。
- 前の BPB だとルートディレクトリに 512 個格納される。
- FileName: 8 文字以内のファイル名  
(0x00:以降未使用, 0xe5:削除, 0x05:本当は 0xe5)
- Ext: 3 文字以内の拡張子
- Atr: ファイルの属性  
(0x01: read-only, 0x02: hidden, 0x10: directory...)
- Time: ファイルの最終変更時刻
- Date: ファイルの最終変更日
- Cls: データが格納されている先頭クラスタの番号
- Size: ファイルの大きさ (バイト単位)

FAT ファイルシステム

5 / 8

## FAT (File Allocation Table)

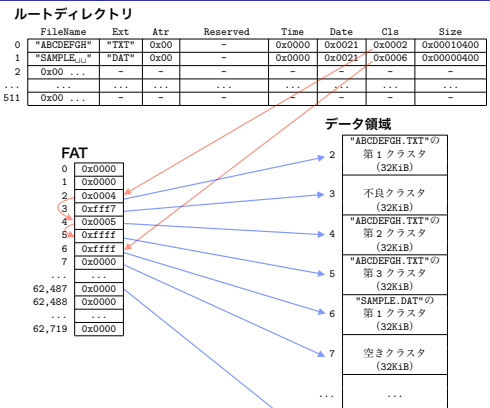
| 0   | 0x0000  | ← 使用不可       |                   |         |
|-----|---------|--------------|-------------------|---------|
| 1   | 0x0000  | ← 使用不可       |                   |         |
| 2   | 0x0004  | ← 次は第 4 クラスタ | 値                 | 意味      |
| 3   | 0xffff7 | ← 不良クラスタ     | 0x0000            | 未使用クラスタ |
| 4   | 0x0005  | ← 次は第 5 クラスタ | 0x0001            | 予約クラスタ  |
| 5   | 0xfffff | ← 終了クラスタ     | 0x0002 ~ 0xffff6  | 普通のクラスタ |
| 6   | 0x0000  | ← 未使用クラスタ    | 0xffff7           | 不良クラスタ  |
| ... | ...     |              | 0xffff8 ~ 0xfffff | 終了クラスタ  |

- クラスタをファイルに割り当てる表
- FAT エントリとクラスタが一対一対応
- FAT16 では FAT エントリが 16 ビット
- 0x0002~0xffff6 が普通のクラスタ番号
- クラスタチェーンを形成する。
- ディレクトリエントリの Cls がチェーンの先頭を指す。

FAT ファイルシステム

6 / 8

FAT ファイルシステムの全体像を示す例



FAT ファイルシステム

7 / 8

ディレクトリファイル

```
$ cd /Volumes/NO\ NAME
$ mkdir A
$ mkdir A/DIR
$ echo AAA > A/A.TXT
$ hexdump -C A
00000000 2e 20 20 20 20 20 20 20 20 20 20 30 00 aa 7d 98 |. 0..}.|
00000010 e4 4c e4 4c 00 00 a9 98 e4 4c 21 00 00 00 00 00 |.L.L....L!....|
00000020 2e 2e 20 20 20 20 20 20 20 20 20 10 00 aa 7d 98 |.. ..}.|
00000030 e4 4c e4 4c 00 00 7d 98 e4 4c 00 00 00 00 00 00 |.L.L.}.L.....|
00000040 44 49 52 20 20 20 20 20 20 20 20 10 00 31 a2 98 |DIR ..1..|
00000050 e4 4c e4 4c 00 00 a2 98 e4 4c 31 00 00 00 00 00 |.L.L....L!....|
00000060 41 20 20 20 20 20 20 20 54 58 54 20 00 13 a9 98 |A TXT ...|
00000070 e4 4c e4 4c 00 00 a9 98 e4 4c 40 00 04 00 00 00 |.L.L....L0....|
00000080 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 |.....|
*
00008000
```

- ルートディレクトリ以外のディレクトリは Attr=0x10 のファイル
- 上の実行例は macOS で FAT16 ファイルシステム上で実験したもの
- 最初の 2 エントリーは「.」と「..」を格納している。

FAT ファイルシステム

8 / 8

## オペレーティングシステム 第 16 章 UNIX ファイルシステム

<https://github.com/tctsigemura/OSTextBook>

UNIX ファイルシステム

1 / 10

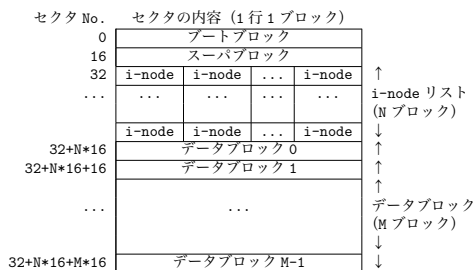
## UNIX ファイルシステム

- UFS (UNIX File System)
- 1979 年にリリースされた Version 7 UNIX のファイルシステムとそれを改良した多くのファイルシステムを UFS と呼ぶ。
- 第 14 章で「UNIX の場合」=「UFS の場合」だった。
  - 木構造のディレクトリシステム
  - ハードリンク、シンボリックリンク
  - ボリュームのマウント
  - ファイルの属性 (所有者: マルチユーザを意識)
  - ファイル操作のシステムコール, ...
- 多くのファイルシステムが UFS に準拠している。
  - Windows の NTFS
  - macOS の HFS+, APFS
  - Linux の ext3, ext4

UNIX ファイルシステム

2 / 10

## ボリューム内部の配置

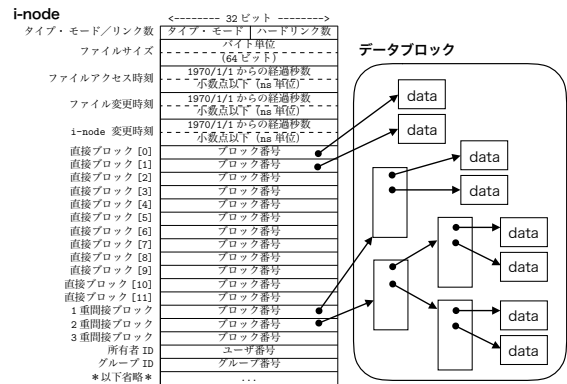


- セクタ 512B, ブロック 16 セクタ
- i-node がファイルを表現する (i-node = ファイル?)
- i-node のサイズ = 128B

UNIX ファイルシステム

3 / 10

## i-node (index node) (1)



UNIX ファイルシステム

4 / 10

## i-node (index node) (2)

- タイプ・モード
  - type/sst/rwxrwxrwx の 16 ビット
  - type: ファイル型 (通常, ディレクトリ, シンボリックリンク等)
  - ss(Set-uid, Set-gid): プログラムファイルを所有者の権限で実行
  - t: UNIX のバージョンによりさまざま
  - rwxrwxrwx: ファイルの保護モード
- リンク数: ハードリンク数
- ファイルサイズ: バイト単位
- 3つの時刻
  - 最終アクセス時刻
  - 変更時刻
  - i-node 変更時刻
 時刻は, 次の 2 つの 32 ビット整数で表現する。
  - 1970 年 1 月 1 日午前 0 時 (UTC) からの経過秒数
  - 秒の小数点以下をナノ秒単位で表現

UNIX ファイルシステム

5 / 10

## i-node (index node) (3)

- 直接ブロック
  - ファイル本体のデータを格納するデータブロックの番号
  - $8KiB \times 12 = 96KiB$  まで表現 (ブロックサイズ 8KiB 時)
  - ファイルの第 0 バイト ~ 第  $96Ki - 1$  バイトまでを管理
- 1 重間接ブロック
  - 直接ブロックだけでは表現できない大きなファイルに使用
  - 1 重間接ブロックを用いてデータブロックの番号を格納
  - 1 重間接ブロックに  $8KiB \div 4B = 2Ki$  個のブロック番号を格納
  - $2Ki$  個のデータブロックは  $8KiB \times 2Ki = 16MiB$  のデータを格納
  - 第  $96Ki$  バイト ~ 第  $96Ki + 16MiB - 1$  バイトの範囲を受け持つ
- 2 重間接ブロック
  - 1 重間接ブロックでも表現できない大きなファイルに使用
  - 2 重間接ブロックは 1 重間接ブロックの番号を格納
  - 2 重間接ブロックに  $8KiB \div 4B = 2Ki$  個のブロック番号を格納
  - $2Ki$  個の 1 重間接ブロックは  $16MiB \times 2Ki = 32GiB$  のデータを格納

UNIX ファイルシステム

6 / 10

## i-node (index node) (4)

- 3 重間接ブロック
  - 2 重間接ブロックを 2Ki 個管理できる
  - 2Ki 個の 2 重間接ブロックは  $32\text{GiB} \times 2\text{Ki} = 64\text{T/B}$  のデータを格納
- 所有者 ID: ファイル所有者のユーザ番号 (マルチユーザ)
- グループ ID: ファイルのグループ番号

**インデクス方式** i-node のような構造を使う方式のこと。  
高速なランダムアクセスができる。

**スパースファイル** 途中に穴の空いたファイルのこと。  
インデクス方式はスパースファイルを表現できる。

UNIX ファイルシステム

7 / 10

## ディレクトリファイル

- ファイルの一種である。→ i-node により表現
- ファイルの型 (type) がディレクトリを表す値のもの
- ディレクトリファイルはファイル名と i-node の対応表
- 対応表の 1 行がディレクトリエントリ

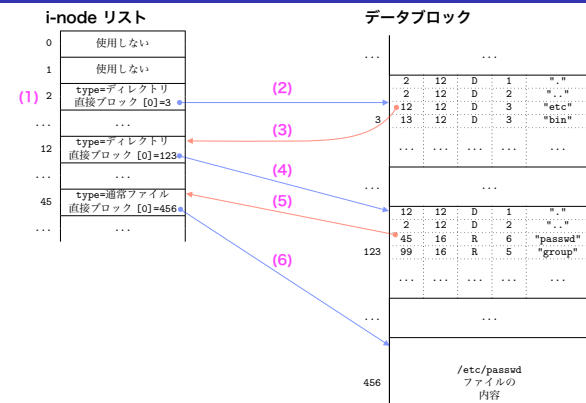
| 32bit                                                                        | 16bit | 8bit | 8bit  | $l_2$ バイト | 詰め物                               |
|------------------------------------------------------------------------------|-------|------|-------|-----------|-----------------------------------|
| i-node 番号                                                                    | $l_1$ | 型    | $l_2$ | ファイル名     | $\backslash 0 \dots \backslash 0$ |
| $\xleftarrow{\quad\quad\quad} l_1 \text{ バイト} \xrightarrow{\quad\quad\quad}$ |       |      |       |           |                                   |

- i-node 番号: ファイルの i-node 番号
- $l_1$ : ディレクトリエントリの大きさ (4 の倍数バイト)
- 型: ファイルの型 (抹消されたディレクトリエントリの表現)
- $l_2$ : ファイル名のバイト数
- ファイル名: 255 文字以内
- 詰め物: エントリが 4 の倍数バイトになるように

UNIX ファイルシステム

8 / 10

## パス名と i-node の対応付け (/etc/passwd を解析)



UNIX ファイルシステム

9 / 10

## パス名と i-node の対応付け (/etc/passwd を解析)

- (1) 絶対パスなのでルートディレクトリから探索を開始する。  
ルートディレクトリの i-node 番号は必ず 2 と決められている。
- (2) ルートディレクトリの i-node から、データブロック 3 にルートディレクトリの内容が格納されていることが分かる。
- (3) データブロック 3 の 3 番目のエントリから、etc が 12 番の i-node に対応すると分かる。
- (4) 12 番目の i-node から etc はディレクトリファイルであること、内容がデータブロック 123 に格納されていることが分かる。
- (5) データブロック 123 の 3 番目のエントリから、passwd が 45 番の i-node に対応することが分かる。
- (6) 45 番目の i-node から passwd は普通のファイルであること、ファイルの内容がデータブロック 456 に格納されていることが分かる。

UNIX ファイルシステム

10 / 10

## オペレーティングシステム 第 17 章 ZFS

<https://github.com/tctsigemura/OSTextBook>

ZFS

1 / 21

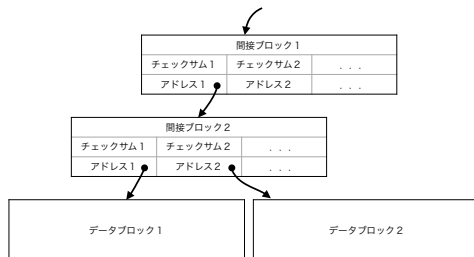
### ZFS の特徴

- 2005 年にサン・マイクロが OpenSolaris に実装して公開
- オープンソースで開発が続いている (FreeBSD 等でも使用可)
- 大きな主記憶、高速マルチプロセッサが前提 (8GiB, 64bit CPU)
- COW (Copy On Write) でデバイスに書き込む。  
(デバイスのブロックを上書きしないので前状態を維持)
- Uberblock を書き込みと同時に新しい状態になる。  
(Uberblock はファイルシステム管理データの根)
- チェックサムにより高い信頼性を確保 (次ページ)
- 一瞬でスナップショットやクローンを作成  
(COW の手法を使用し違いの出たブロックだけコピーし変更)
- ボリュームの代わりにストレージプールを使用 (後のページ)
- ファイルサイズ等の制約が事実上無い ( $Zetta = 2^{70}$ )  
(ファイルサイズ  $2^{64}B$ , ストレージプールサイズ  $2^{70}B$ )
- ミラーや RAID-Z 等が装備され信頼性・可用性が向上
- データ圧縮、重複除去機能があり (ディスク I/O を少なくする)

ZFS

2 / 21

### 全ブロックにわたるチェックサム

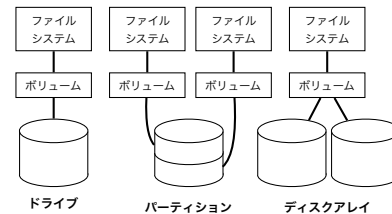


- ブロックポインタがチェックサムを持つ。
- メタデータだけでなく普通のデータにもチェックサムあり。
- チェックサムの不整合が見つかった場合、データの 2 重化 (ミラー) がされていれば、自動的にミラーからデータを修復する。

ZFS

3 / 21

### 従来の方式のボリューム

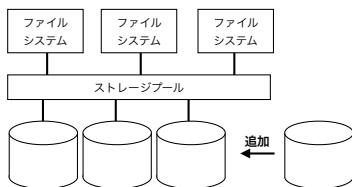


- ファイルシステムの初期化以前にボリュームを決定し、
- 後でサイズの変更などはできない。

ZFS

4 / 21

### ZFS のストレージプール

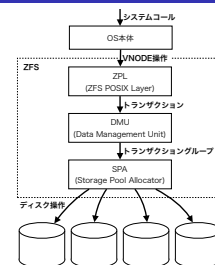


- ストレージプールは沢山のデバイスを収容する。
- ファイルシステムからの要求に応じてブロックを割り付ける。
- C 言語プログラムの `malloc()` や `free()` に似ている。
- ストレージプールに後でデバイスを追加することも可能。

ZFS

5 / 21

### ZFS のソフトウェア構成 (1)

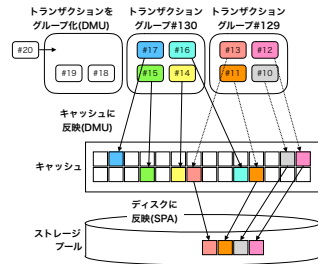


1. システムコールは、OS カーネル本体が VNODE 操作に変換する。
2. ZPL は VNODE 操作を ZFS のトランザクションに変換する。
3. DMU は複数トランザクションをトランザクショングループにする。
4. SPA は、DMU がトランザクショングループをキャッシュに書き込み終わると、キャッシュの内容をデバイスに反映させる。

ZFS

6 / 21

## ZFSのソフトウェア構成 (2)



- DMUは複数トランザクションをトランザクショングループにする。
- SPAは、DMUがトランザクショングループをキャッシュに書き込み終わると、キャッシュの内容をデバイスに反映させる。(バースト)

ZFS

7/21

## ストレージプールの構造 (概要)

ボリュームラベル

デバイス内部の構造

|                             |
|-----------------------------|
| V.L <sub>0</sub> (256KiB)   |
| V.L <sub>1</sub> (256KiB)   |
| ブートコード (3.5MiB)             |
| データ領域                       |
| V.L <sub>2</sub> (256KiB)   |
| V.L <sub>3</sub> (256KiB)   |
| V.L <sub>n</sub> : ボリュームラベル |

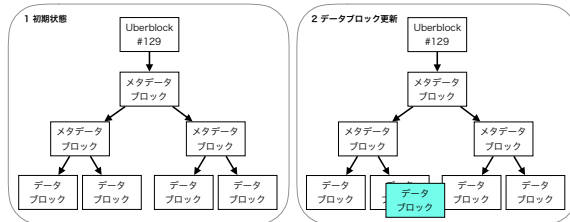
|                                |
|--------------------------------|
| デバイス情報など<br>名前/値ペア<br>(128KiB) |
| Uberblock[0] (1KiB)            |
| Uberblock[1] (1KiB)            |
| Uberblock[2] (1KiB)            |
| ...                            |
| Uberblock[127] (1KiB)          |

- デバイス (ディスク) の4箇所と同じボリュームラベルを書く。
- ボリュームラベルには128個のUberblockを格納できる。
- Uberblockはトランザクショングループ番号を含んでいる。
- Uberblockはトランザクショングループ番号を128で割った余りの位置に書く。

ZFS

8/21

## ストレージプールの更新 (1)

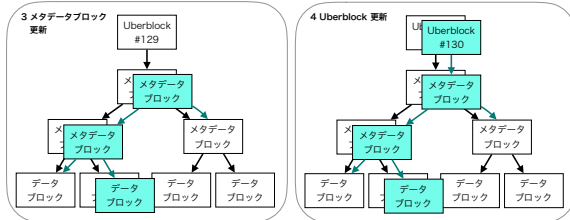


- Uberblockを起点とする木構造でブロックが記録されている。
- 変更するには、新しいブロックを確保し内容を書き込む (COW)。

ZFS

9/21

## ストレージプールの更新 (2)



- メタデータブロックもCOWで更新する。
- Uberblockを新しい領域に書き込む。  
(トランザクショングループ番号が最新のUberblockが有効)  
(古い世代のブロックは解放され、再利用される。)

ZFS

10/21

## ブロックポインタ

図中でブロックを指していた矢印を表現するデータ構造をブロックポインタと呼ぶ。ブロックポインタはデータ多重化のために最大3組のアドレスを記録できる。ブロックポインタの内容は以下の通り。

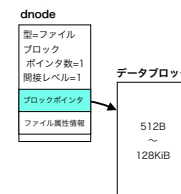
- サイズ: ブロックの大きさに関する情報
- チェックサム (64ビット): ブロックのチェックサム (最大3個)
- ブロックのアドレス: ブロックのストレージプール内での格納位置に関する情報 (最大3個) (デバイス、デバイス内アドレス)
- タイムスタンプ: ブロックを作成したトランザクショングループの番号 (ブロックが削除される時にスナップショットと比較)
- その他: チェックサム計算に使用するアルゴリズムの種類、データ圧縮に使用するアルゴリズムの種類、圧縮後のサイズなど...

ZFS

11/21

## Dnode (基本)

あらゆるオブジェクトを表現する512バイトのデータ構造である。USFのi-nodeに似ているが、ファイル、ディレクトリだけでなく、ファイルシステム、スナップショット、クローンなども表現する。

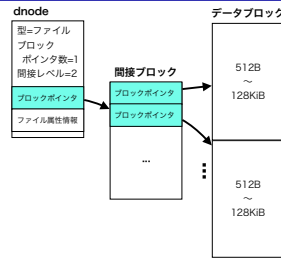


- dnodeは三つ以内のブロックポインタを格納することができる。
- dnodeは表現するオブジェクトに応じたデータを格納する領域を持っている。(この領域はブロックポインタと共用になっている)

ZFS

12/21

## Dnode (データが大きい場合)

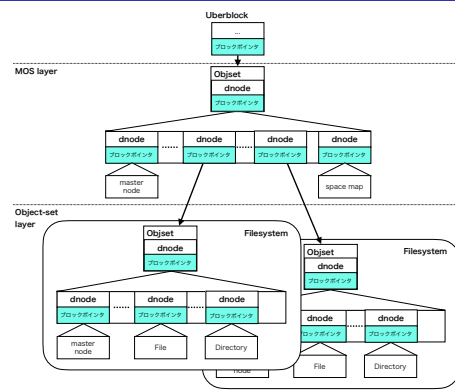


- データの大きさが 128KiB 以内の場合は直接参照 (前ページの図)
- 大きさが 128KiB を超える場合は間接ブロック (上の図))
- 128KiB の間接ブロックはブロックポインタを最大 1Ki 個格納できる.
- $128KiB \times 1Ki = 128MiB$  より大きなデータを表現する時は, 多重の間接ブロック (最大 6 レベル) を用いる. ( $2^{64}$  バイト以上)

ZFS

13 / 21

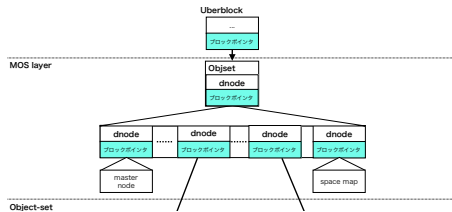
## ストレージプールの全体像



ZFS

14 / 21

## MOS (Meta Object Set) layer

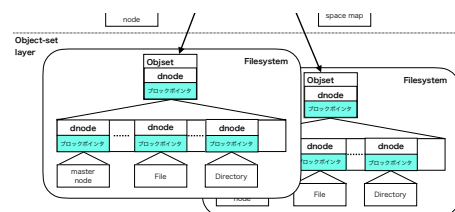


- Objectset は dnode の配列を管理する.
- 配列には, ストレージプール全体の管理に関する情報や, ファイルシステムやスナップショット等の一覧が格納される.
- master node はストレージプールのコンフィグ, プロパティなど.
- space map はストレージプール内のブロックの割付を管理する.

ZFS

15 / 21

## ストレージプールの全体像 (Object-set layer)

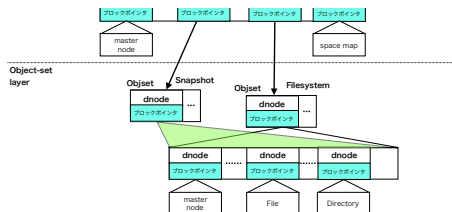


- ファイルシステムは Objectset の dnode 配列で表現する.
- dnode リストが UFS の i-node リストに相当する.
- master node は root ディレクトリの dnode 番号などの情報.
- 図は, 通常ファイルの例とディレクトリファイルの例

ZFS

16 / 21

## スナップショットの作成

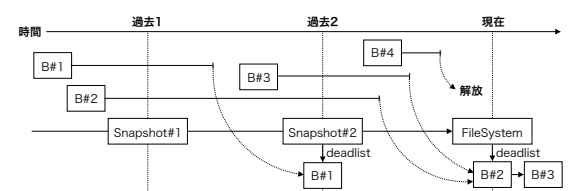


- ファイルシステムを表す Objectset のコピーを作る.
- MOS layer の dnode 配列に登録する.
- スナップショットは一瞬で作成できる.
- 変更がないデータブロックは共用する.
- 内容が書き換わると COW で必要最小限のブロックのコピーを作る.

ZFS

17 / 21

## スナップショットと deadlist

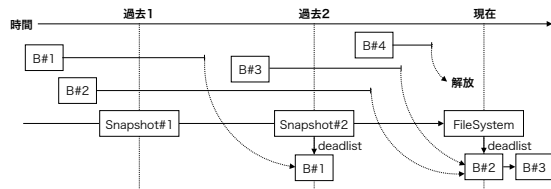


- ファイルシステム (Filesystem) は時間とともに変化する.
- ある瞬間の状態を保存したものがスナップショット.
- B#N と続く線はブロックとブロックの寿命を表す.
- Snapshot#N はスナップショットを表す.
- deadlist は使用されなくなったが解放できないブロックのリスト

ZFS

18 / 21

## ブロックの解放 (1)

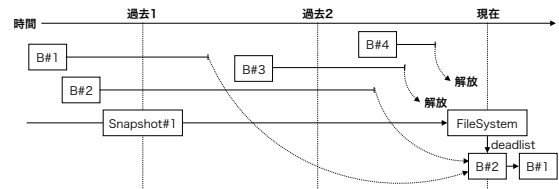


- B#1 は Snapshot#1 で使用されているので解放できない。
- B#2 と B#3 もスナップショットで使用されているので解放できない。
- B#4 はスナップショットで使用されていないので解放できる。
- 解放できないブロックは deadlist に格納される。

ZFS

19 / 21

## ブロックの解放 (2)



- Snapshot#2 を削除した状態。
- B#1 と B#2 は Snapshot#1 で使用されているので解放できない。
- B#3 は Snapshot#2 が削除されたので解放できる。
- B#1 と B#2 は FileSystem の deadlist に格納される。

ZFS

20 / 21

## まとめ

ZFS は大きな主記憶と高速な CPU を前提に設計されている。  
以下の特徴を持つ。

- COW (Copy On Write) を用い既存のブロックを上書きしない。
- COW のお蔭でシステムのクラッシュでも壊れない構造を実現。
- デバイス上の全てのデータについてチェックサムを持つ。
- スナップショットやクローンを一瞬で作ることができる。
- ボリュームの代わりにストレージプールを使用する。
- ストレージプールに後で新しいデバイスを追加可能。

ZFS

21 / 21